Disjunctive Methods for Warm-Starting Solution of Mixed Integer Linear Optimization Problems

by

Shannon Kelley

Presented to the Graduate and Research Committee

of Lehigh University

for the Degree of Doctor of Philosophy

in

Industrial and Systems Engineering

Lehigh University

May 2026

Dissertation Signature Sheet

Disjunctive Methods for Warm-Starting Solution of Mixed Integer Linear Optimization Problems

Shannon Kelley

Approved and recommended for acceptance as a dissertation in partial fulfillment of the requirements for the degree of Doctor of Philosophy.

_____
Date

Committee Members:

_____
Ted Ralphs, Committee Chair

_____
Aleksandr M. Kazachkov

_____
Lawrence Snyder

_____
Luis Zuluaga

Acknowledgements

Thank you to everyone who helped make this dissertation a reality! I've changed a lot over the course of the several years it took me to write this, but what stayed constant was the support of all of those around me, particularly of Ted and my dissertation committee.

Table of Contents

List of Tables

List of Figures

Abstract

Many applications require solving sequences of closely related mixed integer linear optimization problems (MILPs), including decomposition methods and online optimization. In such settings, solving each instance from scratch can discard structural information from earlier solves that remains useful for later ones. This dissertation studies how to warm start branch and cut by reusing *disjunctions*, the branching structures that partition the feasible region and underlie both search and cut generation.

Disjunctions can be reused in two natural ways. They can be reused *explicitly* by applying the branching decisions of a previously explored partial branch-and-bound tree to a new instance, thereby instantiating nodes that warm start the node pool. They can also be reused *implicitly* by transferring certificates that instantiate valid disjunctive inequalities for a new instance, thereby warm starting the cut pool. While explicit reuse of branch-and-bound trees has been studied previously, the machinery needed for implicit disjunction reuse through reusable certificates and parametrically valid disjunctive inequalities has not been developed in comparable form. As a result, there has also been no direct comparison between these two approaches when they are derived from the same underlying disjunction.

This dissertation develops a unified framework for both forms of reuse. We show how certificates of validity can be reused across related MILPs to generate parametrically valid disjunctive inequalities, develop strengthening procedures to preserve their quality under instance changes, and provide a controlled computational comparison between node warm starts and cut warm starts under shared disjunctions. Our results show that disjunction-based warm starting can be effective, but that the preferred reuse mechanism is regime-dependent: explicit node reuse is more effective in shorter solves, whereas implicit cut

reuse becomes more attractive in longer solves. Together, these contributions provide theoretical and computational foundations for structured disjunction reuse in branch-and-cut algorithms.

Chapter 1

Introduction

Many optimization methodologies require solving not one mixed integer linear optimization problem, but a sequence of closely related instances. Such sequences arise throughout mathematical optimization, including in decomposition methods, mixed integer nonlinear optimization, bilevel optimization, multiobjective optimization, and online optimization [16, 18, 41, 32, 60, 33, 53]. In these settings, solving each instance independently can discard structural information obtained on earlier instances that remains useful for later ones.

A central theme of this dissertation is that disjunctions provide such reusable structure. In branch and cut, disjunctions arise naturally from branching decisions and play two roles simultaneously: they partition the feasible region for search and they underlie the generation of strong valid inequalities. This makes them a natural candidate for warm starting sequences of related MILPs. Existing approaches reuse feasible solutions, branching policies, or even partial branch-and-bound trees [40, 52, 37, 29]. However, despite the importance of cutting planes to modern MILP performance [20], the cut generation process itself is rarely warm started in a way that reuses parameters from which the cuts were derived.

This omission is understandable for families of cuts that are inexpensive to regenerate. It is much more limiting, however, for cuts derived from large disjunctions. Such disjunctive cuts can substantially tighten relaxations, but they are often expensive to generate because each instance requires solving auxiliary LPs to recover both the inequality and the certificate proving its validity. Recent work on instance-dependent cut generation suggests

that parameterization itself can be valuable [19, 25, 31], while disjunctive cuts from large disjunctions are among the strongest relaxations available [55, 22, 12] but are often disabled in practice because of their computational cost [20, 1]. When a sequence of instances shares substantial structure, repeatedly paying this cost can be wasteful. The missing capability is a way to transfer the strength of disjunctive cuts across related instances without recomputing the entire generation process from scratch.

This dissertation develops that capability. We study two ways to reuse disjunctions across related MILPs. The first is explicit reuse, in which branching decisions from a previously explored partial branch-and-bound tree are transferred to a new instance to warm start the node pool. The second is implicit reuse, in which the certificate of validity of a disjunctive inequality is transferred and re-evaluated on new instance data to warm start the cut pool. The technical development that follows places both approaches in a common framework and makes it possible to compare them when they are derived from the same underlying disjunction.

We begin by recalling the mixed integer linear optimization setting in which these ideas are developed. A *mixed integer linear optimization problem* (MILP) is a mathematical optimization problem of the form

$$\min_{x \in \mathcal{S}} \quad cx \tag{MILP}$$

where $\mathcal{S} := \{x \in \mathcal{P} : x_j \in \mathbb{Z} \text{ for all } j \in \mathcal{I}\}$ and $\mathcal{P} := \{x \in \mathbb{R}^n : Ax \geq b\}$, where $(A, b, c) \in \mathcal{Y} := \mathbb{Q}^{q \times n} \times \mathbb{Q}^q \times \mathbb{Q}^{1 \times n}$, and $\mathcal{I} \subseteq [n] := \{1, \ldots, n\}$ is the set of indices of integer variables. We assume the constraints include nonnegative upper and lower bounds on all variables.

It is well-known that by relaxing the integrality requirements in MILP, we obtain the *linear optimization problem* (LP)

$$\min_{x \in \mathcal{P}} \quad cx \tag{LP}$$

referred to as the *LP relaxation* of (MILP). Solving the LP relaxation results in a lower bound (more generally referred to as a *dual* bound so as not to be specific to minimization problems). Generally speaking, the goal of all solution algorithms is to strengthen (increase) this initial bound iteratively until it converges to the optimal value of (MILP).

Terminology and Notation. Before moving on, we define some standard notation and terminology. For matrix A, we represent its i^{th} row as $A_{i\cdot}$ and its j^{th} column as $A_{\cdot j}$. Given $E \subseteq [q]$, the submatrix of A consisting of only the rows indexed by E is denoted by A_E. We refer to the set B of indices of any collection of n linearly independent constraints of $\mathcal{P}$, i.e., B is such that A_B is non-singular, as a *basis*. Associated with each basis is a solution obtained by solving an associated system of equations.

Definition 1.0.1. *The basic solution associated with the basis B is $x_B = (A_B)^{-1}b_B$. The basis B is* feasible *if $x_B \in \mathcal{P}$ and infeasible otherwise.*

This notion of basis is equivalent to the one that arises in the theory of LPs when the problem is usually assumed to be in standard form. It is easy to see that bases in standard form are in one-to-one correspondence to what we are here calling a basis.

1.1 Reformulation and Representability

To describe the set of solutions to a given optimization problem in the form (MILP) may require the introduction of additional variables that are not part of the "natural" formulation. To formalize this idea, we state here the standard definition of the notion of *MILP-representability*.

Definition 1.1.1. *A set $\mathcal{S} \subseteq \mathbb{R}^n$ is said to be* MILP-representable *if there exist $p, s \in \mathbb{Z}_+$, rational matrices $A' \in \mathbb{Q}^{m\times n}$, $B' \in \mathbb{Q}^{m\times p}$, $C' \in \mathbb{Q}^{m\times s}$, and vector $d' \in \mathbb{Q}^m$ such that*

$$\mathcal{S} = \text{proj}_u \left\{(u, v, w) \in \mathbb{R}^n \times \mathbb{R}^p \times \mathbb{Z}^s : A'u + B'v + C'w \geq d'\right\}.$$

In other words, $\mathcal{S}$ is the projection onto the u-space of the feasible region of a problem of the form (MILP), which we refer to as an *extended formulation*.

An MILP-representable set is generally described most compactly in the form (MILP), but this description does not expose the structure of the feasible set in a way that we can take advantage of computationally. MILP-representable sets can be described in two alternative ways that, while not compact, do form the basis for the two main classes of algorithms used to solve MILPs in practice. The reformulations we now describe provide a larger and

more explicit description of the feasible region and are central to the understanding of the methodology in this work. The first reformulation is one that exposes the disjunctive nature of the feasible region by expressing the feasible set as a union of polyhedra. The second reformulation exploits the fact that when optimizing with a linear objective function, there is always an optimal solution that is an extreme point of the convex hull of its feasible solutions, which is described by a set of valid inequalities (equivalently as the intersection of half-spaces). These reformulations are equivalent in a formal sense, yet lead to fundamentally different algorithmic approaches.

Disjunctive Reformulation. The disjunctive representation makes the inherently disjunctive nature of the feasible region explicit. The following theorem, due to Jeroslow and Lowe, shows that this projection-based definition is equivalent to a finite union-of-polyhedra representation [43, 17].

Theorem 1.1.2. *A set $\mathcal{S} \subseteq \mathbb{R}^n$ is MILP-representable if and only if there exist a finite collection of rational polytopes $\{P_i\}_{i=1}^{\ell} \subseteq \mathbb{R}^n$ and a finite set $R \subseteq \mathbb{Z}^n$ such that*

$$\mathcal{S} = \bigcup_{i=1}^{\ell} \left(P_i + \operatorname*{cone}_{\mathbb{Z}}(R) \right),$$

where

$$\operatorname{intcone}(R) := \left\{ \sum_{r \in R} \lambda_r r : \lambda_r \in \mathbb{Z}_+ \right\}.$$

This characterization shows that every MILP-representable set can be expressed as a finite union of rational polyhedra, each translated by vectors from a common finitely generated integer cone. Thus, linear inequalities describe the convex components, while the finite union encodes the combinatorial structure introduced by integrality.

Polyhedral Reformulation. A complementary reformulation is obtained by considering the convex hull of feasible solutions. Let $\mathcal{S}$ denote the feasible region of MILP. A fundamental result due to Meyer [51] formalizes the structure of this set.

Theorem 1.1.3. *If A and b are rational, then $\operatorname{conv}(\mathcal{S})$ is a rational polyhedron.*

Thus, every MILP admits an equivalent formulation as an LP over $\text{conv}(\mathcal{S})$. In practice, such a description is not available explicitly, but it can be approximated by iteratively strengthening the relaxation $\mathcal{P}$ with inequalities valid for $\mathcal{S}$ (see the formal definition in Section 1.2.2). In contrast to the disjunctive reformulation, which exposes disjunctive structure, the convex hull representation highlights the role of valid inequalities in producing a formulation that is amenable to linear optimization. As we will see, however, valid inequalities also arise naturally from disjunctive logic.

1.2 Algorithms

The two reformulations of the previous section lead directly to the two main algorithmic paradigms for solving MILPs, each of which reduces the problem of solving (MILP) to that of solving a sequence of LPs, as well as a third paradigm that is a hybrid of the two.

1.2.1 Branch and Bound

One of the earliest general methods for solving MILP instances is *branch and bound*. The method works by systematically *partitioning* a problem into smaller subproblems and deriving both primal and dual *bounds* on the optimal values of those subproblems to eliminate ones that cannot yield an improved solution [47]. From the viewpoint of Theorem 1.1.2, branching exposes the disjunctive structure of the feasible region explicitly by partitioning the feasible region through the use of *valid disjunctions*. We therefore introduce some definitions here that will form the basis for describing how branch-and-bound works.

Definition 1.2.1. *A* valid disjunction *for a set $\mathcal{S} \subseteq \mathbb{R}^n$ is a collection of sets $\{\mathcal{X}^t\}_{t\in T}$, where T is an index set, $\mathcal{X}^t \subset \mathbb{R}^n$ for $t \in T$, and such that $\mathcal{S} \subseteq \mathcal{X} := \bigcup_{t\in T} \mathcal{X}^t$.*

A valid disjunction represents a logical constraint that is known to be satisfied by the members of $\mathcal{S}$ and can be imposed in order to further restrict the feasible region of the LP relaxation and provide a better approximation of $\mathcal{S}$.

In this work, it will be natural to assume that valid disjunctions are defined by polyhedral sets, i.e., $\mathcal{X}^t := \left\{x \in \mathbb{R}^n : D^t x \geq D_0^t\right\}$, where $D^t \in \mathbb{Q}^{q_t \times n}$ and $D_0^t \in \mathbb{Q}^{q_t}$. In most cases, we

further assume $\mathcal{X}^t$ is a hyperrectangle obtained by restricting the bounds on one or more variables.

Imposing such a disjunctive constraint on a problem of the form (MILP) means intersecting each set in the collection defining the disjunction with the feasible region of the LP relaxation to obtain the collection of sets $\{\mathcal{Q}^t\}_{t\in T}$ defined by

$$\mathcal{Q}^t := \mathcal{P} \cap \mathcal{X}^t = \left\{x \in \mathbb{R}^n : A^t x \geq b^t\right\}, \quad \text{where} \quad A^t := \begin{bmatrix} A \\ D^t \end{bmatrix}, \quad b^t := \begin{bmatrix} b \\ D_0^t \end{bmatrix}$$

for $t \in T$. Clearly, we have that $\mathcal{S} \subseteq \bigcup_{t\in T} \mathcal{Q}^t$ and thus $\{\mathcal{Q}^t\}_{t\in T}$ can also be seen as a valid disjunction. We refer to the individual members of the collection as *disjunctive terms* and their convex hull as the *disjunctive hull*, denoted as $\mathcal{P}_D := \operatorname{cl}\operatorname{conv}\left(\bigcup_{t\in T} \mathcal{Q}^t\right)$. We denote the set of disjunctive terms that are non-empty as $T_{\text{feas}} := \{t \in T : \mathcal{Q}^t \neq \emptyset\}$. Finally, the LP relaxation associated with each disjunctive term $\mathcal{X}^t$ is

$$\min_{x\in\mathcal{Q}^t}\{cx\}. \tag{LP-t}$$

with optimal solutions defined $\bar{x}^t \in \arg\min_{x\in\mathcal{Q}^t}\{cx\}$.

The branch-and-bound algorithm can be seen as a method of iteratively constructing a valid disjunction that, when imposed on the initial LP relaxation, results in an improved bound that eventually converges to the optimal value of the original problem. A more formal definition of branch and bound relies on the addition of the following notation. Assuming minimization, we define the *primal bound* as $z^P \in \{z \in \mathbb{R} : z \geq cx \text{ for some } x \in \mathcal{S}\}$ and the *dual bound* as $z^D \in \{z \in \mathbb{R} : z \leq cx \text{ for all } x \in \mathcal{S}\}$. Let subproblem t of MILP be denoted as

$$\min_{x\in\mathbb{R}^n}\{cx : x \in \mathcal{S}^t\} \tag{MILP-t}$$

where $\mathcal{S}^t := \mathcal{S} \cap \mathcal{X}^t$.

We define S to be a collection of known feasible solutions and Q to be a priority queue of the indices of the active disjunction. We include a mapping of a disjunctive term to the one that created it with $P := \{t \mapsto t' \in \mathcal{X}\}_{t\in T}$. Additionally, we declare the three following

subroutines:

- **bound** (MILP-t), which returns $\bar{x}^t \in \{x \in \mathbb{R}^n : cx \leq cx' \text{ for all } x' \in \mathcal{S}^t\}$.
- **select** ($\bar{x}^t$, MILP-t), which returns $j \in \mathcal{I} \cap \left\{j \in [n] : \bar{x}^t_j - \left\lfloor \bar{x}^t_j \right\rfloor \neq 0\right\}$
- Q.**pop** (), which removes and returns the highest priority element t from Q

We stop short of definitions for the above subroutines as **bound** [39, 18, 28], **select** [48, 6, 50, 44], and **pop** [48, 50, 4] as each have a variety of possible implementations, which will be narrowed as needed in later sections. Lastly, we assume `num_terms` to represent the maximum number of terms in the active disjunction at which branch and bound terminates. We leverage the added definitions and declarations to give a more formal definition to branch and bound in Algorithm 1.

Branch and bound terminates with several key outputs. First is S, the collection of all integer-feasible solutions discovered during the search. Second is the *branch and bound tree*, $(\{\mathcal{X}^t\}_{t\in T}, P)$, which records the sequence of branching decisions made throughout the algorithm. Partial subtrees of this structure naturally define disjunctions, which we exploit to warm start subsequent MILP instances, as discussed throughout the remainder of this work. As long as a disjunction provided as input is valid, branch and bound never eliminates any integer-feasible point from $\{\mathcal{X}^t\}_{t\in T}$; consequently, disjunctions derived from partial branch and bound trees are guaranteed to be valid for the underlying MILP instance.

1.2.2 The Cutting Plane Method

An apparently very different approach for solving (MILP) is the *Cutting Plane Method* [39]. Instead of partitioning the feasible region, the cutting plane method iteratively strengthens the LP relaxation (LP) by adding dynamically generated valid inequalities in order to approximate $\text{conv}(\mathcal{S}^t)$, consistent with the reformulation of Theorem 1.1.3.

Definition 1.2.2. *A* valid inequality *for a set* $\mathcal{S} \subseteq \mathbb{R}^n$ *is a pair* $(\alpha, \beta) \in \mathbb{R}^n \times \mathbb{R}$ *such that* $\mathcal{S} \subseteq \left\{x \in \mathbb{R}^n : \alpha^\top x \geq \beta\right\}$.

For MILP, this approach alternates between (1) *solving* (an augmented version of) the LP relaxation of LP and (2) *separating* the solution $\bar{x}$ of the LP relaxation from $\text{conv}(\mathcal{S})$

Algorithm 1 `Branch and Bound`

Require: MILP, S (optional, default: $\emptyset$), $\{\mathcal{X}^t\}_{t\in T}$ (optional, default: $\{\mathbb{R}^n\}$), `num_terms` (optional, default: ∞)

$Q \leftarrow T$ ▷ indices of all unprocessed subproblems
$z^P \leftarrow \min\{\{\infty\} \cup \{cx : x \in S\}\}$ ▷ initialize primal and dual bounds
$z^D \leftarrow \min_{t\in T} \min_{x\in\mathcal{Q}^t} \{cx\}$
while $z^P > z^D$ and $|T| <$ `num_terms` **do**
  $t \leftarrow Q$.**pop** () ▷ process highest priority subproblem
  $\bar{x}^t \leftarrow$ **bound** (MILP-t)
  if $\mathcal{Q}^t \neq \emptyset$ and $c\bar{x}^t < z^P$ **then** ▷ MILP-t could contain solution improving z^P
    if $\bar{x}^t$ feasible for MILP **then** ▷ new best solution found
      $z^P \leftarrow c\bar{x}^t$
      $S \leftarrow S \cup \{\bar{x}^t\}$
    else ▷ integer infeasible
      $j \leftarrow$**select** ($\bar{x}^t$, MILP-t)
      $d \leftarrow |\mathcal{X}| + 1$ and $u \leftarrow |\mathcal{X}| + 2$ ▷ set indices for new terms
      $\mathcal{X}^d \leftarrow \mathcal{X}^t \cap \left\{x \in \mathbb{R}^n : x_j \leq \left\lfloor \bar{x}^t_j \right\rfloor\right\}$ ▷ branch down
      $\mathcal{X}^u \leftarrow \mathcal{X}^t \cap \left\{x \in \mathbb{R}^n : x_j \geq \left\lceil \bar{x}^t_j \right\rceil\right\}$ ▷ branch up
      $\mathcal{X} \leftarrow \mathcal{X} \cup \{\mathcal{X}^d, \mathcal{X}^u\}$ ▷ add children to set of all disjunctive terms
      $T \leftarrow (T \cup \{d, u\}) \setminus \{t\}$ ▷ update the active disjunction
      $P[d] \leftarrow t$ and $P[u] \leftarrow t$ ▷ record parent-child relationships
      $Q \leftarrow Q \cup \{d, u\}$ ▷ add new subproblems to queue
      $z^D \leftarrow \min\{c\bar{x}^t : t \in Q\}$ ▷ update dual bound
    end if
  end if
end while
return $S, \{\mathcal{X}^t\}_{t\in T}, P$

with valid inequalities. The goal is to iteratively tighten a polyhedral relaxation until it more closely approximates $\text{conv}(\mathcal{S})$. To make this explicit, let $\mathcal{H}$ be any polyhedron satisfying $\text{conv}(\mathcal{S}) \subseteq \mathcal{H} \subseteq \mathcal{P}$, and let $\Pi := \{(\alpha^i, \beta^i)\}_{i\in W^t} \subset \mathbb{R}^n \times \mathbb{R}$ denote a *cut pool*, or set of valid inequalities available to strengthen the current LP relaxation. We denote by **separate** $(\bar{x}, \mathcal{H})$ a function which returns either a collection

$$\{(\alpha^i, \beta^i) : {\alpha^i}^\top \bar{x} < \beta^i \text{ and } {\alpha^i}^\top x \geq \beta^i \text{ for all } x \in \mathcal{S}\}_{i\in W^t}$$

if $\bar{x} \in \mathcal{H} \setminus \text{conv}(\mathcal{S})$ or $\emptyset$ otherwise. Equivalently, **separate** can be sequentially formulated

as the optimization problem

$$\min \left\{ \alpha^\top \bar{x} - \beta : \alpha^\top x \geq \beta \ \forall x \in \mathcal{H},\ \beta = 1 \right\}. \tag{SEP}$$

If the optimal value of SEP is negative, then the corresponding inequality (α, β) separates $\bar{x}$ from $\mathcal{P}$ and can be added to $\mathcal{H}$ tightening it toward $\text{conv}(\mathcal{S})$.

Solving SEP directly is typically too expensive to be practical. Instead, implementations of **separate** usually target specific classes of valid inequalities (e.g. [27, 38, 9, 14]) and then select from the generated candidates those that most effectively tighten the relaxation [8, 15, 5, 61]. In this way, **separate** often returns improving cuts for the current LP relaxation solution $\bar{x}$ with substantially better computational complexity than solving SEP exactly. For this reason, we leave its implementation vague, but these preliminaries are sufficient at this point to formally define `Cutting Plane Method` in Algorithm 2.

Algorithm 2 `Cutting Plane Method`

Require: MILP, Π (optional, default: $\emptyset$), `max_iters` (optional, default: 100)

$\mathcal{P}_0 \leftarrow \mathcal{P}$ ▷ set initial LP relaxation
$i \leftarrow 0$
if $\Pi \neq \emptyset$ **then**
    $\mathcal{P}_0 \leftarrow \mathcal{P}_0 \bigcap \{x \in \mathbb{R}^n : \alpha^\top x \geq \beta \text{ for all } (\alpha, \beta) \in \Pi\}$ ▷ tighten LP relaxation
end if
while $i <$ `max_iters` **do**
    $\bar{x} \leftarrow \arg\min_{x \in \mathcal{P}_i} cx$ ▷ solve LP relaxation of MILP
    if $\bar{x}$ feasible for MILP **then**
        break
    end if
    $\Pi \leftarrow \Pi \cup$ **separate**$(\bar{x}, \text{conv}(\mathcal{S}))$ ▷ generate cutting planes
    $\mathcal{P}_{i+1} \leftarrow \mathcal{P} \bigcap \{x \in \mathbb{R}^n : \alpha^\top x \geq \beta \text{ for all } (\alpha, \beta) \in \Pi\}$ ▷ tighten LP relaxation
    $i \leftarrow i + 1$
end while
return $\bar{x}, \Pi$

Under certain conditions, the cutting plane method can be guaranteed to converge to the optimal solution to (MILP). In practice, however, it is mainly used to tighten the LP relaxations of the subproblems in branch-and-bound. This hybrid technique is known as *branch-and-cut* and is described next.

1.2.3 Branch and Cut

Throughout the remainder of this work, the term *branch and cut* refers to branch and bound with **bound** employing the cutting plane method. Branching alone can require a very large search tree, while cutting planes alone often converge slowly or stall; in practice, their combination is far more effective and underlies modern general purpose MILP solvers [8] to which this work contributes.

To describe branch and cut formally, we now reintroduce cut pools at the node level. This is accomplished by updating **bound** to take the form of the cutting plane method, which in this context is overloaded: rather than solving a node LP relaxation all the way to MILP optimality, it is used only until additional cut generation becomes less effective, from a solve-time standpoint, than branching and restarting the process on a new node [8]. Branching performance can be further improved by allowing all successors of a node to inherit the cuts generated for that node, thereby seeding cut generation and improving the tightening of subsequent LP relaxations. Accordingly, we define $\Pi \coloneqq \{\Pi_t\}_{t \in T}$, where $\Pi_t \coloneqq \{(\alpha^i, \beta^i)\}_{i \in W^t} \subset \mathbb{R}^n \times \mathbb{R}$ denotes the cut pool associated with node t, and we generalize **bound** and **select** to return the node's current cut pool so that it can be passed to its descendants. With this added node-level state, branch and cut can be implemented as in Algorithm 3.

1.3 Certifying Optimality

At a high level, solving an MILP via branch and cut can be interpreted as the iterative construction of increasingly tighter lower (dual) and upper (primal) bounds that converge to the optimal objective value. Equivalently, the algorithm builds progressively sharper approximations of an underlying *value function*.

1.3.1 The Value Function

The classical value function is a function of the right-hand side, but because we consider more general parameterizations in this work, we also consider a generalized notion of the value function, as defined next.

Algorithm 3 `Branch and Cut`

Require: MILP, S (optional, default: $\emptyset$), $\{\mathcal{X}^t\}_{t\in T}$ (optional, default: $\{\mathbb{R}^n\}$), Π (optional, default: $\emptyset$), `num_terms` (optional, default: ∞)

$Q \leftarrow T$ ▷ indices of all unprocessed subproblems
$z^P \leftarrow \min\{\{\infty\} \cup \{cx : x \in S\}\}$ ▷ initialize primal and dual bounds
$z^D \leftarrow \min_{t\in T} \min_{x\in\mathcal{Q}^t} \{cx\}$
while $z^P > z^D$ and $|T| <$ `num_terms` **do**
  $t \leftarrow Q$.**pop** () ▷ process highest priority subproblem
  $(\bar{x}^t, \Pi_t) \leftarrow$ **bound** (MILP-t, Π_t)
  if $\mathcal{Q}^t \neq \emptyset$ and $c\bar{x}^t < z^P$ **then** ▷ MILP-t could contain solution improving z^P
    if $\bar{x}^t$ feasible for MILP **then** ▷ new best solution found
      $z^P \leftarrow c\bar{x}^t$
      $S \leftarrow S \cup \{\bar{x}^t\}$
    else ▷ integer infeasible
      $(j, \Pi_t) \leftarrow$**select** ($\bar{x}^t$, MILP-t, Π_t)
      $d \leftarrow |\mathcal{X}| + 1$ and $u \leftarrow |\mathcal{X}| + 2$ ▷ set indices for new terms
      $\mathcal{X}^d \leftarrow \mathcal{X}^t \cap \left\{x \in \mathbb{R}^n : x_j \leq \left\lfloor \bar{x}^t_j \right\rfloor\right\}$ ▷ branch down
      $\mathcal{X}^u \leftarrow \mathcal{X}^t \cap \left\{x \in \mathbb{R}^n : x_j \geq \left\lceil \bar{x}^t_j \right\rceil\right\}$ ▷ branch up
      $\mathcal{X} \leftarrow \mathcal{X} \cup \{\mathcal{X}^d, \mathcal{X}^u\}$ ▷ add children to set of all disjunctive terms
      $T \leftarrow (T \cup \{d, u\}) \setminus \{t\}$ ▷ update the active disjunction
      $P[d] \leftarrow t$ and $P[u] \leftarrow t$ ▷ record parent-child relationships
      $Q \leftarrow Q \cup \{d, u\}$ ▷ add new subproblems to queue
      $z^D \leftarrow \min\{c\bar{x}^t : t \in Q\}$ ▷ update dual bound
      $\Pi_d \leftarrow \Pi_t$ and $\Pi_u \leftarrow \Pi_t$ ▷ propagate cut pool
    end if
  end if
end while
return $S, \{\mathcal{X}^t\}_{t\in T}, P$

Definition 1.3.1. *The* value function *is defined by*

$$\phi(A, b, c) := \inf\{cx : x \in \mathbb{R}^n,\ Ax \geq b,\ x_j \in \mathbb{Z} \text{ for all } j \in \mathcal{I}\}$$

for $(A, b, c) \in \mathcal{Y}$.

In linear optimization, value functions can be characterized through dual solutions and sensitivity analysis. For MILPs, the value function may still be piecewise linear under certain restricted parameterizations, but it generally loses convexity and is far harder to evaluate or represent explicitly [40]. As a result, MILP algorithms operate indirectly by constructing bounds that approximate ϕ from below and above.

1.3.2 Primal and Dual Bounding Functions

We formalize this viewpoint by introducing bounding functions parameterized by structural objects θ.

Definition 1.3.2. *A function F_θ is a* dual bounding function *if $F_\theta(A, b, c) \leq \phi(A, b, c)$ for all $(A, b, c) \in \mathcal{Y}$.*

A function G_θ is a primal bounding function *if $\phi(A, b, c) \leq G_\theta(A, b, c)$ for all $(A, b, c) \in \mathcal{Y}$ with $G_\theta(A, b, c) < +\infty$.*

Dual functions can be constructed in a variety of ways, including using valid disjunctions, as in

$$F_{\{\mathcal{X}^t\}_{t\in T}}(A, b, c) \coloneqq \min_{t\in T} \min\left\{cx : x \in \mathbb{R}^n,\ Ax \geq b,\ D^t x \geq D_0^t\right\},$$

or sets of valid disjunctive inequalities (denoted $\Pi = \{(\alpha^w, \beta^w)\}_{w\in W}$), as in

$$F_\Pi(A, b, c) \coloneqq \min\{cx : x \in \mathbb{R}^n,\ Ax \geq b,\ {\alpha^w}^\top x \geq \beta^w\ \forall w \in W\}.$$

The first class of functions arises from the optimal values of the disjunctive term relaxations, while the second is induced by a strengthened LP relaxation. These two examples foreshadow the two reusable structures studied later: disjunctions themselves and the valid inequalities certified by them.

As with dual functions, primal functions can be constructed in a variety of ways, including from a given collection of feasible solutions (denoted $x \in \mathbb{R}^n$),

$$G_x(A, b, c) \coloneqq \begin{cases} cx, & \text{if } x \in \mathbb{R}^n,\ Ax \geq b,\ x_j \in \mathbb{Z} \text{ for all } j \in \mathcal{I}, \\ +\infty, & \text{otherwise,} \end{cases}$$

and by given disjunctions, which may induce feasible candidates through optimal LP term solutions. Let

$$X_t^*(A, b, c) \coloneqq \arg\min\left\{cy : y \in \mathbb{R}^n,\ Ay \geq b,\ D^t y \geq D_0^t\right\}.$$

Then the primal bound induced by $\{\mathcal{X}^t\}_{t\in T}$ is

$$G_{\{\mathcal{X}^t\}_{t\in T}}(A,b,c) \coloneqq \min\left\{cy : t \in T,\ y \in X_t^*(A,b,c),\ y_j \in \mathbb{Z} \text{ for all } j \in \mathcal{I}\right\},$$

with the convention that the minimum over an empty set equals $+\infty$.

An MILP algorithm may therefore be viewed as constructing dual and primal bounding functions indexed by structural objects such as disjunctions, cut pools, and feasible solutions. A solve progresses by refining these objects, thereby tightening the associated lower and upper bounds on $\phi(A,b,c)$ until they meet.

Definition 1.3.3. *Let $(A,b,c) \in \mathcal{Y}$ and let F_{θ_F} be a dual bounding function and G_{θ_G} be a primal bounding function. We say that* strong duality holds at (A,b,c) *for the pair $(F_{\theta_F}, G_{\theta_G})$ if $F_{\theta_F}(A,b,c) = \phi(A,b,c) = G_{\theta_G}(A,b,c)$. In this case, we call (θ_F,θ_G) a* certificate of optimality at (A,b,c).

For example, given a valid disjunction $\{\mathcal{X}^t\}_{t\in T}$ and a feasible solution $\bar{x} \in \mathbb{R}^n$ satisfying $A\bar{x} \geq b$ and $\bar{x}_j \in \mathbb{Z}$ for all $j \in \mathcal{I}$, the pair $(\{\mathcal{X}^t\}_{t\in T}, \bar{x})$ is a certificate of optimality at (A,b,c) whenever $F_{\{\mathcal{X}^t\}_{t\in T}}(A,b,c) = G_{\bar{x}}(A,b,c)$.

In linear optimization, strong duality is certified by a dual feasible solution whose objective value matches that of a primal feasible solution. Here, the dual certificate is not a vector of multipliers but a structural object, such as a disjunction or a collection of valid inequalities, that induces a dual bounding function.

This interpretation clarifies the role of branch and cut. For fixed data (A,b,c), the algorithm maintains a primal bounding function through the incumbent solution (Algorithm 3, Step 9) and a dual bounding function through LP relaxations strengthened by cutting planes at active nodes (Algorithm 3, Step 20). The leaves of a partially explored branch-and-bound tree induce a disjunction, and the optimal values of the corresponding term relaxations yield the associated composite lower bound.

1.3.3 Certifying Validity of Inequalities

Dual bounding functions are the key link between disjunctions and disjunctive cuts. A disjunction induces a dual bounding function through the optimal values of its term re-

laxations, and that same dual viewpoint tells us how to certify inequalities valid for the corresponding disjunctive hull. This subsection makes that connection explicit so that later node and cut warm starts can be understood as two uses of the same underlying certificate machinery.

The first step is the classical disjunctive cut principle: validity of an inequality for a disjunctive hull is equivalent to the existence of nonnegative termwise multipliers. The following theorem specializes Balas [11, Theorem 3.1] to our setting.

Theorem 1.3.4 (Disjunctive cut principle)**.** *Let $\{\mathcal{X}^t\}_{t\in T}$ be a valid disjunction for MILP. Assume $x \geq 0$ is among the constraints defining each term polyhedron $\mathcal{Q}^t$. Then (α, β) is valid for $\mathcal{P}_D$ if and only if there exists a collection $\{v^t\}_{t\in T}$ such that*

$$\begin{aligned} \alpha_j &\geq \max_{t\in T}\{v^t A^t_{\cdot j}\} && \forall j \in [n], \\ \beta &\leq \min_{t\in T}\{v^t b^t\}, && \\ v^t &\in \mathbb{R}^{1\times(q+q_t)}_{\geq 0} && \forall t \in T. \end{aligned} \tag{FL}$$

Definition 1.3.5. *Let $\{\mathcal{X}^t\}_{t\in T}$ be a valid disjunction for MILP. We say that $\{v^t\}_{t\in T}$ is a* (Farkas) certificate of validity *for (α, β) relative to $\mathcal{P}_D$ if $\{v^t\}_{t\in T}$ and (α, β) satisfy* (FL)*.*

In other words, a certificate of validity is the object that explains *why* the inequality is valid. From this perspective, generating disjunctive cuts amounts to finding coefficients (α, β) together with multipliers $\{v^t\}_{t\in T}$ that satisfy (FL). We refer to these certificates of validity as *Farkas certificates*, and sometimes as *Farkas multipliers*, because Farkas' lemma underpins the proof of Theorem 1.3.4 [34].

The same principle also explains why a branch-and-bound lower bound is itself a valid inequality: the leaf-node LP dual solutions provide exactly the required multipliers.

Corollary 1.3.6 (Objective bounds as disjunctive inequalities)**.** *Let $\{\mathcal{X}^t\}_{t\in T}$ be the disjunction induced by the leaves of a partial branch-and-bound tree for MILP. For each $t \in T$, let u^t be an optimal solution to the dual of LP-t, and define*

$$LB := F_{\{\mathcal{X}^t\}_{t\in T}}((A, b, c)).$$

Then $(c^\top, LB)$ is valid for $\mathcal{P}_D$, and hence $c^\top x \geq LB$ is valid for the mixed integer feasible region of MILP. Moreover, $\{u^t\}_{t\in T}$ is a certificate of validity for this objective inequality.

Proof. By the definition of the disjunction-based dual bounding function from Section 1.3.2,

$$F_{\{\mathcal{X}^t\}_{t\in T}}(A, b, c) = \min_{t\in T} \min\left\{cx : x \in \mathbb{R}^n,\ Ax \geq b,\ D^t x \geq D_0^t\right\}.$$

For each $t \in T$, strong duality for LP-t gives

$$\min\left\{cx : x \in \mathbb{R}^n,\ Ax \geq b,\ D^t x \geq D_0^t\right\} = u^t b^t.$$

Hence,

$$LB = F_{\{\mathcal{X}^t\}_{t\in T}}(A, b, c) = \min_{t\in T}\{u^t b^t\}.$$

Now, for each $t \in T$, dual feasibility of u^t for LP-t implies $c_j \geq u^t A^t_{\cdot j}$ for all $j \in [n]$, while the identity above gives $LB \leq u^t b^t$. Applying Theorem 1.3.4 with $\alpha = c$ and $\beta = LB$ yields the claim. □

Corollary 1.3.6 shows that an objective cut, one separating relaxation solutions with objective value below the dual bound, together with its certificate of validity, can be obtained directly from the leaf dual solutions. This is the bridge from node-based dual bounds to cut-based certificates.

Generalizing that viewpoint, if we prescribe the left-hand side of a desired cut, then the same term relaxations can be solved with that prescribed vector as the objective to obtain a valid right-hand side.

Corollary 1.3.7 (Modified-objective bounds)**.** *Let $\alpha \in \mathbb{R}^n$. For each $t \in T$, let v^t be an optimal solution to the dual of the term LP obtained from LP-t by replacing objective c with α, and define*

$$\beta := \min_{t\in T}\{v^t b^t\}.$$

Then (α, β) is valid for $\mathcal{P}_D$.

Proof. Apply Corollary 1.3.6 after replacing the objective c with α. Since v^t is dual optimal

for the corresponding term LP for each $t \in T$, that result gives

$$F_{\{\mathcal{X}^t\}_{t\in T}}((A, b, \alpha)) = \min_{t\in T}\{v^t b^t\} = \beta,$$

and therefore implies that (α, β) is valid for $\mathcal{P}_D$. □

This converse viewpoint is the one most relevant for disjunctive cut generation. Once the left-hand side α is prescribed, the same disjunction-induced term relaxations can be solved with objective α, producing a right-hand-side value β and a certificate of validity for (α, β). In this sense, a Farkas certificate for $\alpha^\top x \geq \beta$ is exactly the certificate that would be produced by the same branch-and-bound tree if the objective vector were α instead of c.

For fixed $\alpha \in \mathbb{R}^n$, the resulting map from problem data to the disjunction-based lower bound is a dual bounding function for the MILP with objective $\alpha^\top x$. This connects the construction to the classical subadditive framework, in which applying a subadditive function to the columns of the constraint matrix and to the right-hand side yields a valid inequality [42, 56]. Our construction uses the same functional template, but validity is derived from the disjunctive certificate rather than from subadditivity itself.

In summary, a disjunction used as parameters of a dual bounding function yields a valid inequality for the disjunctive hull together with a reusable certificate explaining why that inequality is valid. This is the mechanism we later reuse for implicit warm starts.

1.4 Parameterization

The previous section interprets validity through dual bounding functions evaluated under different objective vectors. To reuse such certificates beyond a single MILP, we now introduce the parametric family from which later instances are drawn. Throughout this dissertation, we allow the objective coefficients, constraint matrix coefficients, and right-hand sides to vary, but keep the dimension and integrality structure fixed. In particular, while our techniques apply as long as the set of integer-constrained variables is consistent across instances, we assume for simplicity that all admissible instances are defined over the same

variables and number of constraints. Let

$$\Omega := \Omega_A \times \Omega_b \times \Omega_c \subseteq \mathcal{Y}.$$

Thus, each $(A, b, c) \in \Omega$ specifies admissible matrix, right-hand-side, and objective data for a MILP with the fixed dimension and integer-variable set introduced above. This fixed structure lets a disjunction or certificate define inequalities with compatible dimensions across the family.

For $(A, b, c) \in \Omega$, define

$$\mathcal{P}(A, b) := \{x \in \mathbb{R}^n : Ax \geq b\}, \qquad \mathcal{S}(A, b) := \{x \in \mathcal{P}(A, b) : x_j \in \mathbb{Z} \text{ for all } j \in \mathcal{I}\}.$$

For a disjunction $\{\mathcal{X}^t\}_{t \in T}$, also define

$$\mathcal{Q}^t(A, b) := \mathcal{P}(A, b) \cap \mathcal{X}^t, \qquad \mathcal{P}_D(A, b) := \operatorname{cl}\operatorname{conv}\left(\bigcup_{t \in T} \mathcal{Q}^t(A, b)\right).$$

In this dissertation, the relevant reusable structural objects are disjunctions and valid inequalities.

1.4.1 Parametrically Valid Disjunctions

A disjunction is useful for warm starting only if it remains valid beyond the instance from which it was extracted. This motivates the following notion.

Definition 1.4.1. *A disjunction $\mathcal{X} = \{\mathcal{X}^t\}_{t \in T}$ is* parametrically valid *over Ω if $\mathcal{S}(A, b) \subseteq \mathcal{X}$ for all $(A, b) \in \Omega_A \times \Omega_b$.*

Equivalently, every feasible solution of every admissible feasible region satisfies at least one term of the disjunction. In that case, the same logical partition can be applied throughout the family, and the corresponding term relaxations $\mathcal{Q}^t(A, b)$ remain well defined for every $(A, b) \in \Omega_A \times \Omega_b$.

In this work, we assume our disjunctions arise from branching only on integer variables. Such disjunctions are lattice-free and therefore inherit a natural form of parametric validity:

because the index set of integer-constrained variables is fixed across the family, the branching statements defining the disjunction continue to cover the integer-feasible region of every instance. This is the mechanism that allows a partial branch-and-bound tree generated for one instance to be meaningfully reinterpreted on another.

1.4.2 Parametrically Valid Disjunctive Inequalities

For disjunctive inequalities, parametric validity is most naturally expressed in terms of certificates. Rather than asking that a single fixed inequality remain valid across the family, we allow the coefficients of the inequality to vary with the feasible-region data, provided they all share a common certificate.

Definition 1.4.2. *A family of disjunctive inequalities* $\{(\alpha(A,b),\beta(A,b))\}_{(A,b)\in\Omega_A\times\Omega_b}$ *is* parametrically valid *over* Ω *if there exists* $\{v^t\}_{t\in T}$ *such that* $\{v^t\}_{t\in T}$ *is a certificate of validity for* $(\alpha(A,b),\beta(A,b))$ *relative to* $\mathcal{P}_D(A,b)$ *for all* $(A,b)\in\Omega_A\times\Omega_b$.

We refer to such parametrically valid families of disjunctive inequalities as *parametric disjunctive inequalities* (PDIs). In other words, the certificate is the object that transfers across admissible feasible-region data, while the instantiated inequality coefficients are obtained by evaluating that certificate against the data of the current instance. This viewpoint is central to our treatment of disjunctive cuts: the multipliers encode the logical and geometric reason the cut is valid, and this reason can persist even as the matrix coefficients and right-hand sides vary.

1.5 Warm Starting

We now distinguish the parametric family Ω from the realized sequence of instances that must actually be solved. Let $\mathcal{K}$ index a finite sequence of MILP instances, not necessarily known in advance, with data $(A^k,b^k,c^k)\in\Omega$ for every $k\in\mathcal{K}$. We write MILP-k for the realized instance with data (A^k,b^k,c^k) and use the superscript k to denote evaluation of previously defined objects at that data:

$$\mathcal{P}^k := \mathcal{P}(A^k,b^k),\quad \mathcal{S}^k := \mathcal{S}(A^k,b^k),\quad \mathcal{Q}^{kt} := \mathcal{Q}^t(A^k,b^k),\quad \mathcal{P}_D^k := \mathcal{P}_D(A^k,b^k).$$

Equivalently, all objects depending on A, b, and/or c may be indexed by k when evaluated at the data of instance MILP-k; for example, $A^{kt} := \begin{bmatrix} A^k \\ D^t \end{bmatrix}$ and $b^{kt} := \begin{bmatrix} b^k \\ D_0^t \end{bmatrix}$. With this notation,

$$\min_{x \in \mathcal{S}^k} \quad c^k x \tag{MILP-k}$$

and the LP relaxation of term t is

$$\min_{x \in \mathcal{Q}^{kt}} \quad c^k x. \tag{LP-kt}$$

The preceding sections identify the objects that remain valid across the parametric family: parametrically valid disjunctions and PDIs. Warm starting is the algorithmic use of these objects on the realized sequence. From the bounding-function viewpoint, a warm start consists of evaluating on a new instance a dual bounding function that was constructed from structural information obtained on a previously solved instance, and then initializing branch and cut with the corresponding reusable artifacts.

In this dissertation, we study two complementary forms of disjunctive warm starting. The first reuses a disjunction directly and is therefore *explicit*. The second reuses inequalities certified by that disjunction and is therefore *implicit*. In both cases, the first dual bound for the new instance is obtained by evaluating a previously constructed dual bounding function at the current problem data; the remainder of the solve then refines that initialized bound in the usual way.

1.5.1 Explicit Reuse: Node Warm Starts

Let $\{\mathcal{X}^t\}_{t \in T}$ be a parametrically valid disjunction. Because $\{\mathcal{X}^t\}_{t \in T}$ remains valid across the family, it can be applied directly at the current datum (A, b, c). This induces the collection of term relaxations $\{\mathcal{Q}^t(A, b)\}_{t \in T}$ and therefore the disjunction-based dual bounding function

$$F_{\{\mathcal{X}^t\}_{t \in T}}(A, b, c) := \min_{t \in T} \min_{x \in \mathcal{Q}^t(A,b)} cx.$$

Evaluating this function gives an immediate valid lower bound at datum (A, b, c). This is the *node warm start* or *explicit reuse* viewpoint.

Algorithmically, the same reused branching decisions that define $\{\mathcal{X}^t\}_{t\in T}$ also define a partial branch-and-bound tree for the current datum. The leaves of that induced tree correspond to the term relaxations $\mathcal{Q}^t(A,b)$, so they can be used to instantiate the initial node pool for the new solve. Thus, explicit reuse means evaluating the disjunction-induced dual bounding function at the current datum and initializing branch and cut with the nodes created by the corresponding disjunctive terms.

1.5.2 Implicit Reuse: Cut Warm Starts

The second approach reuses not the disjunction itself as a collection of nodes, but the inequalities it certifies. Let $\{(\alpha^w(A,b),\beta^w(A,b))\}_{w\in W}$ be a collection of parametrically valid families of inequalities induced by shared certificates. Evaluating these certificates at the current datum yields the concrete cut pool $\Pi(A,b) \coloneqq \{(\alpha^w(A,b),\beta^w(A,b))\}_{w\in W}$, valid for the current problem. The associated dual bounding function is

$$F_{\Pi(A,b)}(A,b,c) \coloneqq \min\{cx : x \in \mathcal{P}(A,b),\ \alpha^w(A,b)^\top x \geq \beta^w(A,b)\ \ \forall w \in W\}.$$

As before, this evaluation immediately gives a valid lower bound at datum (A,b,c). This is the *cut warm start* or *implicit reuse* viewpoint.

The distinction is that validity is now carried by certificates rather than by an explicit node pool. Each shared certificate guarantees that its instantiated inequality is valid at the current datum, so the resulting strengthened LP relaxation defines a valid dual bound. Algorithmically, implicit reuse therefore initializes branch and cut with a pool of cuts derived from previously generated structural information, rather than with a pool of inherited nodes.

These two warm starts arise from the same geometric source but emphasize different algorithmic representations. The union-of-polyhedra viewpoint leads naturally from branch and bound to disjunction-induced dual bounds, then to parametrically valid disjunctions, and finally to explicit disjunctive warm starts. The valid-inequality viewpoint leads from cutting planes to cut-pool dual bounds, then to PDIs, and finally to implicit disjunctive warm starts. A central theme of this dissertation is that both forms of reuse can be understood within a single bounding-function framework, with certificates providing the bridge

from disjunctive structure to reusable optimization artifacts.

1.5.3 Complexity of Warm-Started MILPs

Warm starting can substantially improve performance in practice, but it does not make the underlying optimization problem easy in a complexity-theoretic sense. To frame the computational results that follow, we record a basic hardness observation for warm-started MILPs.

Definition 1.5.1. *The* Mixed integer Linear Optimization Problem (MP) *takes as input the formulation MILP for datum* (A, b, c) *and returns an optimality certificate* $(\bar{x}, \bar{\mathcal{X}})$ *if* $\mathcal{S}(A, b) \neq \emptyset$ *and (NULL, NULL) otherwise.*

Definition 1.5.2. *The* Warm-Started Mixed integer Linear Optimization Problem (WMP) *takes as inputs the formulation MILP at datum* (A, b, c)*, the same formulation at datum* (A', b', c')*, and an optimality certificate* $(x, \mathcal{X})$ *for datum* (A, b, c)*, such that exactly one of the following statements is true:*

- $c \neq c'$*, or*
- *there exists exactly one* $i \in [q]$ *such that* $A'_{i\cdot} \neq A_{i\cdot}$ *or* $b'_i \neq b_i$*.*

It returns a certificate $(\bar{x}, \bar{\mathcal{X}})$ *for datum* (A', b', c') *if* $\mathcal{S}(A', b') \neq \emptyset$ *and (NULL, NULL) otherwise.*

Using these definitions, we construct a Cook reduction to show that warm-starting MILPs is NP-hard.

Theorem 1.5.3. *The Warm-Started Mixed integer Linear Optimization Problem is NP-hard.*

Proof. We reduce MP to WMP via `Cook Reduction`. Let (A, b, c) be the target datum, let (A', b', c') be the intermediate datum updated during the reduction, and let (A'', b'', c'') be the temporary datum whose certificate is supplied to WMP. Then $A, A', A'' \in \mathbb{R}^{q \times n}$, $\bar{x} \in \mathbb{R}^n$, and $|\bar{\mathcal{X}}| < 2^n$, so the inputs to WMP are asymptotically constant bound. Consider $(x, \mathcal{X})$ to be an optimality certificate for MILP created by solving MILP as an MP. Then $(x, \mathcal{X})$ is a

feasible output for solving MILP via our `Cook Reduction` because the reduction returns an optimality certificate for MILP after solving a sequence of restrictions ending with MILP. Conversely, if $(x, \mathcal{X})$ is an optimality certificate for MILP created by solving MILP via our `Cook Reduction`, then $(x, \mathcal{X})$ is also a feasible output for solving MILP directly as an MP. Therefore, a bijection exists between the feasible instances of MP and WMP. Assuming we have a polynomial-time algorithm for WMP, our `Cook Reduction`, constant bounds on the sizes of inputs to WMP, and the bijection between the feasible instances of MP and WMP imply that MP can be solved in polynomial time. Since MP is NP-hard, WMP is NP-hard as well.

Algorithm 4 `Cook Reduction`

Require: MILP

$A'' = 0_{q \times n}$, $b'' = 0_q$, $c'' = 0_n$ ▷ Instantiate temporary datum (A'', b'', c'')

$\bar{x} = 0_n$, $\bar{\mathcal{X}} = \{\mathbb{R}^n\}$ ▷ Initial certificate for (A'', b'', c'') is trivial

$(A', b', c') = (A'', b'', c'')$ ▷ Instantiate intermediate datum (A', b', c') as a copy of (A'', b'', c'')

for $i \in [q]$ **do**

  $A'_{i\cdot} = A_{i\cdot}$, $b'_i = b_i$ ▷ Add the next constraint of target datum (A, b, c) to (A', b', c')

  $\bar{x}, \bar{\mathcal{X}} = \text{WMP}((A'', b'', c''), (A', b', c'), \bar{x}, \bar{\mathcal{X}})$ ▷ Warm-start intermediate datum (A', b', c') with the certificate from (A'', b'', c'')

  if $\bar{x}$ is NULL **then** ▷ Infeasibility at (A', b', c') implies infeasibility at (A, b, c)

    return (NULL, NULL)

  end if

  $(A'', b'', c'') = (A', b', c')$ ▷ Update the temporary datum

end for

$\bar{x}, \bar{\mathcal{X}} = \text{WMP}((A'', b'', c''), (A, b, c), \bar{x}, \bar{\mathcal{X}})$ ▷ Warm-start target datum (A, b, c) with the final certificate

return $\bar{x}, \bar{\mathcal{X}}$

□

This hardness result cautions against overinterpreting negative computational outcomes. If a warm-start strategy occasionally fails to improve solve time, that does not by itself indicate a defect in the idea; rather, it reflects that reoptimization remains computationally difficult even when nontrivial structural information is provided in advance.

1.6 Summary of Contributions

The explicit reuse of disjunctions through inherited branch-and-bound nodes has been studied previously, particularly in Güzelsoy [40]. In contrast, the machinery needed for *implicit* disjunction reuse—that is, reusing certificates to instantiate valid disjunctive inequalities on a new instance—has not previously been developed in a comparable form. As a consequence, there has also been no direct comparison between these two ways of reusing the same underlying disjunctive structure. This dissertation is organized around filling that gap.

- We develop PDIs from reusable certificates, yielding an implicit warm-start mechanism for disjunction reuse.
- We develop strengthening procedures to preserve the quality of these PDIs when instance changes weaken them.
- We compare explicit node warm starts and implicit cut warm starts under shared disjunctions to identify regimes in which each is most effective.

1.7 Literature Review

The literature review below provides the context for these contributions and shows that they fill a genuine gap in the existing body of work.

This section situates the dissertation within two bodies of work: warm starting branch and cut for sequences of related MILPs, and the theory and practice of cutting planes—particularly disjunctive cuts. The emphasis is on methods that reuse information induced by branching, either explicitly through partial branch and bound trees or implicitly through certificates that can generate valid inequalities for related instances.

Most warm-start strategies focus on reusing information that is inexpensive to store and straightforward to transfer, such as primal solutions, branching guidance, and solver parameters [53, 36, 32, 54, 37, 29]. In contrast, comparatively little work addresses warm starting the cut-generation process itself, especially for cut families whose strength is tied

to expensive auxiliary computations (e.g., solving cut-generating LPs for large disjunctions) [40, 52, 31, 19]. This is a natural omission in many settings because standard cutting-plane routines are relatively cheap to rerun. Two developments motivate revisiting this conventional wisdom. First, recent work suggests that instance-dependent choices in cut generation can materially affect performance, creating an opportunity for parameter transfer or learning-based tuning [19, 25, 31]. Second, some of the strongest cut families—particularly disjunctive cuts from large disjunctions—are often avoided in practice because generating them can be computationally intensive [55, 22, 12, 20, 1]. The methods developed in this dissertation target precisely this regime by reusing disjunction-induced certificates to amortize the cost of generating strong disjunctive cuts across related solves.

1.7.1 Warm-Starting

We define warm-starting a MILP instance as the process of leveraging information from previous instances to improve the solver's performance metrics on a new instance. This approach can include supplying full or partial solutions, where in the latter case the solution is completed by heuristics, to tighten the primal bound [54]. The dual bound can also be strengthened by incorporating previously derived structural information, like disjunctions [40, 52] and parameterized cuts [40]. Additionally, warm-starting can involve recycling branching pseudo-costs from prior instances to enhance the convergence rate of the primal-dual gap [54]. Our research addresses a gap in this field by investigating the reuse of parameters specifically for generating disjunctive cuts, akin to the method of constructing Gomory cuts explored in Güzelsoy [40].

In recent years, warm-starting branch and cut heuristics using trained neural networks has gained popularity. Many standard warm-starting techniques can be adapted into a machine learning framework through offline training steps. Examples include predicting full or partial primal solutions [53], selecting initial disjunctions [62], setting parameters for cut generation [31, 30], and initializing branching pseudo-costs [53, 37]. Compared to our approach, machine-learning warm-starts exchange the need for such a well-defined problem space with a demand for substantial development time and computational resources during training [53]. Our work aims to contribute more choices for practitioners to select a warm-

starting strategy tailored to their specific needs.

We primarily build on earlier work that develops a methodology for reusing branch and bound trees to solve a new instance by continuing in the same tree as a previous instance (after modifying the relaxations in the leaf nodes) [58, 40]. This approach is fully implemented in the SYMPHONY MILP solver [57] and is the basis, for example, of a generalized Benders algorithm for solving two-stage stochastic optimization problems [41]. The main weakness of this method is that it is difficult to predict *a priori* when the previous disjunction will be effective for the new instance; when it is not, the resulting tree can grow dramatically with limited recourse. In the present work, we aim to retain more flexibility by warm starting with valid inequalities derived from prior runs—information that can be discarded if needed—thereby reducing the risk of large regressions while still capturing benefits when the prior disjunction remains informative.

1.7.2 Cutting Planes

Cutting planes are generated to separate a point—typically the optimal solution of an LP relaxation of a MILP—from the convex hull of MILP solutions by exploiting the integrality of variables [26]. There are two primary methods for generating cuts: the first involves applying general polyhedral or rounding principles to the LP relaxation [38, 24, 27, 14]; the second leverages specific problem structures, such as knapsack capacities [10] or conflict graphs [7], to derive constraints that exclude infeasible solutions closely aligned with application-specific constraints. Modern MILP solvers incorporate these cut generation techniques [20, 1], as they significantly enhance solver performance creating a profound impact on overall efficiency [8, 20]. The substantial improvements that cuts provide to solver performance motivate our desire to contribute to this area of research by exploring novel approaches to cut generation.

Of specific interest in this work is the history of disjunctive cuts, with the first significant computational progress being lift-and-project cuts [14, 8] from a variable (two-term) split disjunction. Even for this simple special case, avoiding the higher-dimensional original formulation of these cuts is crucial for a practical implementation [13, 21], making it computationally prohibitive to realize otherwise theoretically-attractive methods for deriving

disjunctive cuts from branch and bound trees [22, 23]. Perregaard and Balas [55] propose an alternative without a lifted space, based on reverse polars, an idea that recurs through the literature [49]. This notion is further developed by Balas and Kazachkov [12], who eliminate the expensive row generation step found in earlier works and present an efficient method for calculating $\mathcal{V}$*-polyhedral disjunctive cuts* (VPCs), which we employ in this paper. Additionally, we draw on the research of Kazachkov and Balas [45], which details the computation of the Farkas certificate of validity for VPCs. Although disjunctive cuts are implemented in modern solvers [20, 1], they are often disabled by default due to their average negative impact on runtime performance [20, 12]. However, recent advancements in their strength, particularly those in VPCs [12, 45], motivate further investigation, as even modest improvements in performance could have significant practical impacts given their existing integration in solver frameworks.

Warm-starting offers a promising setting to explore the utility of disjunctive cuts. Their effectiveness is often limited by generation time, but warm-starting allows their strength to be retained while distributing the cost across instances. Further, their tight approximation of the disjunctive hull [12] makes disjunctive cuts a natural cut-based alternative to warm-starting with disjunctions. Understanding the potential of these advantages relies on developing a robust and efficient parameterization scheme, which is the focus of this work.

1.8 Organization of the Dissertation

This chapter develops the notation, background, and technical framework needed for the remainder of the dissertation and situates the work within the relevant literature. Chapter 2 develops PDIs from reusable certificates. Chapter 3 studies how to strengthen those PDIs when instance changes weaken them. Chapter 4 compares explicit node warm starts and implicit cut warm starts under shared disjunctions. Chapter 5 concludes with a summary of findings and directions for future research.

Chapter 2

Parametric Disjunctive Inequalities

Chapter 1 framed this dissertation around a gap in reoptimization with disjunctions. Prior work shows how to reuse disjunctions explicitly by transferring information from partial branch-and-bound trees to warm start the node pool, but comparatively little is known about warm-starting the cut pool by deriving new parametrically valid disjunctive inequalities at a subsequent solve. This chapter develops that missing framework.

The setting is a parametric family of admissible data Ω. A disjunctive inequality that is valid at one datum $(A, b, c) \in \Omega$ need not remain valid throughout the family. Our main result is that parametric validity of PDIs is encoded by their Farkas certificates, which can be reused to instantiate strong inequalities at nearby admissible data. We refer to initializing the cut pool in Algorithm 3 with such instantiated inequalities as a *cut warm start*.

At a high level, a disjunction defines parametrically a disjunctive hull, against which we desire a family of disjunctive inequalities, but generating members of that family repeatedly can be expensive. The key observation is that much of this cost can be captured in the family's certificate of validity. Once such a certificate is available at one datum, it can be applied to another admissible datum to instantiate a new valid inequality, transferring the disjunction implicitly through the cut pool rather than through repeated term evaluation.

The remainder of the chapter makes these ideas formal. Section 2.1 reviews disjunctive cut generation and the recovery of Farkas certificates. Section 2.2 establishes theoretical results that characterize when certificate reuse yields PDIs and provides sufficient conditions

under which the resulting inequalities support the disjunctive hull or separate LP relaxation solutions. Section 2.3 then evaluates the computational implications of this machinery on sequences of perturbed MIPLIB instances, quantifying the extent to which certificate reuse can recover the strength of disjunctive cutting while avoiding its full regeneration cost.

2.1 Disjunctive Cut Generation

As shown in Section 1.3.2, warm starting MILPs with disjunctions need not rely on their explicit reuse. This work shows that disjunctions can also be leveraged implicitly by reusing their associated certificates of validity to efficiently generate cutting planes. In doing so, we build on foundational results from disjunctive programming and disjunctive cut generation, which we briefly review below. Throughout this section, let $(A, b, c) \in \Omega$ be an arbitrary admissible datum and let $\{\mathcal{X}^t\}_{t \in T}$ be a valid disjunction. We use the family notation of Section 1.4, so the associated LP relaxation, disjunctive terms, and disjunctive hull are $\mathcal{P}(A, b)$, $\mathcal{Q}^t(A, b)$, and $\mathcal{P}_D(A, b)$, respectively.

2.1.1 Cut-Generating LP

The well-studied *cut-generating LP* (CGLP) provides a direct mechanism for generating inequalities that are valid for the disjunctive hull $\mathcal{P}_D(A, b)$ with respect to a given valid disjunction $\{\mathcal{X}^t\}_{t \in T}$. The validity of the resulting cuts is inherent to the formulation of the CGLP itself: it is posed in a higher-dimensional space that includes variables both for the cut coefficients and for the Farkas multipliers that certify the validity of the cut relative to the disjunction.

The CGLP can be viewed as an explicit construction of inequalities satisfying FL. In practice, it is typically formulated as a separation problem that maximizes the violation of a point $\bar{x} \notin \mathcal{P}_D(A, b)$, subject to a normalization constraint, the choice of which can have a significant impact on the strength and numerical behavior of the resulting cuts [35]. A common and straightforward normalization is to scale the inequality so that its right-hand side is fixed to $\bar{\beta} := 1$. Under this normalization, generating a valid disjunctive cut maximally violating a point $\bar{x} \notin \mathcal{P}_D(A, b)$ reduces to solving CGLP-$(\mathcal{X})$.

$$
\begin{aligned}
\min_{(\alpha,\beta)\in\mathbb{R}^n\times\mathbb{R}} \quad & \alpha^\top \bar{x} - \beta & & \\
& \alpha^\top = v^t A^t & \text{for } t \in T & \\
& \beta \leq v^t b^t & \text{for } t \in T & \\
& \beta = \bar{\beta} & & \\
& v^t \in \mathbb{R}^{1\times(q+q_t)}_{\geq 0} & \text{for } t \in T &
\end{aligned}
\tag{CGLP-($\mathcal{X}$)}
$$

2.1.2 Point-Ray LP

Solving the CGLP becomes prohibitively expensive in practice when formulated for disjunctions with more than two terms, despite the strength of the resulting cuts [55]. To address this, Balas and Kazachkov [12] introduce the VPC framework, which targets cuts derived from much larger disjunctions while modifying the cut-generation procedure in two key ways: first, by avoiding the solution of LPs with more than n variables, and second, by reducing the number of constraints through the generation of cuts against a relaxation of the disjunctive hull.

First, they consider the generation of cuts using the $\mathcal{V}$-*polyhedral* description of $\mathcal{P}_D(A,b)$, through its extreme points and rays, as opposed to the inequality description involved in the CGLP. Let $\mathcal{E}^t(A,b)$ be the set of extreme points and $\mathcal{R}^t(A,b)$ be the set of extreme rays of $\mathcal{Q}^t(A,b)$. Further, define $\mathcal{E}(A,b) \coloneqq \cup_{t\in T}\mathcal{E}^t(A,b)$ and $\mathcal{R}(A,b) \coloneqq \cup_{t\in T}\mathcal{R}^t(A,b)$. Since $x \in \mathcal{P}_D(A,b)$ if and only if $x \in \text{conv}(\mathcal{E}(A,b)) + \text{cone}(\mathcal{R}(A,b))$, it holds that (α,β) is valid for $\mathcal{P}_D(A,b)$ if and only if (α,β) satisfy the following system defining a *point-ray polyhedron* (PRP):

$$
\begin{aligned}
& \alpha^\top p \geq \beta & \text{for all } p \in \mathcal{E}(A,b) \\
& \alpha^\top r \geq 0 & \text{for all } r \in \mathcal{R}(A,b).
\end{aligned}
\tag{PRP}
$$

Cuts valid for (PRP) are referred to as VPCs [12]. The advantage of PRP over CGLP-($\mathcal{X}$) is the PRP involves only n variables, those of the cut coefficients.

However, the number of constraints of the PRP may be exponential in n and q, as opposed to the polynomial-sized CGLP. This leads to the second remedy used by Balas

and Kazachkov [12]: relax the disjunctive terms by replacing the full set of constraints of $\mathcal{Q}^t(A,b)$ with just those associated with an optimal basis. Let B^t index those constraints and let $\mathcal{C}^t(A,b)$ denote the resulting basis cone for term t. Crucially, for all $t \in T$, $\mathcal{C}^t(A,b)$ has a compact $\mathcal{V}$-polyhedral description consisting of a single point and n rays. Balas and Kazachkov [12] exploit this by replacing the point-ray collection $(\mathcal{E}(A,b), \mathcal{R}(A,b))$ used in (PRP) with the union of the points and rays defining these basis cones for all $t \in T$, leading to a *point-ray LP* (PRLP) with at most $|T| \cdot (n+1)$ constraints.

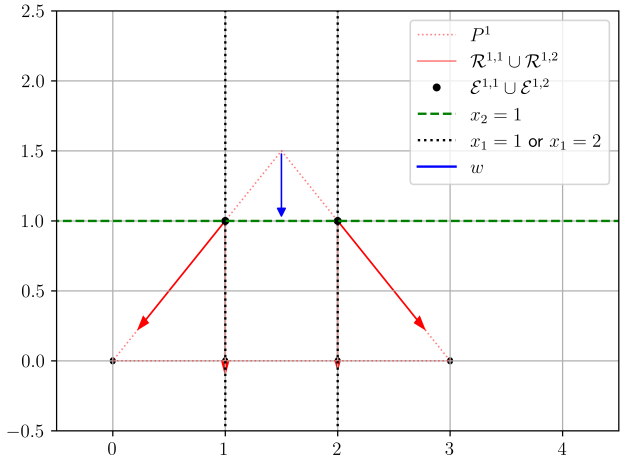
$$\begin{aligned} \min_{(\alpha,\beta)\in\mathbb{R}^n\times\mathbb{R}} \quad & \alpha^\top w & & \\ & \alpha^\top p \geq \beta \qquad & \text{for all } p \in \mathcal{E}(A,b) & \\ & \alpha^\top r \geq 0 \qquad & \text{for all } r \in \mathcal{R}(A,b) & \text{(PRLP-}(\mathcal{X},w)\text{)} \\ & \beta = \bar{\beta} & & \end{aligned}$$

Similar to CGLP-$(\mathcal{X})$, PRLP-$(\mathcal{X},w)$ requires normalizing, which can again be accomplished by constraining $\beta = \bar{\beta}$, appropriate values for which are discussed in Balas and Kazachkov [12]. Cuts obtained from PRLP-$(\mathcal{X},w)$ are referred to as *simple* VPCs, and are the ones used in our experiments. When the disjunction $\{\mathcal{X}^t\}_{t\in T}$ comes from the leaf nodes of a partial branch and bound tree, for each $t \in T$, the point and rays defining $\mathcal{C}^t(A,b)$ can be readily obtained from the optimal tableau of the LP relaxation at the corresponding leaf node.

Example 2.1.1. *This example illustrates how to generate a VPC by solving a PRLP given MILP-1, the disjunction $\{\mathcal{X}^1, \mathcal{X}^2\}$, and objective direction w as defined in Figure 2.1.*

We define the following additional notation for ease of exposition. We reference the LP relaxation of MILP-k in standard form with $\tilde{\mathcal{P}}^k := \{\tilde{x} \in \mathbb{R}^{n+q} : \tilde{A}^k\tilde{x} = b^k\} = \{(x,s) \in \mathbb{R}^{n+q} : A^k x - s = b^k, s \geq 0\}$. $\mathcal{B} \in \mathbb{N}^q$ and $\mathcal{N} \in \mathbb{N}^n$ partition $[n+q]$ and index basic and nonbasic columns, respectively, of basic solution $\tilde{x}^{k,\mathcal{B}}$ of the LP relaxation of MILP-k, which is defined $\tilde{x}^{k,\mathcal{B}}_{\mathcal{B}} := (\tilde{A}^k_{\cdot\mathcal{B}})^{-1}b^k$ and $\tilde{x}^{k,\mathcal{B}}_{\mathcal{N}} := 0$ since $x \geq 0$. $\tilde{\mathcal{Q}}^{kt} \in \mathbb{R}^{n+q+q_t}$, $\mathcal{B}^t \in \mathbb{N}^{q+q_t}$, and $\mathcal{N}^t \in \mathbb{N}^n$ are defined for LP analogously to $\tilde{\mathcal{P}}^k$, $\mathcal{B}$, and $\mathcal{N}$. Similarly, basic solution $\tilde{x}^{kt,\mathcal{B}^t}$ is defined $\tilde{x}^{kt,\mathcal{B}^t}_{\mathcal{B}^t} := (A^{kt}_{\cdot\mathcal{B}^t})^{-1}b$ and $\tilde{x}^{kt,\mathcal{B}^t}_{\mathcal{N}^t} := 0$.

Let $\mathcal{B}^1 = [1,2,4,5,6]$ and $\mathcal{N}^1 = [3,7]$ be the optimal basic and nonbasic columns of



$$\begin{aligned} \min_{x \in \mathbb{R}^2} \quad & -x_2 \\ x_1 \quad & -x_2 \geq 0 \\ -x_1 \quad & -x_2 \geq -3 \\ x_1 \quad & \geq 0 \\ & x_2 \geq 0 \\ x_1, \quad & x_2 \in \mathbb{Z} \end{aligned} \tag{MILP-1}$$

Figure 2.1: Solving PRLP-1$(\{\mathcal{X}^1, \mathcal{X}^2\}, w)$ *with* $w = \begin{bmatrix} 0 \\ -1 \end{bmatrix}$, $\mathcal{X}^1 = \{x \in \mathbb{R}^2 : -x_1 \geq -1\}$, *and* $\mathcal{X}^2 = \{x \in \mathbb{R}^2 : x_1 \geq 2\}$ *yields the VPC* $x_2 \leq 1$, *a valid inequality for MILP-1.*

LP-$1,1$ *, respectively, and let* $\mathcal{B}^2 = [1,2,3,5,6]$ *and* $\mathcal{N}^2 = [4,7]$ *be the optimal basic and nonbasic columns of LP-*$1,2$ *, respectively. Thus, we have*

$$\left(\tilde{A}^{1,1}_{\cdot\mathcal{B}^1}\right)^{-1} = \begin{bmatrix} 0 & 0 & 0 & 0 & -1 \\ -1 & 0 & 0 & 0 & -1 \\ 1 & -1 & 0 & 0 & 2 \\ 0 & 0 & -1 & 0 & -1 \\ -1 & 0 & 0 & -1 & -1 \end{bmatrix}, \; \tilde{A}^{1,1}_{\cdot\mathcal{N}^1} = \begin{bmatrix} -1 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & -1 \end{bmatrix}, \; b^{1,1} = \begin{bmatrix} 0 \\ -3 \\ 0 \\ 0 \\ -1 \end{bmatrix}$$

$$\left(\tilde{A}^{1,2}_{\cdot\mathcal{B}^2}\right)^{-1} = \begin{bmatrix} 0 & 0 & 0 & 0 & 1 \\ 0 & -1 & 0 & 0 & -1 \\ -1 & 1 & 0 & 0 & 2 \\ 0 & 0 & -1 & 0 & 1 \\ 0 & -1 & 0 & -1 & -1 \end{bmatrix}, \; \tilde{A}^{1,2}_{\cdot\mathcal{N}^2} = \begin{bmatrix} 0 & 0 \\ -1 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & -1 \end{bmatrix}, \; \text{and } b^{1,2} = \begin{bmatrix} 0 \\ -3 \\ 0 \\ 0 \\ 2 \end{bmatrix}.$$

For $t \in T$, *the basis cone is*

$$A^{kt}_{B^t\cdot} = \bar{x}^{kt} + cone\left\{r^{k,t,h} : r^{k,t,h} = \text{proj}_x\left(\left(-\tilde{A}^{kt}_{\cdot\mathcal{B}^t}\right)^{-1} \tilde{A}^{kt}_{\cdot j}\right), \; j = \mathcal{N}^t_h\right\},$$

where

$$\bar{x}^{11} = \operatorname{proj}_x\left(\left(\tilde{A}^{1,1}_{\cdot\mathcal{B}^1}\right)^{-1} b^{1,1}\right) = \begin{bmatrix}1\\1\end{bmatrix},$$

$$\bar{x}^{12} = \operatorname{proj}_x\left(\left(\tilde{A}^{1,2}_{\cdot\mathcal{B}^2}\right)^{-1} b^{1,2}\right) = \begin{bmatrix}2\\1\end{bmatrix},$$

$$r^{1,1,1} = -\operatorname{proj}_x\left(\left(\tilde{A}^{1,1}_{\cdot\mathcal{B}^1}\right)^{-1} \tilde{A}^{1,1}_{\cdot\mathcal{N}^1_1}\right) = \begin{bmatrix}0\\-1\end{bmatrix},$$

$$r^{1,1,2} = -\operatorname{proj}_x\left(\left(\tilde{A}^{1,1}_{\cdot\mathcal{B}^1}\right)^{-1} \tilde{A}^{1,1}_{\cdot\mathcal{N}^1_2}\right) = \begin{bmatrix}-1\\-1\end{bmatrix},$$

$$r^{1,2,1} = -\operatorname{proj}_x\left(\left(\tilde{A}^{1,2}_{\cdot\mathcal{B}^2}\right)^{-1} \tilde{A}^{1,2}_{\cdot\mathcal{N}^2_1}\right) = \begin{bmatrix}0\\-1\end{bmatrix},$$

$$r^{1,2,2} = -\operatorname{proj}_x\left(\left(\tilde{A}^{1,2}_{\cdot\mathcal{B}^2}\right)^{-1} \tilde{A}^{1,2}_{\cdot\mathcal{N}^2_2}\right) = \begin{bmatrix}1\\-1\end{bmatrix},$$

yielding the translated cones pictured in Figure 2.1. Therefore,

$$\mathcal{E}^k = \left\{\begin{bmatrix}1\\1\end{bmatrix}, \begin{bmatrix}2\\1\end{bmatrix}\right\} \quad \textit{and} \quad \mathcal{R}^k = \left\{\begin{bmatrix}0\\-1\end{bmatrix}, \begin{bmatrix}-1\\-1\end{bmatrix}, \begin{bmatrix}0\\-1\end{bmatrix}, \begin{bmatrix}1\\-1\end{bmatrix}\right\}.$$

*Hence PRLP-*1$(\mathcal{X}, w)$ *has optimal solution* $(\alpha, \beta) = ((0, -1), -1)$*, implying* $-x_2 \geq -1$*, or* $x_2 \leq 1$ *as written above.*

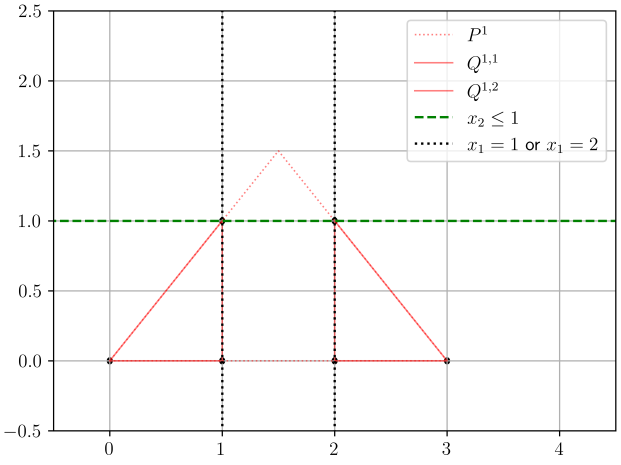
2.1.3 Deriving Farkas Certificates

Given a valid disjunction $\{\mathcal{X}^t\}_{t\in T}$ and an admissible datum $(A, b, c) \in \Omega$, let (α, β) be a simple VPC valid for $\mathcal{P}_D(A, b)$, generated via (PRLP-$(\mathcal{X}, w)$). As such, the Farkas certificate for this cut is not a byproduct of cut generation. Fortunately, Kazachkov and Balas [45, Section 3.1] show that, for the specific setting of simple VPCs, the Farkas certificate can be recovered efficiently.

Lemma 2.1.2. *If* $\alpha^\top x \geq \beta$ *is valid for* $\mathcal{C}^t(A, b)$*, then the multiplier on constraint* $i \in [q + q_t]$

has value $v_i^t = \bar{\alpha}^\top r^h$*, where* r^h *is column* h *of* $(A^t_{B^t\cdot})^{-1}$*, if there exists* $h \in [n]$ *such that* $B^t_h = i$*, else 0.*

Lemma 2.1.2 allows us to compute the Farkas certificate using the rays describing the basis cone, which can be read from the optimal tableau for disjunctive term t. In principle, for simple VPCs derived from partial branch and bound trees, this tableau does not require additional computation due to our choice of a term-optimal solution as the reference point for t, which is already calculated for each leaf node by the solver when constructing the partial tree.



$$\begin{aligned} \min_{x \in \mathbb{R}^2} \quad & -x_2 \\ x_1 \quad & -x_2 \geq 0 \\ -x_1 \quad & -x_2 \geq -3 \\ x_1 \quad & \geq 0 \\ & x_2 \geq 0 \\ x_1, \quad & x_2 \in \mathbb{Z} \end{aligned} \tag{MILP-1}$$

Figure 2.2: The LP relaxation of MILP-1, $\mathcal{P}^1$*, overlaid with the simple VPC* $-x_2 \geq -1$ *and LP relaxations* $\mathcal{Q}^{1,1}$ *and* $\mathcal{Q}^{1,2}$*, which arise from the application of disjunction* $\{\mathcal{X}^1, \mathcal{X}^2\}$ *where* $\mathcal{X}^1 := \{x \in \mathbb{R}^2 : -x_1 \geq -1\}$ *and* $\mathcal{X}^2 := \{x \in \mathbb{R}^2 : x_1 \geq 2\}$*.*

Example 2.1.3. *This example illustrates how to calculate a Farkas certificate* $\{v^1, v^2\}$ *given MILP-1, the disjunction* $\{\mathcal{X}^1, \mathcal{X}^2\}$*, and the simple vpc* $-x_2 \geq -1$ *as defined in Figure 2.2. Let* $\mathcal{N}^{1,1} := [1, 5]$ *and* $\mathcal{N}^{1,2} := [2, 5]$ *be the optimal nonbasis elements for LP-*$1, 1$ *and*

LP-1, 2 *, respectively. Let*

$$C^{1,1} := A^{1,1}_{\mathcal{N}^{1,1}\cdot} = \begin{bmatrix} 1 & -1 \\ -1 & 0 \end{bmatrix}, \qquad C^{1,1}_0 := b^{1,1}_{\mathcal{N}^{1,1}} = \begin{bmatrix} 0 \\ -1 \end{bmatrix},$$
$$C^{1,2} := A^{1,2}_{\mathcal{N}^{1,2}\cdot} = \begin{bmatrix} -1 & -1 \\ 1 & 0 \end{bmatrix}, \text{ and } C^{1,2}_0 := b^{1,2}_{\mathcal{N}^{1,2}} = \begin{bmatrix} -3 \\ 2 \end{bmatrix}.$$

Then by Lemma 2.1.2, we have that

$$v^1_{\mathcal{N}^{1,1}} = \alpha^\top (C^{1,1})^{-1} = \begin{bmatrix} 0 & -1 \end{bmatrix} \begin{bmatrix} 0 & -1 \\ -1 & -1 \end{bmatrix} = \begin{bmatrix} 1 & 1 \end{bmatrix} \text{ and}$$
$$v^2_{\mathcal{N}^{1,2}} = \alpha^\top (C^{1,2})^{-1} = \begin{bmatrix} 0 & -1 \end{bmatrix} \begin{bmatrix} 0 & 1 \\ -1 & -1 \end{bmatrix} = \begin{bmatrix} 1 & 1 \end{bmatrix},$$

which means $v^1 = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 \end{bmatrix}$ *and* $v^2 = \begin{bmatrix} 0 & 1 & 0 & 0 & 1 \end{bmatrix}$. *Equation FL holds when*

inputting the certificate $\{v^1, v^2\}$ *and the VPC* $-x_2 \geq -1$*:*

$$v^1 A^{1,1} x \geq v^1 b^{1,1} \implies \begin{bmatrix} 1 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ -1 & -1 \\ 1 & 0 \\ 0 & 1 \\ -1 & 0 \end{bmatrix} x \geq \begin{bmatrix} 1 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ -3 \\ 0 \\ 0 \\ -1 \end{bmatrix}$$

$$\implies \begin{bmatrix} 0 & -1 \end{bmatrix} x \geq -1$$

$$v^2 A^{1,2} x \geq v^2 b^{1,2} \implies \begin{bmatrix} 0 & 1 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ -1 & -1 \\ 1 & 0 \\ 0 & 1 \\ 1 & 0 \end{bmatrix} x \geq \begin{bmatrix} 0 & 1 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ -3 \\ 0 \\ 0 \\ 2 \end{bmatrix}$$

$$\implies \begin{bmatrix} 0 & -1 \end{bmatrix} x \geq -1$$

Thus, the certificate $\{v^1, v^2\}$ *is correct.*

2.2 Parametric Disjunctive Inequalities

Our work develops theory around two key ideas. First, PDIs can reduce the feasible search space while preserving optimality. In Sections 2.2.1 - 2.2.3, we establish their validity against the generating disjunctive hull and provide conditions under which they both support and separate it from the LP relaxation. Second, while parameterization can aid warm-starting, it is not a universal solution. In Section 1.5.3, we prove that warm-starting MILP's in general is NP-hard, meaning PDIs may not always simplify problem-solving. Section 2.3's computational results explore where practical outcomes fall between these extremes.

2.2.1 Validity

To reuse disjunctive inequalities across a parametric family while preserving validity, we derive parametric inequalities for the disjunctive relaxation via a procedure that guarantees

their validity by ensuring such validity is satisfied by a known certificate. For $(A, b) \in \Omega_A \times \Omega_b$ and $t \in T$, let

$$A^t := \begin{bmatrix} A \\ D^t \end{bmatrix}, \qquad b^t := \begin{bmatrix} b \\ D_0^t \end{bmatrix},$$

and similarly define A'^t and b'^t for $(A', b') \in \Omega_A \times \Omega_b$.

In this setting, it is useful to identify which datum–disjunction pairs could have given rise to a particular Farkas certificate.

Definition 2.2.1. *Feasible-region data $(A, b) \in \Omega_A \times \Omega_b$ and disjunction $\{\mathcal{X}^t\}_{t\in T}$* induce *the Farkas certificate $\{v^t\}_{t\in T}$ if $v^{t_1} A^{t_1} = v^{t_2} A^{t_2}$ for all $t_1, t_2 \in T$ such that $\mathcal{Q}^t(A, b)$ is feasible for both t_1 and t_2.*

It is also useful to identify the basis used to construct the Farkas coefficients for a given feasible disjunctive term $\mathcal{Q}^t(A, b)$. Intuitively, the nonzero components of v^t correspond to constraints that are active in the construction of the cut coefficients. We therefore associate v^t with a basis B^t of $\mathcal{Q}^t(A, b)$, consisting of the constraints whose linear combination yields the cut coefficients α.

Definition 2.2.2. *The basis B^t (collection of bases $\{B^t\}_{t\in T}$)* determines *the Farkas certificate term v^t (Farkas certificate $\{v^t\}_{t\in T}$) if $\{i \in [q + q_t] : v_i^t > 0\} \subseteq B^t$ (for all $t \in T$).*

We now establish procedures for generating parametric families of valid inequalities by taking a previously generated Farkas certificate of validity as input and generating a cut from it. By taking a certificate as an input, we avoid solving an LP to compute it, while hopefully retaining the strength of the cut due to similarity amongst admissible data in the family.

Definition 2.2.3. *Let disjunction $\{\mathcal{X}^t\}_{t\in T}$ and feasible-region data $(A, b) \in \Omega_A \times \Omega_b$ induce Farkas certificate $\{v^t\}_{t\in T}$. For each $(A, b) \in \Omega_A \times \Omega_b$, define*

$$(\alpha(A, b), \beta(A, b)) := \left(\left[\max_{t\in T} \{v^t A_{\cdot 1}^t\}, \ldots, \max_{t\in T} \{v^t A_{\cdot n}^t\} \right]^\top, \min_{t\in T} \{v^t b^t\} \right).$$

Then $\{(\alpha(A, b), \beta(A, b))\}_{(A,b)\in\Omega_A\times\Omega_b}$ is a family of Farkas PDIs.

For ease of exposition, when we describe families of PDIs going forward, we mean *Farkas* PDIs since we will be working with Farkas certificates. The following result shows they are parametrically valid over the entire admissible family. For clarity, we use (A, b) to denote the feasible-region data inducing the Farkas certificate and (A', b') to denote the feasible-region data to which it is applied.

Theorem 2.2.4. *Let $(A, b), (A', b') \in \Omega_A \times \Omega_b$. Given a Farkas certificate $\{v^t\}_{t\in T}$ induced by disjunction $\{\mathcal{X}^t\}_{t\in T}$ and feasible-region data (A, b), define*

$$\alpha'_j := \max_{t\in T}\{v^t A'^t_{\cdot j}\}, \qquad \beta' := \min_{t\in T}\{v^t b'^t\}$$

for all $j \in [n]$. Then ${\alpha'}^\top x \geq \beta'$ for all $x \in \mathcal{P}_D(A', b')$.

Proof. Let $\gamma^t := v^t A'^t$ and $\gamma^t_0 := v^t b'^t$ for all $t \in T$. We will show that ${\alpha'}^\top x \geq \beta'$ is valid for $\mathcal{Q}^t(A', b')$ for a fixed (arbitrary) index $t \in T$. We have that $\gamma^t x \geq \gamma^t_0$ is valid for $\mathcal{Q}^t(A', b')$. Then, if $x \in \mathcal{Q}^t(A', b')$, since we have nonnegativity on the variables ($\mathcal{Q}^t(A', b') \subseteq \mathbb{R}^n_{\geq 0}$), it follows that,

$${\alpha'}^\top x = \sum_{j\in[n]} \max_{t'\in T}\{\gamma^{t'}_j\} x_j \geq \sum_{j\in[n]} \gamma^t_j x_j \geq \gamma^t_0 \geq \min_{t'\in T}\{\gamma^{t'}_0\} = \beta'.$$

Hence, ${\alpha'}^\top x \geq \beta'$ is valid for $\bigcup_{t\in T} \mathcal{Q}^t(A', b') \supseteq \mathcal{P}_D(A', b')$. □

Figure 2.3 illustrates a family of Farkas PDIs with the annotation "3) generate PDI" corresponding to a cut produced via Theorem 2.2.4. A detailed example of this approach can be found in Example 2.2.6.

Naturally, the result in Theorem 2.2.4 extends to all feasible solutions of any instance in an arbitrary family when the disjunction contains all integer points.

Corollary 2.2.5. *Let $(A, b), (A', b') \in \Omega_A \times \Omega_b$. Let $\{v^t\}_{t\in T}$ be a Farkas certificate induced by disjunction $\mathcal{X}$ and feasible-region data (A, b), where $\mathcal{X}$ is valid for $\mathbb{Z}^n$. Define α' and β' as in Theorem 2.2.4. Then ${\alpha'}^\top x \geq \beta'$ for all $x \in \mathcal{S}(A', b')$.*

Proof. Let $x \in \mathcal{S}(A', b')$. Since $\mathcal{S}(A', b') \subseteq \mathcal{P}_D(A', b')$, $x \in \mathcal{P}_D(A', b')$. By Theorem 2.2.4, ${\alpha'}^\top x \geq \beta'$. □

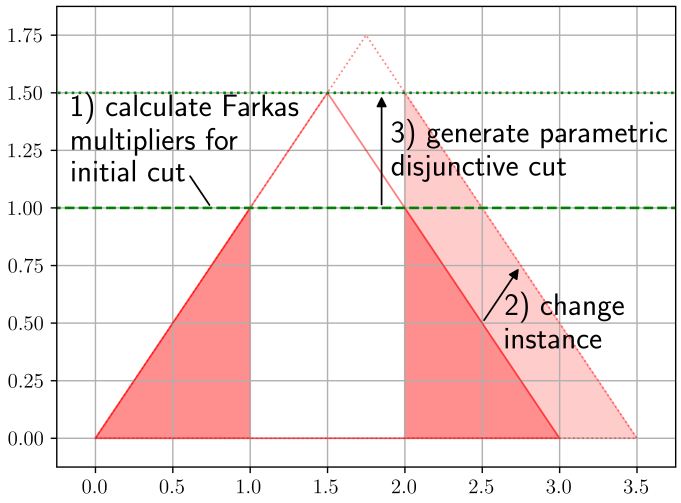
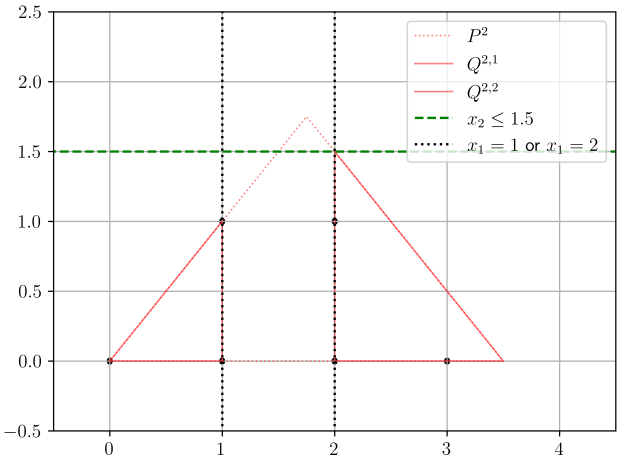


Figure 2.3: Certificate reuse process for a family of Farkas PDIs.

This result highlights the advantage of selecting disjunctions that are valid for all integer-feasible points. Such disjunctions are particularly important in parameterized settings, where a disjunction that is valid for one instance may fail to remain valid for another if the feasible region expands beyond the original disjunctive hull. For this reason, our computational experiments focus on disjunctions with this property, as they preserve the efficiency gains guaranteed by Theorem 2.2.4 while maintaining validity across all integer-feasible points.



$$\begin{aligned} \min_{x \in \mathbb{R}^2} \quad & -x_2 \\ x_1 \quad & -x_2 \geq 0 \\ -x_1 \quad & -x_2 \geq -3.5 \\ x_1 \quad & \geq 0 \\ & x_2 \geq 0 \\ x_1, \quad & x_2 \in \mathbb{Z} \end{aligned} \tag{MILP-2}$$

Figure 2.4: The LP relaxation of MILP-2, $\mathcal{P}^2$, overlaid with the LP relaxations $\mathcal{Q}^{2,1}$ and $\mathcal{Q}^{2,2}$ arising from the application of disjunction $\{\mathcal{X}^1, \mathcal{X}^2\}$. Also, plotted is $-x_2 \geq -1.5$, the parameterization of the simple VPC $-x_2 \geq -1$ from Example 2.1.3.

Example 2.2.6. *Applying the disjunction* $\{\mathcal{X}^1, \mathcal{X}^2\}$ *and the Farkas certificate* $\{v^1, v^2\}$ *from Example 2.1.3 to Theorem 2.2.4 and MILP-2 yields a valid parameterized disjunctive cut for MILP-2. This cut is calculated as follows:*

$$v^1 A^{2,1} x \geq v^1 b^{2,1} \implies \begin{bmatrix} 1 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ -1 & -1 \\ 1 & 0 \\ 0 & 1 \\ -1 & 0 \end{bmatrix} x \geq \begin{bmatrix} 1 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ -3.5 \\ 0 \\ 0 \\ -1 \end{bmatrix} \implies \begin{bmatrix} 0 & -1 \end{bmatrix} x \geq -1$$

$v^2 A^{2,2} x \geq v^2 b^{2,2}$

$$\implies \begin{bmatrix} 0 & 1 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ -1 & -1 \\ 1 & 0 \\ 0 & 1 \\ 1 & 0 \end{bmatrix} x \geq \begin{bmatrix} 0 & 1 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ -3.5 \\ 0 \\ 0 \\ 2 \end{bmatrix}$$

$$\implies \begin{bmatrix} 0 & -1 \end{bmatrix} x \geq -1.5$$

$\alpha_1 = \max\{v^1 A^{2,1}_{\cdot 1}, v^2 A^{2,2}_{\cdot 1}\}$

$= 0$

$\alpha_2 = \max\{v^1 A^{2,1}_{\cdot 2}, v^2 A^{2,2}_{\cdot 2}\}$

$= -1$

$$\alpha = \begin{bmatrix} \alpha_1 \\ \alpha_2 \end{bmatrix}$$

$$= \begin{bmatrix} 0 \\ -1 \end{bmatrix}$$

$\beta = \min\{v^1 b^{2,1}, v^2 b^{2,2}\}$

$= \min\{-1, -1.5\} = -1.5$

This yields the cut $-x_2 \geq -1.5$*, which is plotted in Figure 2.4.*

2.2.2 Strength

A natural question arises when generating a PDI for a disjunction $\{\mathcal{X}^t\}_{t\in T}$ applied at feasible-region data $(A, b) \in \Omega_A \times \Omega_b$: does the resulting inequality support the disjunctive hull $\mathcal{P}_D(A, b)$?

Definition 2.2.7. *The inequality (α, β) valid for a polyhedron $\mathcal{P}$* supports *$\mathcal{P}$ if there exists $x \in \mathcal{P}$ such that $\alpha^\top x = \beta$.*

Although disjunctive cuts generated directly via the CGLP or PRLP support the disjunctive hull, inequalities obtained by reusing their Farkas certificates across an admissible parametric family need not preserve this property. We characterize conditions under which the procedure of Theorem 2.2.4 yields PDIs that continue to support the disjunctive hull in Lemma 2.2.8.

Lemma 2.2.8. *Let $(A, b), (A^{'}, b^{'}) \in \Omega_A \times \Omega_b$; let $\{v^t\}_{t\in T}$ be a Farkas certificate induced by (A, b) and disjunction $\{\mathcal{X}^t\}_{t\in T}$; let $\{(\alpha(A, b), \beta(A, b))\}_{(A,b)\in\Omega_A\times\Omega_b}$ be the associated family of Farkas PDIs; and let B^t be the basis determining v^t for all $t \in T$. Then $(\alpha(A', b'), \beta(A', b'))$ supports $\mathcal{P}_D(A', b')$ if*

1. *$\{v^t\}_{t\in T}$ is induced by $(A^{'}, b^{'})$ and $\{\mathcal{X}^t\}_{t\in T}$, and*
2. *B^t is feasible for $\mathcal{Q}^t(A', b')$ for all $t \in T$.*

Proof. Let $t' = \arg\min_{t\in T}\{v^t b^{'t}\}$. Our hypothesis provides

$$v^t A^t = v^t A^{'t} = \alpha(A', b') \qquad \text{for all } t \in T,$$

so

$$\max_{t\in T}\{v^t A^{'t}\} = v^{t'} A^{'t'}.$$

Applying Theorem 2.2.4 yields

$$(\alpha(A', b'), \beta(A', b')) = \left(\max_{t\in T}\{v^t A^{'t}\}, \min_{t\in T}\{v^t b^{'t}\}\right) = (v^{t'} A^{'t'}, v^{t'} b^{'t'}),$$

a valid inequality for $\operatorname{cl}\operatorname{conv}(\cup_{t\in T}\mathcal{Q}^t(A',b'))$. Assumptions on B^t imply

$$\begin{aligned}\alpha(A',b') &= v^{t'}_{B^t}A'^{t'}_{B^t\cdot},\\ v^{t'}_{B^t} &= \alpha(A',b')(A'^{t'}_{B^t\cdot})^{-1},\\ v^{t'}_{B^t}b'^{t'}_{B^t} &= \alpha(A',b')(A'^{t'}_{B^t\cdot})^{-1}b'^{t'}_{B^t},\end{aligned}$$

and

$$(A'^{t'}_{B^t\cdot})^{-1}b'^{t'}_{B^t} \in \mathcal{Q}^t(A',b').$$

Since $\beta(A',b') = v^{t'}_{B^t}b'^{t'}_{B^t}$, the inequality $(\alpha(A',b'),\beta(A',b'))$ supports $\mathcal{Q}^t(A',b')$. Because

$$\mathcal{P}_D(A',b') = \operatorname{cl}\operatorname{conv}\left(\bigcup_{t\in T}\mathcal{Q}^t(A',b')\right)$$

is convex, $(\alpha(A',b'),\beta(A',b'))$ supports $\mathcal{P}_D(A',b')$ as well. □

This result clarifies when generated cuts support the disjunctive hull, helping to interpret unexpected computational outcomes and rule out cut strength as a factor. Notably, this excludes cases where the coefficient matrix is perturbed: as shown in Figure 2.5, such perturbations can produce cuts that fail to support the disjunctive hull. While this complicates analysis, it remains practically useful, as small changes tend to yield similar, nearly tight cuts. We also exclude cases where a basis from the Farkas certificate becomes infeasible. In practice, this often results from an infeasible term, which can be dropped from the disjunction without compromising the tightness of the parametric cut. In rarer cases, the basis becomes infeasible while the overall problem remains feasible; like matrix perturbations, this typically results in a cut that is still nearly tight for small changes. We build on these insights to identify conditions under which PDIs reliably separate the LP relaxation optimal solution from the disjunctive hull.

2.2.3 Sufficient Conditions for Separation

We first provide sufficient conditions under which a PDI separates two basic solutions. We then extend this reasoning to establish sufficient conditions for separating the LP-relaxation

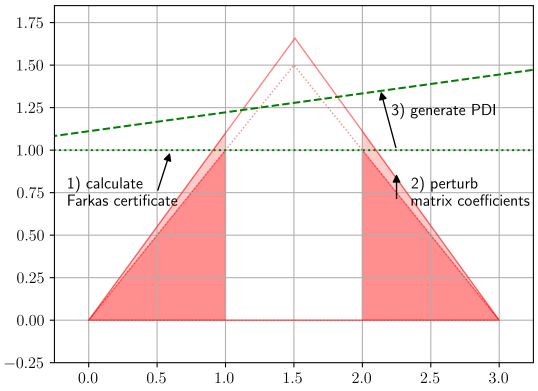


Figure 2.5: PDIs can lose disjunctive hull support under matrix perturbation.

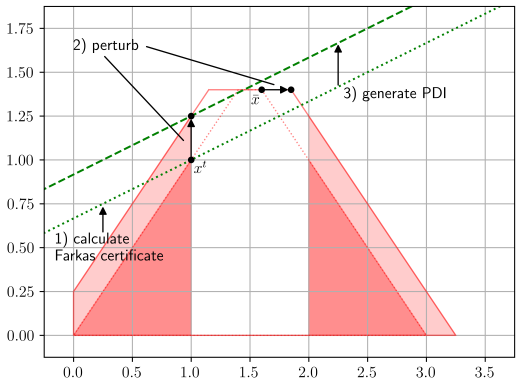


Figure 2.6: PDIs may not separate basic solutions more than one pivot from binding vertex.

solution from the disjunctive hull.

Lemma 2.2.9. *Let $(A, b, c) \in \Omega$ and $t \in T$ index a term of disjunction $\{\mathcal{X}^t\}_{t\in T}$. Let B and B' be active constraint index sets of the term relaxation $\mathcal{Q}^t(A, b)$ and let $x, x' \in \mathbb{R}^n$ be corresponding basic solutions. Let $v^t \in {\mathbb{R}_{\geq 0}}^{n+q+q_t}$ be a term of a Farkas certificate induced by $(\{\mathcal{X}^t\}_{t\in T}, (A, b, c))$ and let $(\alpha, \beta) := ({v^t}^\top A^t, {v^t}^\top b^t) \in \mathbb{R}^n \times \mathbb{R}$ represent an inequality. If*

1) $\emptyset \neq B' \setminus B \subseteq \{i \in [q + q_t] : A^t_{i.}x' = b^t_i \geq A^t_{i.}x\}$,

2) $\{i \in [n + q + q_t] : v^t_i > 0\} \subseteq B'$,

3) and there exists $i \in B' \setminus B$ such that $v^t_i > 0$ and $A^t_{i.}x' = b^t_i > A^t_{i.}x$,

Then (α, β) separates x and x'.

Proof. By hypothesis, for all $i \in B' \setminus B$, we have that $A^t_{i.}x' = b^t_i \geq A^t_{i.}x$, implying $A^t_{i.}x - A^t_{i.}x' \leq 0$. Additionally, for all $i \in B' \cap B$, we have that $A^t_{i.}x' = b^t_i = A^t_{i.}x$, implying $A^t_{i.}x - A^t_{i.}x' = 0$. Since $v^t_i \geq 0$ for all $i \in B'$ and there exists $i \in B'$ such that $v^t_i > 0$ and $A^t_{i.}x' = b^t_i > A^t_{i.}x$, we have the following system of implied inequalities:

$$0 > \sum_{i\in B'\cap B} v^t_i(A^t_{i.}x - A^t_{i.}x') + \sum_{i\in B'\setminus B} v^t_i(A^t_{i.}x - A^t_{i.}x') = \sum_{i\in B'} v^t_i(A^t_{i.}x - A^t_{i.}x').$$

By hypothesis, $v_i^t = 0$ for all $i \in [n+q+q_t] \setminus B'$, thus

$$= \sum_{i=1}^{n+q+q_t} v_i^t (A_{i.}^t x - A_{i.}^t x') = \sum_{i=1}^{n+q+q_t} v_i^t A_{i.}^t (x - x') = \alpha^\top (x - x')$$

.

This implies $\alpha^\top x < \alpha^\top x' = \beta$, which is the definition of separation. □

This result tells us that if a cut is a non-negative linear combination of constraints defining one basic solution, and none of those constraints are strictly satisfied by another basic solution—while at least one is violated and has positive weight—then the cut separates the two. This structural result underpins the separation guarantee established in Theorem 2.2.10 for LP-relaxation solutions.

Theorem 2.2.10. *Let $(A, b, c) \in \Omega$. Let B be the active constraint set for a basic solution $x \in \mathcal{P}(A, b)$. Let $\{v^t\}_{t \in T}$ be a Farkas certificate, let $\{\mathcal{X}^t\}_{t \in T}$ be a disjunction, and let $\{B^t\}_{t \in T}$ be the collection of active constraint sets yielding $x^t \in \mathcal{Q}^t(A, b)$ such that $x^t \neq x$ for all $t \in T$. Let (α, β) be a Farkas PDI evaluated at (A, b, c). If*

1) $(\{\mathcal{X}^t\}_{t \in T}, (A, b, c))$ induce $\{v^t\}_{t \in T}$,

2) x_j is binary for all $j \in \mathcal{I}$,

3) $i \in [q + q_t] \setminus [q]$ (i.e. indexes a branching constraint in the term formulation) for all $i \in B^t \setminus B$ and $t \in T$,

4) $\{B^t\}_{t \in T}$ determines $\{v^t\}_{t \in T}$,

5) there exists

$$i \in B^t \setminus B$$

such that $v_i^t > 0$, $A_{i.}^t x^t = b_i^t > A_{i.}^t x$, and $t = \arg\min_{t \in T} \{v^t b^t\}$,

Then (α, β) separates x from $\mathcal{P}_D(A, b)$.

Proof. Since $\mathcal{P}_D(A,b) \subseteq \operatorname{cl}\operatorname{conv}(\cup_{t\in T}\mathcal{Q}^t(A,b))$, it is sufficient to show (α,β) separates x from $\operatorname{cl}\operatorname{conv}(\cup_{t\in T}\mathcal{Q}^t(A,b))$ by showing (α,β) is valid for $\operatorname{cl}\operatorname{conv}(\cup_{t\in T}\mathcal{Q}^t(A,b))$ while violating x.

Let $t \in T$. By hypothesis, $\alpha = A^t v^t$ and $v^t b^t \geq \beta$. Since $\{i \in [n+q+q_t] : v^t_i > 0\} \subseteq B^t$ and $v^t \geq 0$, $A^t v^t = A^t_{B^t\cdot} v^t_{B^t}$ and $v^t b^t = v^t_{B^t} b^t_{B^t}$, implying $\alpha^\top x \geq \beta$ for all $x \in \mathcal{Q}^t(A,b)$. Since t is arbitrary, it follows $\alpha^\top x \geq \beta$ for all $x \in \operatorname{cl}\operatorname{conv}(\cup_{t\in T}\mathcal{Q}^t(A,b))$.

Consider $t = \arg\min_{t\in T}\{v^t b^t\}$. By hypothesis, $\{i \in [n+q+q_t] : v^t_i > 0\} \subseteq B^t$ and there exists $i \in B^t \setminus B$ such that $v^t_i > 0$, $A^t_{i\cdot}x^t = b^t_i > A^t_{i\cdot}x$. Let $i \in B^t \setminus B$. Since i indexes a branching constraint on a binary variable x_j, $A^t_{i\cdot}x = x_j \geq 1$ or $A^t_{i\cdot}x = -x_j \geq 0$ for all $x \in \mathcal{Q}^t(A,b)$. Given $0 \leq x_j \leq 1$ and $x^t \in \mathcal{Q}^t(A,b)$, $A^t_{i\cdot}x^t = b^t_i > A^t_{i\cdot}x$. i being arbitrary implies $B^t \setminus B \subseteq \{i \in [q+q_t] : A^t_{i\cdot}x^t = b^t_i \geq A^t_{i\cdot}x\}$. Applying Lemma 2.2.9 yields $\alpha^\top x < \beta$. □

Many of the above conditions can often be guaranteed throughout a MILP series in practice. For example, 1) holds when the coefficient matrices remain fixed and 2) when integrality constraints are binary. Conditions 3) and 4) are ensured when Farkas certificates are drawn from basis cones differing from B only by branching constraints. In such cases, only 5) remains: the basis cone supporting (α,β) must include a branching constraint that both violates the LP-relaxation solution and appears with positive weight in the cut's generating combination.

If this occurs for a disjunctive cut generated by the CGLP or PRLP, then by Theorem 2.2.10, the parametric form of the cut will continue to separate the same LP-relaxation solution from the disjunctive hull, provided the problem changes only slightly. If that solution remains optimal, the separation guarantee extends accordingly. Since the LP-relaxation solution often approximates the MILP optimum, it is typically the primary target for cutting planes. Thus, when this solution stays within the necessary bounds throughout the series, our PDIs will consistently separate it from the disjunctive hull and meaningfully tighten the relaxation.

Given the prevalence of binary integrality constraints and decomposition methods that fix coefficient matrices, this situation arises frequently for mildly perturbed series. Still, in

practice, one might relax these assumptions by allowing Farkas certificates with support beyond branching and initially active constraints, as illustrated in Figure 2.6. Though this weakens strict separation guarantees, it can yield deeper cuts that often still separate the LP-relaxation solution. Even when they don't, such cuts may tighten the relaxation and improve performance deeper in the branch and bound tree. For more, see Balas and Kazachkov [12].

2.3 Computational Results

The overarching goal of our experiments was to compare the effect generating disjunctive cuts in different ways has on various performance metrics when solving a series of related MILPs. This section is divided by the setup for the experiments (Section 2.3.1) and their results (Sections 2.3.2, 2.3.3, and 2.3.4).

2.3.1 Computational Setup

Before conducting our experiments, we generated series of randomly perturbed MILP instances from MIPLIB 2017 and implemented four disjunctive cut generators with varying degrees of parameterization. We then tested these generators on a server cluster using existing MILP solvers to produce the data analyzed in the following sections. This setup ensures a robust evaluation of disjunctive cuts' impact on solver performance.

Test Instances. The MILP instances used in our experiments consist of two groups: a set of *base instances*, indexed by $\mathcal{B}$, and a collection of *test instances*, indexed by $\mathcal{T}$. Base instances are drawn from the MIPLIB 2017 dataset and include those with at most 5,000 variables and 5,000 constraints after presolving. Test instances, subindexed by $(k, u, \theta) \in \mathcal{B} \times \{A, b, c\} \times \{.5, 2\}$, are derived from a base instance k using Algorithm 6(u^k, θ). These instances are created by perturbing c^k if $u = c$, b^k if $u = b$, or A^k if $u = A$, up to a specified *degree* θ. A test instance $\ell \in \mathcal{T}_{ku\theta}$ is considered to be within θ degree(s) of perturbation of the base instance k if Algorithm 5(u^k, u^ℓ) returns a value no greater than θ. For each $(k, u, \theta) \in \mathcal{T}$, Algorithm 6$(u^k, \theta)$ is executed iteratively, stopping after one of the following

conditions is met: 1000 attempts, 4 hours, or the successful generation of three test instances (unless specified otherwise). Consequently, the number of test instances generated for each base instance varies.

Algorithm 6 employs two black-box subroutines: `toVector` and `toMatrix`. The former converts a matrix to a vector, while the latter restores the resulting vector to the shape of the original matrix.

Algorithm 5 `Find Degree`

Require: u^k, u_{new}

angle_difference $\leftarrow \cos^{-1}\left(\frac{u^k \cdot u_{\text{new}}}{\|u^k\|\|u_{\text{new}}\|}\right)$ ▷ Angle between vectors

norm_difference $\leftarrow \left|\frac{\|u^k\| - \|u_{\text{new}}\|}{\|u^k\|}\right|$ ▷ Relative change in vector norms

return max{angle_difference, norm_difference}

Algorithm 6 `Find Perturbation`

Require: u^k, θ ▷ starting vector, desired perturbation

$u^k \leftarrow$ `toVector`(u^k) if $u^k \in \mathbb{R}^{q \times n}$ else u^k ▷ Flatten to vector if needed

$u_{\text{final}}, \epsilon \leftarrow$ NULL, 1

while $\epsilon \geq 10^{-6}$ and $u_{\text{final}} ==$ NULL **do** ▷ until tolerance met or perturbed vector found

  $u_{\text{new}}, u_{\text{prev}} \leftarrow u^k$, NULL

  while `Find Degree`$(u^k, u_{\text{new}}) < \theta$ **do** ▷ Until desired perturbation reached

    $u_{\text{prev}} \leftarrow u_{\text{new}}$

    $i \leftarrow$ `random`$([|u|])$ ▷ Select random element

    $u_{\text{new}}[i] \leftarrow u_{\text{new}}[i] +$ `random`$([-\epsilon, \epsilon])$ ▷ Randomly perturb by $\pm\epsilon$

  end while

  if $u_{\text{prev}} \neq u^k$ **then** ▷ If an iteration produced a vector within the desired perturbation

    $u_{\text{final}} \leftarrow u_{\text{prev}}$

  end if

  $\epsilon \leftarrow \epsilon/2$

end while

return `toMatrix`(u_{final}) if $u_{\text{final}} \in \mathbb{R}^{qn}$ else u_{final} ▷ Return as matrix if needed

Implemented Disjunctive Cut Generators. Our experiments explore variations in how disjunctive cuts are generated. We use d to denote the number of disjunctive terms in the branch and bound queue at the point when a disjunction is generated from the leaves of the solve's branch and bound tree. For each combination of $d \in \{4, 16, 64\}$ and $(k, u, \theta) \in \mathcal{T}$, we evaluate four approaches to generating disjunctive cuts. Algorithm 7

(`Default`) tests solving the series $\mathcal{T}_{ku\theta}$ using default solver settings without disjunctive cuts. Algorithm 8 (`Calculated Disjunction, Calculated Cuts`) follows the disjunctive cut generation method prescribed in Balas and Kazachkov [12, Algorithm 1]. Algorithm 9 (`Parametric Disjunction, Calculated Cuts`) and Algorithm 10 (`Parametric Disjunction, Parametric Cuts`) apply disjunctive cuts with varying levels of parameterization to solve the series. The difference between `Param Disj, Calc Cuts` and `Param Disj, Param Cuts` is that the latter generates cuts as described in Theorem 2.2.4 while the former recylces a previously found disjunction when solving PRLPs.

Algorithm 7 `Default`

Require: $k, u, \theta, \mathcal{T}$

1: **for all** $\ell \in \mathcal{T}_{ku\theta}$ **do**
2: `Branch-and-Cut` (MILP-ℓ) ▷ Solve each test instance under default settings
3: **end for**

Algorithm 8 `Calculated Disjunction, Calculated Cuts`

Require: $d, k, u, \theta, \mathcal{T}$

1: **for all** $\ell \in \mathcal{T}_{ku\theta}$ **do**. ▷ For each test instance
2: $\{\mathcal{X}^t\}_{t\in T} \leftarrow$ `Branch-and-Bound` (MILP-ℓ, d) ▷ Generate a new disjunction
3: $T^\ell_{\text{feas}} \leftarrow \{t \in T : \mathcal{Q}^{\ell t} \neq \emptyset\}$ ▷ Using feasible terms only
4: $\Pi^\ell \leftarrow$ Balas and Kazachkov [12, Algorithm 1](MILP-ℓ, $\{\mathcal{X}^t\}_{t\in T^\ell_{\text{feas}}}$) ▷ Get cuts from PRLPs
5: `Branch-and-Cut` (MILP-ℓ, Π^ℓ) ▷ Solve with disjunctive cuts
6: **end for**

Algorithm 9 `Parametric Disjunction, Calculated Cuts`

Require: $d, k, u, \theta, \mathcal{T}$

1: $\{\mathcal{X}^t\}_{t\in T} \leftarrow$ `Branch-and-Bound` (MILP-k, d) ▷ Generate initial disjunction to parameterize
2: **for all** $\ell \in \mathcal{T}_{ku\theta}$ **do**. ▷ For each test instance
3: $T^\ell_{\text{feas}} \leftarrow \{t \in T : \mathcal{Q}^{\ell t} \neq \emptyset\}$ ▷ Using feasible terms only
4: $\Pi^\ell \leftarrow$ Balas and Kazachkov [12, Algorithm 1](MILP-ℓ, $\{\mathcal{X}^t\}_{t\in T^\ell_{\text{feas}}}$) ▷ Get cuts from PRLPs
5: `Branch-and-Cut` (MILP-ℓ, Π^ℓ) ▷ Solve with disjunctive cuts
6: **end for**

Algorithms 7 - 10 use the subroutines `Branch-and-Cut` and `Branch-and-Bound`. The routine `Branch-and-Cut` solves a MILP using a standard solver (e.g., CBC or Gurobi) with default settings unless specified otherwise. Its inputs are the instance MILP and, optionally,

Algorithm 10 `Parametric Disjunction, Parametric Cuts`

Require: $d, k, u, \theta, \mathcal{T}$

$\{\mathcal{X}^t\}_{t\in T} \leftarrow$ `Branch-and-Bound` (MILP-k, d) ▷ Generate initial disjunction to parameterize
$T^k_{\text{feas}} \leftarrow \{t \in T : \mathcal{Q}^{kt} \neq \emptyset\}$ ▷ Using feasible terms only
$\Pi^k \leftarrow$ Balas and Kazachkov [12, Algorithm 1](MILP-k, $\{\mathcal{X}^t\}_{t\in T^k_{\text{feas}}}$) ▷ Get base cuts
$\mathcal{V} \leftarrow \emptyset$
for all $(\alpha^k, \beta^k) \in \Pi^k$ **do**. ▷ For each cut
    $\{v^t\}_{t\in T} \leftarrow \{0_{q+q_t}\}_{t\in T}$
    for all $t \in T$ such that $\mathcal{Q}^{kt} \neq \emptyset$ **do**. ▷ For each feasible term
        $B^t \leftarrow B$ such that $A^{kt}_{B\cdot}\bar{x}^{kt} = b^{kt}$ ▷ Get term's optimal basis
        $v^t \leftarrow$ Lemma 2.1.2(MILP-k, $A^{kt}_{B^t\cdot}, \alpha, \beta$) ▷ Calculate its certificate
    end for
    $\mathcal{V} \leftarrow \mathcal{V} \cup \{v^t\}_{t\in T}$
end for
for all $\ell \in \mathcal{T}_{ku\theta}$ **do** ▷ For each test instance
    $\Pi^\ell \leftarrow \emptyset$
    $T^\ell_{\text{feas}} \leftarrow \{t \in T : \mathcal{Q}^{\ell t} \neq \emptyset\}$ ▷ Using feasible terms only
    for all $\{v^t\}_{t\in T} \in \mathcal{V}$ **do** ▷ For each certificate
        $(\alpha^\ell, \beta^\ell) \leftarrow$ Theorem 2.2.4(MILP-ℓ, $\{\mathcal{X}^t\}_{t\in T^\ell_{\text{feas}}}, \{v^t\}_{t\in T^\ell_{\text{feas}}}$) ▷ Generate PDI
        $\Pi^\ell \leftarrow \Pi^\ell \cup \{(\alpha^\ell, \beta^\ell)\}$
    end for
    `Branch-and-Cut` (MILP-ℓ, Π^ℓ) ▷ Solve with disjunctive cuts
end for

$d \in \mathbb{N}$, the number of queued nodes for termination; $\{\mathcal{X}^t\}_{t\in T} \subseteq \mathbb{R}^n$, a starting disjunction such that $\bigcap_{t\in T} \mathcal{X}^t = \emptyset$; and $\Pi \subseteq \mathbb{R}^{n+1}$, an initial pool of cutting planes. It outputs the terminating disjunction $\{\mathcal{X}^{t\bar{}}\}_{t\in T} \subseteq \mathbb{R}^n$. The routine `Branch-and-Bound` is similar, but disables cut generation and omits the initial cutting-plane pool. Its configuration follows Balas and Kazachkov [12, Appendix C].

Algorithms 7 - 10 require several parameter choices. For each $d \in \{4, 16, 64\}$, a disjunction $\{\mathcal{X}^t\}_{t\in T}$ generated from partially solving MILP-$k$includes the d queued terms, the infeasible terms, and terms implicitly added by tightening during strong branching. While Balas and Kazachkov [12] exclude pruned terms in their experiments, we must compute Farkas certificates for all terms because perturbations can invalidate a term's pruning. Consequently, we include all feasible terms in the PRLP and set the Farkas certificate for infeasible terms to 0, since Lemma 2.1.2 requires MILP-kt to be feasible for $t \in T$. When applying Theorem 2.2.4 to ℓ, $\mathcal{X}$, and $\{v^t\}_{t\in T}$, we disregard terms $t \in T$ for which $\mathcal{Q}^{\ell t} = \emptyset$.

The number of disjunctive cuts generated from PRLPs does not exceed the number of fractional integer variables in the root LP relaxation. Each PRLP is solved with a sequence of carefully chosen objective functions that produces a single round of rank-one cuts. These cuts are valid for the same disjunction and are generated nonrecursively. For further details, see [12].

Experimental Setup. Our experiments relied on the following computing resources. We used the C++ library VPC [3] to generate disjunctions and disjunctive cuts from PRLPs, creating a fork so that disjunctions would be lattice free. We used Gurobi 10 as the `Branch-and-Cut` solver, with a callback to add our cuts at the root node, and CBC 2.10 for `Branch-and-Bound`. The experiments were conducted on a server cluster with 16 AMD Opteron Processor 6128s; 2 CPUs and up to 15GB of RAM were dedicated to each experiment. We allowed 3600 seconds both for generating disjunctive cuts and for solving each MILP instance.

We use these resources to evaluate disjunctive cuts' impact on Gurobi, starting with a high-performing subset (Section 2.3.2) before expanding our analysis to assess broader performance trends across related MILPs and varying input parameters. For each test instance, we evaluate performance using two approaches, leading to several metrics categorized as follows: (1) *Root node performance*: Measures the strength of disjunctive cuts and the time required to generate them. (2) *branch and cut performance*: Reports the time and number of nodes processed to reach optimality. Root node and branch and cut performance are analyzed in detail in subsections Section 2.3.3 and Section 2.3.4, respectively.

2.3.2 High Performing Subset of Experiments

Our experiments in Sections 2.3.3 and 2.3.4 are motivated by two key findings on selected MILP series: PDIs can significantly reduce total solve time, with greater improvement as parameterization increases (Table 2.1), and test instance solve time reductions tend to grow with the number of disjunctive terms used for cut generation (Table 2.2).

Table 2.1 is structured to highlight this first point. Column 1 represents the perturbation "Degree" ($\theta \in \{.5, 2\}$), while Column 2 indicates the number of "Terms" ($d \in \{4, 16, 64\}$)

Table 2.1: Solve time (s) for representative series with substantial improvement from disjunctive cut generation.

Degree	Terms	Type	Base Instance	Default	Calc Disj Calc Cuts	Param Disj Calc Cuts	Param Disj Param Cuts	Series Length
0.5	4	matrix	neos-631517	1282	1785	1081	855	11
	16	objective	arki001	1408	508	388	337	11
	16	rhs	tr12-30	2471	1797	1793	1652	10
2.0	64	matrix	tr12-30	431	455	411	397	11
	16	objective	neos-631517	3880	6813	4102	2758	9
	64	rhs	neos-631517	383	500	265	119	5

Table 2.2: For each combination of perturbation degree and disjunctive terms, average solve time (in seconds) for each of 372 bm23 test instances excluding the time to generate disjunctive cuts.

Degree	Terms	Default	Calc Disj Calc Cuts	Param Disj Calc Cuts	Param Disj Param Cuts
0.5	4	0.202	0.184	0.181	0.170
	16	0.187	0.174	0.167	0.166
	64	0.188	0.140	0.140	0.159
2.0	4	0.179	0.178	0.174	0.175
	16	0.172	0.166	0.160	0.162
	64	0.177	0.137	0.132	0.142

queued in a partial branch and bound solve when generating a disjunction. Column 3 specifies the "Type" of perturbation differentiating test instances from their base instance, and Column 4 lists the base instance name from MIPLIB 2017. Columns 5 through 8 tell us for given degree θ, number of disjunctive terms d, type of perturbation u, and base instance k, the total amount of time (in seconds) to solve base instance k plus all test instances in $\mathcal{T}_{ku\theta}$ for each of our four disjunctive cut generation algorithms. Column 9 tells us the number of instances in the series, including the base instance.

The rows of Table 2.1 were selected to illustrate key observations. First, despite the complexity of generating disjunctive cuts by solving PRLPs for the base instance in `Calc Disj, Calc Cuts`, `Param Disj, Calc Cuts`, and `Param Disj, Param Cuts`, total series solve time can be significantly reduced compared to `Default`, where no disjunctive cuts are used. Greater improvements are achieved when parameterizing disjunctive cuts (`Param Disj, Calc Cuts` and `Param Disj, Param Cuts`). Additionally, reusing both Farkas multipliers and the disjunction that generated previous cuts (`Param Disj, Param Cuts`) yields

even better results compared to reusing only the disjunction (`Param Disj, Calc Cuts`).

This subset was chosen to show that these improvements are not limited to a specific "right" combination of parameters. Table 2.1 demonstrates that substantial reductions in total solve time occur across a variety of perturbation degrees, perturbation types, disjunctive term counts, and base instances. These results align with our expectations: disjunctive cuts, particularly the disjunctive cuts used in this study, are exceptionally strong. If their generation cost can be amortized across a series of related MILPs, total solve time can be significantly reduced. Since this impact can be substantial, improvements are achievable across a wide range of parameter settings.

Relationship Between Solve Time and the Number of Disjunctive Terms. We structure Table 2.2 to illustrate the second point. This table is indexed by its first two columns, which have the same meaning as in Table 2.1. Columns 3 through 6 report the average solve time (in seconds) for each test instance in $\mathcal{T}_{ku\theta}$, given degree θ, disjunctive terms d, and base instance bm23. These values are averaged across all perturbation types and *exclude disjunctive cut generation time* for each of the four disjunctive cut generation algorithms.

This table highlights the relationship between test instance solve time and the number of disjunctive terms used to generate cuts, showing that solve time tends to decrease as more terms are added—at least when excluding disjunctive cut generation time. This trend aligns with expectations, as additional disjunctive terms produce stronger cuts that can accelerate solving. Since this effect is independent of perturbation degree, we anticipate the pattern observed in Table 2.2, where solve time (excluding disjunctive cut generation) improves consistently with more disjunctive terms, regardless of perturbation degree. Additionally, we expect that as test instances deviate further from the base instance (i.e., with higher perturbation degrees), parameterized cuts become less effective at refining the LP relaxation near the optimal solution. This diminished effectiveness is generally reflected in Table 2.2.

Table 2.3: The number of base and test instances where VPC generation succeeds for all combinations of possible degree of perturbation and terms.

Type	Base Instances	Test Instances
matrix	42	103
objective	101	361
rhs	29	75

2.3.3 Root Node Performance

The next subsections evaluate the strength and generation time of disjunctive cuts across our four generators, showing that less parameterized cuts tend to be stronger, while more parameterized ones generate faster. These trends become more pronounced as the number of disjunctive terms increases.

For consistency, only test instances for which disjunctive cuts are generated for all combinations of possible degree of perturbation and terms are considered when evaluating root node performance, counts of which are detailed in Table 2.3. Given a time limit on VPC generation and that many pruned, feasible terms can be added to disjunctions when solving a PRLP, some test instances hit their time limit for `Calc Disj, Calc Cuts` and `Param Disj, Calc Cuts` resulting in their exclusion. Further, fewer base instances were tested for matrix and constraint bound (rhs) perturbations than base instances as our method for creating test instances often resulted in infeasibility when perturbing the feasible region. Table 2.3 breaks down the total number of base and test instances considered for each type of perturbation. For reference, 104 MIPLIB 2017 base instances meet our requirements of at most 5000 presolved rows and columns.

Root-Node Dual-Bound Performance. To measure the strength of disjunctive cuts, we calculate the amount of integrality gap they close (defined in appendix A) at the root node. We look at this from two perspectives. The first way is to look at the gap closed by disjunctive cuts and their generating disjunctions alone. The results of this can be seen in Table 2.4. The second way is to look at the gap closed by disjunctive cuts when combined with the default cuts CBC generates at the root node. The results are in Table 2.5.

In Table 2.4, column 1 represents the "Degree" ($\theta \in \{.5, 2\}$) of perturbation, while

Table 2.4: Average percent integrality gap closed by disjunctive cuts and implied by disjunctions, grouped by degree of perturbation, size of disjunction, and type of perturbation.

Degree	Terms	Type	Calc Disj	Param Disj	Calc Disj Calc Cuts	Param Disj Calc Cuts	Param Disj Param Cuts
0.5	4	matrix	6.09	4.99	4.07	3.48	3.42
		objective	5.49	4.92	3.90	3.72	3.42
		rhs	5.29	3.94	3.10	3.02	2.61
	16	matrix	11.09	9.41	6.11	6.06	4.99
		objective	9.52	8.64	5.62	5.37	4.70
		rhs	9.37	7.40	5.89	5.73	4.65
	64	matrix	16.83	13.94	8.59	8.32	6.30
		objective	14.39	12.65	8.46	7.59	6.39
		rhs	13.72	10.52	9.06	7.63	6.25
2.0	4	matrix	9.25	4.79	4.85	3.04	2.07
		objective	6.13	4.69	4.32	3.54	3.11
		rhs	12.84	3.51	3.33	2.77	1.51
	16	matrix	16.13	7.99	7.43	5.10	1.83
		objective	10.71	7.81	6.05	5.17	3.93
		rhs	16.75	6.64	5.85	4.81	2.64
	64	matrix	22.50	10.66	10.46	6.41	1.48
		objective	15.78	11.50	8.98	7.15	5.41
		rhs	21.42	9.31	8.47	6.52	3.34

column 2 indicates the number of "Terms" ($d \in \{4, 16, 64\}$) queued in a partial branch and bound solve when a disjunction is generated. Columns 3 and 4 show the average integrality gap closed by the disjunctions used to generate our disjunctive cuts, where `Calc Disj` corresponds to `Calc Disj, Calc Cuts`, and `Param Disj` corresponds to `Param Disj, Calc Cuts` and `Param Disj, Param Cuts`. The final three columns present the average integrality gap closed at the root node, considering only the disjunctive cuts added according to the specified disjunctive cut generator.

The trends amongst disjunctive cuts and their generating disjunctions regarding integrality gap closed at the root node in Table 2.4 are to be expected. First, as the size of the disjunctions used grow, the integrality gap closed by disjunctive cuts and their generating disjunctions improves. This is unsurprising as the disjunctions are based off of partial branch and bound trees, and the dual bound improves monotonically as branch and bound progresses. Second, the disjunctions (`Calc Disj`, `Param Disj`) close at least

as much gap as the cuts they generate (`Calc Disj, Calc Cuts`, `Param Disj, Calc Cuts`, `Param Disj, Param Cuts`), which follows from a property of disjunctive cut generation. Third, disjunctions generated in `Calc Disj` close more integrality gap than disjunctions generated in `Param Disj`, and the same relationship exists between the cuts they generate (`Calc Disj, Calc Cuts` and `Param Disj, Calc Cuts`, respectively). For a well-tuned solver like Gurobi, this is again a predictable result as changes to a MILP instance often result in a solver applying different branching decisions in an attempt to more efficiently close the integrality gap. Fourth, differences in integrality gap closed between `Calc Disj` and `Param Disj` as well as between `Calc Disj, Calc Cuts` and `Param Disj, Calc Cuts` both increase as degree of perturbation increased. This is to be expected as greater differences in problem structure lead to greater differences in disjunctions employed while solving them, causing a weakening of old disjunctions and farkas coefficients. Finally, we see disjunctive cuts generated from `Param Disj, Param Cuts` tend to not close as much integrality gap as disjunctive cuts generated from `Param Disj, Calc Cuts`. `Param Disj, Param Cuts` recycles Farkas certificates generating disjunctive cuts whereas `Param Disj, Calc Cuts` tailors its disjunctive cuts' Farkas certificates by solving PRLPs, which often results in stronger cuts. Further, this difference tends to be most pronounced for matrix perturbations when broken down by perturbation type. Although Lemma 2.2.8 shows PDIs can support the disjunctive hull when the coefficient matrix does not change, the result does not hold for general coefficient matrix perturbations, as shown by Figure 2.5.

The columns in Table 2.5 can be understood as follows. "Degree" and "Terms" take the same meaning as in Table 2.4. The columns `Default`, `Param Disj, Param Cuts`, and `Calc Disj, Calc Cuts` refer to the integrality gap Gurobi closed at the root node with its default cut generators plus disjunctive cuts generated by the respective disjunctive cut generator.

With respect to Table 2.4, a couple of similar results are apparent in Table 2.5. Even with the addition of default cuts generated at the root node, disjunctive cuts from `Calc Disj, Calc Cuts` still close at least as much integrality gap than disjunctive cuts from `Param Disj, Param Cuts`, and, when holding perturbation constant, disjunctive cuts close more integrality gap when they are generated against more disjunctive terms. Again, there

Table 2.5: Average percent root integrality gap closed by Gurobi's default cut generators together with selected disjunctive cuts, grouped by perturbation setup.

Degree	Terms	Type	Default	Param Disj Param Cuts	Calc Disj Calc Cuts
0.5	4	matrix	70.12	70.11	70.32
		objective	59.96	60.55	60.51
		rhs	59.76	59.96	60.27
	16	matrix	70.12	70.17	70.62
		objective	59.96	61.10	61.06
		rhs	59.76	61.00	61.58
	64	matrix	70.12	70.71	70.55
		objective	59.96	61.56	61.62
		rhs	59.76	61.87	63.27
2.0	4	matrix	75.50	75.31	75.80
		objective	60.39	60.42	60.39
		rhs	61.70	61.15	61.24
	16	matrix	75.50	75.81	75.81
		objective	60.39	60.46	60.58
		rhs	61.70	61.42	62.52
	64	matrix	75.50	75.54	75.90
		objective	60.39	60.96	61.64
		rhs	61.70	62.17	64.27

appears to be an inverse relationship between the degree of perturbation and difference of integrality gap closed between `Calc Disj, Calc Cuts` to `Param Disj, Param Cuts`.

Table 2.5 also yields a couple of new trends. Instances with disjunctive cuts (`Param Disj, Param Cuts`, `Calc Disj, Calc Cuts`) often see an improvement in root integrality gap closed compared to the same instances without disjunctive cuts (`Default`). This aligns with the results recorded by Balas and Kazachkov [12] in similar experiments and could be due to the strength of disjunctive cuts or that disjunctive cuts separate a MILP's relaxation from more extrema than its optimal solution. The difference of gap closed between disjunctive cuts in `Param Disj, Param Cuts` and `Calc Disj, Calc Cuts` is much smaller when applied along with root cuts as compared to Table 2.4, which suggests that default root cuts recover some of the refined relaxation lost from using disjunctive cuts from `Param Disj, Param Cuts` instead of from `Calc Disj, Calc Cuts`. Additionally, the inverse relationship between perturbation and differences of integrality gap closed extends to `Default` when

compared to each of `Calc Disj, Calc Cuts` and `Param Disj, Param Cuts`. Lastly, the effectiveness of default cuts for matrix perturbed instances obscures if there is a performance degradation due to these cuts not supporting the disjunctive hull, which could be due to our perturbation method only generating test instances for a subset of the base instances.

Table 2.6: Average root-node processing time (in seconds) by disjunctive-cut configuration and perturbation type.

Degree	Terms	Type	Default	Param Disj Param Cuts	Param Disj Calc Cuts	Calc Disj Calc Cuts
0.5	4	matrix	0.96	0.98	3.15	33.36
		objective	1.01	1.09	8.12	5.85
		rhs	0.49	0.58	1.20	1.34
	16	matrix	0.89	1.37	8.61	56.71
		objective	1.04	1.51	20.35	26.80
		rhs	0.49	0.90	3.24	4.99
	64	matrix	0.90	2.07	47.80	34.16
		objective	1.02	2.42	67.67	70.95
		rhs	0.48	1.20	13.47	18.46
2.0	4	matrix	0.71	0.78	5.17	5.62
		objective	1.03	1.07	15.38	4.30
		rhs	0.48	0.46	0.96	2.43
	16	matrix	0.70	1.35	9.20	11.02
		objective	1.01	1.56	20.63	22.25
		rhs	0.41	0.67	2.56	4.11
	64	matrix	0.71	2.63	31.44	20.93
		objective	0.99	2.44	81.49	64.11
		rhs	0.42	1.11	10.63	13.19

Cut Generation Time. The columns in Table 2.6 are defined as follows. "Degree," "Terms," and "Type" retain their previously established meanings. The columns `Default`, `Param Disj, Param Cuts`, `Param Disj, Calc Cuts`, and `Calc Disj, Calc Cuts` represent the time (in seconds) that Gurobi requires to process the root node under each respective disjunctive cut generator.

The trends observed in Table 2.6 align with expectations. Processing time is shortest for `Default`, as it involves only a subset of the tasks performed by the other three options. `Param Disj, Calc Cuts` and `Calc Disj, Calc Cuts` take significantly longer than `Param`

`Disj, Param Cuts` since the first two solve a series of PRLPs, which the third does not. Among the two, `Param Disj, Calc Cuts` tends to be faster than `Calc Disj, Calc Cuts` because it reuses an existing disjunction rather than generating a new one through a partial branch and bound solve. Processing times for `Param Disj, Param Cuts`, `Param Disj, Calc Cuts`, and `Calc Disj, Calc Cuts` increase with the number of terms, as `Param Disj, Param Cuts` has more terms to parameterize and the PRLPs solved by `Param Disj, Calc Cuts` and `Calc Disj, Calc Cuts` grow larger. By contrast, `Default` remains relatively unaffected by changes in degree, terms, or type, with any minor variations possibly attributable to differences in the underlying test sets and problem structures.

A couple trends were expected to be absent in Table 2.6, and this was indeed the case. For instance, while generating a disjunction from a partial branch and bound tree, some perturbations add more pruned nodes than others, especially as the number of terms queued before generating a disjunction increases. This might explain the variance observed for `Param Disj, Param Cuts`, `Param Disj, Calc Cuts`, and `Calc Disj, Calc Cuts`. Furthermore, there are no consistent trends distinguishing the type of perturbation, as it does not affect the complexity of any disjunctive cut generator.

Table 2.7: Root node summary statistics by disjunctive cut generator with average optimality gap closed (in %) and processing time (in seconds).

Disjunctive Cut Generator	Gap Closed (%)	Time
`Default`	62.67	0.90
`Param Disj, Param Cuts`	63.21	1.53
`Param Disj, Calc Cuts`	63.26	27.95
`Calc Disj, Calc Cuts`	63.47	27.87

Tradeoff Between Generation Time and Strength. Table 2.7 summarizes the relationship between root node processing time and optimality gap closed for the four disjunctive cut generation methods. Column 1 identifies the cut generation method, while Columns 2 and 3 report the average integrality gap closed (percentage) and the time (seconds) required for root node processing, respectively.

As shown in Table 2.7, the relationship of optimality gap closed and processing time at the root node is nearly Pareto optimal. This is expected, as increased complexity in

the disjunctive cut generators allows for more tailored cuts to the instance, but also demands more processing time. The slight exception arises from a negligible decrease in time from `Param Disj, Calc Cuts` to `Calc Disj, Calc Cuts`, which is reasonable since the complexity lies almost entirely in solving the same series of PRLPs, whose variance may obscure the minor time increase associated with generating a new disjunction. Moreover, for small perturbations, PDIs capture the majority of the bound improvement achieved by generating disjunctive cuts from PRLPs, all while requiring only a fraction of the time.

2.3.4 Branch and Cut Performance

In the following paragraphs, we compare test instances based on the time required to reach optimality for branch and cut and, where relevant, the number of nodes processed. This analysis is conducted from two perspectives: the distribution of runtime changes relative to `Default` for the other three disjunctive cut generators, and the proportion of test instances where `Param Disj, Param Cuts` or `Default` *win*, defined as one significantly outperforming the other.

To ensure consistency, we evaluate performance using test instances solved to optimality within a time frame of 10 seconds to 1 hour, as detailed in Table 2.9. The lower limit minimizes the effects of machine variability, while the upper limit reflects our computational resource constraints. Disjunctive cut generation time is excluded from the 1-hour limit to focus on their impact on the broader branch and cut process, though it is included for holistic comparisons.

As shown in Table 2.9, test set counts generally decrease with higher degrees of perturbation and more terms, as increased perturbation often leads to infeasible instances (which are discarded), and disjunctive cut generation is more likely to exceed its time limit. Notably, the set of test instances examined in this section differs from the previous one. Many instances that are successfully perturbed and generate root cuts fail to reach optimality within 1 hour, while some that complete branch and cut within the time limit for one degree of perturbation or term count fail under different conditions. By analyzing each set independently rather than restricting to their intersection, we increase the sample size for each scenario, yielding more robust results. Each combination of degree and terms includes

at least 300 test instances solved within 1 hour.

Impact on Solve Time. One way to measure the effectiveness of disjunctive cuts is by recording the change in time to reach optimality for each test instance when run with `Param Disj, Param Cuts`, `Param Disj, Calc Cuts`, and `Calc Disj, Calc Cuts` relative to `Default`. This analysis is presented using cumulative distributions, which show the proportion of test instances achieving a specific amount of improvement or degradation. For example, a y-value at -0.25 indicates the proportion of instances solved at least 25% faster with a given disjunctive cut generator compared to `Default`, while a y-value at +0.25 indicates those solved no more than 25% slower. We compare these cumulative distributions across disjunctive cut generators, the number of terms ($d \in \{4, 16, 64\}$) queued at partial branch and bound solves where disjunctions are generated, and the degree of perturbation ($\theta \in \{.5, 2\}$).

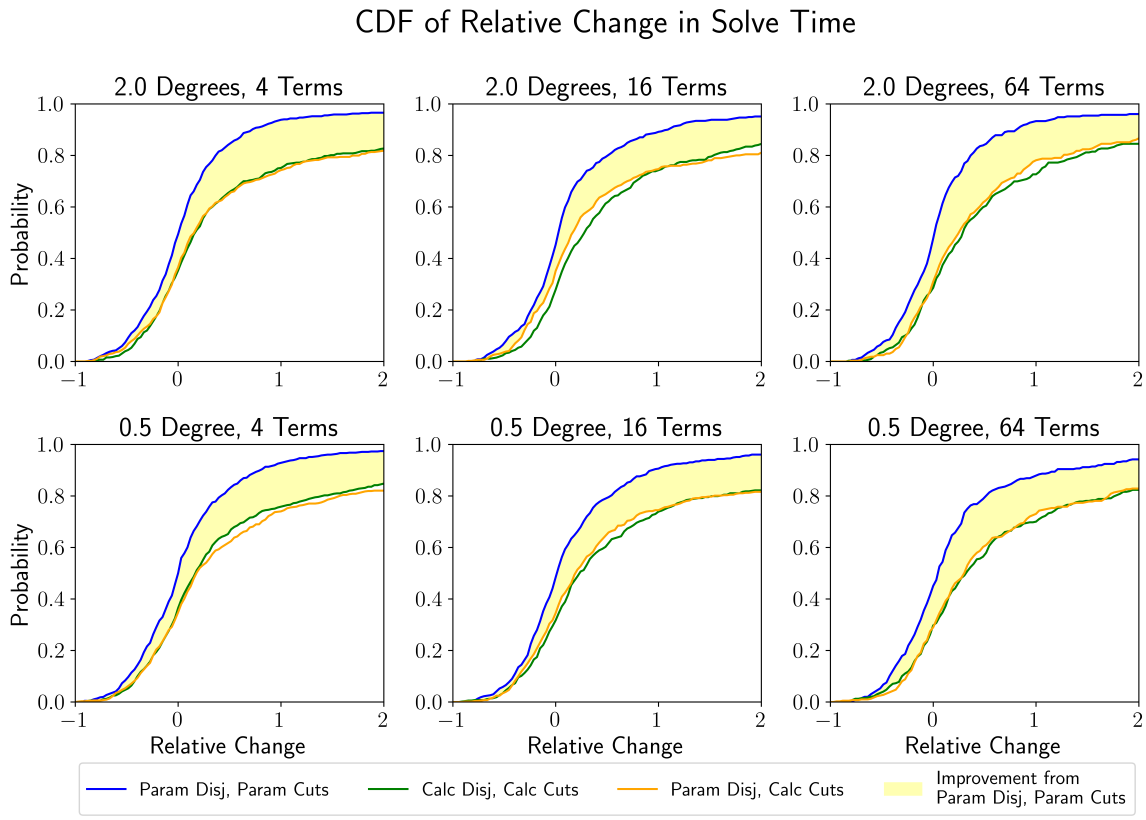


Figure 2.7: Cumulative distribution of change in run time relative to ***`Default`*** *for remaining disjunctive cut generators.* ***`Param Disj, Param Cuts`*** *advantage over* ***`Param Disj, Calc Cuts`*** *and* ***`Calc Disj, Calc Cuts`*** *highlighted in yellow.*

In Figure 2.7, columns and rows correspond to the number of disjunctive terms and the degree of perturbation, respectively. Each plot presents the cumulative distribution of runtime changes for `Param Disj, Param Cuts`, `Param Disj, Calc Cuts`, and `Calc Disj, Calc Cuts` relative to `Default`. The yellow-highlighted region shows the additional proportion of test instances where `Param Disj, Param Cuts` achieves a given improvement or better compared to both `Param Disj, Calc Cuts` and `Calc Disj, Calc Cuts`, each relative to `Default`.

The trends in Figure 2.7 align with expectations. `Param Disj, Param Cuts` achieves a comparable improvement in the optimality gap closed at the root node to `Param Disj, Calc Cuts` and `Calc Disj, Calc Cuts` but in significantly less time, leading to a higher proportion of instances reaching a given level of improvement relative to `Default`. A similar pattern emerges between `Param Disj, Calc Cuts` and `Calc Disj, Calc Cuts` as the number of disjunctive terms increases: `Param Disj, Calc Cuts` becomes more efficient while retaining most of the optimality gap closed, as shown in Table 2.8. Consequently, `Param Disj, Calc Cuts` begins to outperform `Calc Disj, Calc Cuts` with more disjunctive terms.

Table 2.8: Root node summary statistics for instances solved by branch and cut in 10 seconds to 1 hour, reporting average optimality gap closed (%) and root node processing time (s) for `Default` (D), `Calc Disj, Calc Cuts` (CD,CC), `Param Disj, Calc Cuts` (PD,CC), and `Param Disj, Param Cuts` (PD,PC).

Degree	Terms	Gap Closed (%)				Time (s)			
		D	CD,CC	PD,CC	PD,PC	D	CD,CC	PD,CC	PD,PC
0.5	4	17.31	20.36	20.47	20.06	1.181	6.706	5.843	1.331
	16	20.37	21.61	22.55	21.41	0.968	10.761	8.350	1.477
	64	20.89	24.26	23.81	22.69	0.587	13.447	9.786	1.505
2.0	4	19.75	21.02	20.51	20.24	1.367	8.000	7.113	1.512
	16	18.71	20.41	19.01	18.94	1.061	13.321	10.514	1.722
	64	19.34	22.95	20.62	19.95	0.681	14.999	10.636	1.756

In Figure 2.8, the columns represent the degree of perturbation, and each plot shows the cumulative distribution of runtime changes for `Param Disj, Param Cuts` relative to `Default`, broken out by the number of disjunctive terms. As highlighted in the previous section, reducing generation time while retaining improvements in the root optimality gap closed is critical for improving time to reach optimality. Accordingly, we would expect

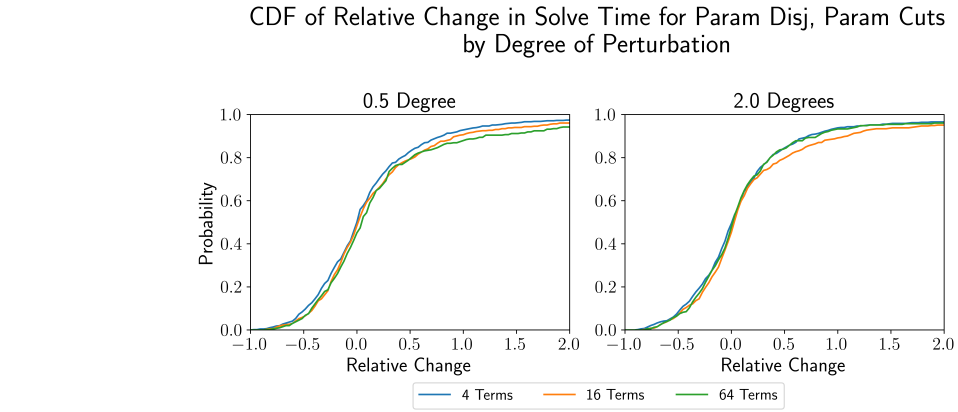


Figure 2.8: Cumulative distribution of change in run time relative to ***`Default`*** *for* ***`Param Disj, Param Cuts`*** *broken out by degree of perturbation.*

fewer terms to yield better performance, which holds true for a perturbation degree of 0.5. However, this pattern does not persist for a perturbation degree of 2, where the outlier is 16 disjunctive terms. As shown in Table 2.8, the processing time and optimality gap closed at the root node for 16 terms are worse than for 64 terms, explaining the deviation.

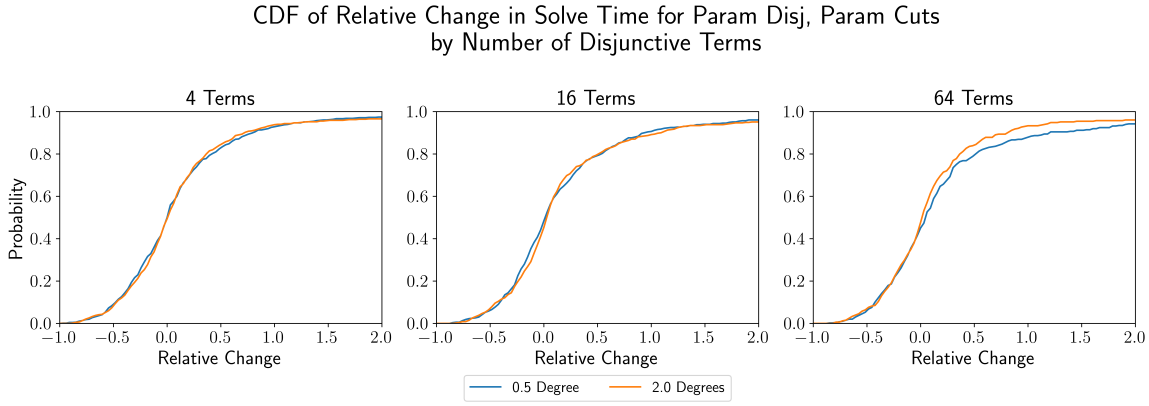


Figure 2.9: Cumulative distribution of change in run time relative to ***`Default`*** *for* ***`Param Disj, Param Cuts`*** *broken out by terms.*

In Figure 2.9, the columns represent the number of disjunctive terms, and each plot shows the cumulative distribution of runtime changes for `Param Disj, Param Cuts` relative to `Default`, broken out by the degree of perturbation. For a given number of terms, we would expect less perturbation to perform better due to the advantage of stronger cuts, especially since the increase in root node processing time for `Param Disj, Param Cuts` relative to `Default` is expected to remain consistent across degrees of perturbation. However, as discussed earlier, solve time is highly sensitive to root node processing time.

This explains why, for 4 and 16 terms, the better-performing degree of perturbation aligns with whichever achieves faster root node processing, as shown in Table 2.8.

The case of 64 terms, however, is an exception. Here, the cuts are weaker, and root node processing time is longer, but yet solve time performance is better, breaking both expected trends. While the exact cause remains unclear, this behavior highlights an open opportunity to further explore the complex relationship between cut strength, root node processing time, and overall runtime performance at higher disjunctive term counts.

Predicting Effectiveness. Ultimately, we aim to determine when disjunctive cuts should be used and which generator is most effective. Building on the previous section, which identified `Param Disj, Param Cuts` as the most performant cut generator, we now focus on understanding when it should be used versus leaving disjunctive cuts disabled, as in `Default`. To measure this, we record the proportion of the test set where a disjunctive cut generator *wins* for a given metric compared to `Default`. A win for `Param Disj, Param Cuts` is defined as achieving at least a 10% reduction in the given metric for a test instance relative to `Default`, and vice versa.

Table 2.9: Winning ratios of test instances for `Param Disj, Param Cuts` (`PD,PC`) and `Default` by performance metric and counts of base and test instances. Wins are defined by one disjunctive cut generator recording a value 10% better than the corresponding value for the other disjunctive cut generator and a given metric, which includes nodes processed, solve time, and solve time excluding VPC generation.

Degree	Terms	Nodes Processed		Time		Time w/o VPC Gen.		Base	Test
		`PD,PC`	`Default`	`PD,PC`	`Default`	`PD,PC`	`Default`		
0.5	4	36.30	34.15	37.73	38.31	38.16	37.88	100	697
	16	35.60	34.88	38.46	39.00	40.25	38.10	96	559
	64	37.28	32.59	35.06	41.98	37.78	38.27	74	405
2.0	4	34.47	36.98	37.52	35.19	38.06	34.47	100	557
	16	34.43	35.31	31.80	37.72	35.31	35.31	90	456
	64	35.33	35.03	31.74	39.82	33.83	34.43	74	334

In Table 2.9, column 1 lists the Degree" of perturbation, $\theta \in \{.5, 2\}$, and column 2 shows the number of Terms", $d \in \{4, 16, 64\}$, queued in a partial branch and bound solve where a disjunction is generated. Columns 3 and 4 report the percentage of the test set where `Param Disj, Param Cuts` and `Default`, respectively, win based on the number of

nodes processed. Columns 5 and 6 show the corresponding percentages for solve time, while columns 7 and 8 present results for solve time excluding the time required to generate disjunctive cuts. Finally, columns 9 and 10 provide the counts of base and test instances used in each experiment. All proportions are expressed as percentages.

The trends in Table 2.9 largely align with expectations. As demonstrated by Balas and Kazachkov [12], `Calc Disj, Calc Cuts` regularly outperforms `Default` when considering its best performance across multiple solve repetitions. Since `Param Disj, Param Cuts` retains much of `Calc Disj, Calc Cuts`'s strength at a fraction of the generation cost, as shown in Table 2.8, we similarly expect `Param Disj, Param Cuts` to win consistently, even with a single repetition. Columns 5 and 6 confirm this expectation for solve time, and similar trends appear for both nodes processed and solve time excluding disjunctive cut generation, as these metrics are closely related to overall solve time.

Another clear trend is that, for a fixed metric and number of disjunctive terms, lower perturbation increases the likelihood of `Param Disj, Param Cuts` winning. This is expected, as less perturbation allows cuts to retain more of their strength while maintaining comparable generation times. Similarly, when holding the degree of perturbation constant and increasing the number of terms, `Param Disj, Param Cuts` achieves more wins for nodes processed. This aligns with the idea that stronger root cuts can significantly reduce the size of the branch and cut tree. Combining these observations, we would also expect that increasing the number of terms while decreasing perturbation would further improve `Param Disj, Param Cuts` 's performance relative to `Default`. This pattern is indeed evident for nodes processed.

However, for solve time, the expected increase in wins with more disjunctive terms is less pronounced. While disjunctive cuts strengthen as terms increase, the performance of `Param Disj, Param Cuts` relative to `Default` remains highly sensitive to root node cut generation time, which also increases with more terms. This sensitivity helps explain why solve time wins decline as terms increase. When disjunctive cut generation time is excluded, we observe win percentages shifting back in favor of smaller disjunctions for `Param Disj, Param Cuts`, as expected.

Despite this, the patterns for solve time differ from those for nodes processed, where

more terms and less perturbation consistently improve performance. This discrepancy can be explained by the properties of disjunctive cuts: their density often slows down pivot algorithms, increasing processing time per node and, consequently, overall solve time, even when fewer nodes are processed. For a more detailed discussion of this phenomenon, we refer the reader to Balas and Kazachkov [12].

Overall Strategy. While `Param Disj, Param Cuts` often provides significant advantages over `Default`, the reverse is also true, prompting a desire to combine their strengths. In practice, both approaches could be run in parallel, terminating when the first satisfies a termination condition—similar to how Gurobi simultaneously runs primal simplex, dual simplex, and barrier methods when solving LPs. Another possibility is to predict when PDIs would be advantageous over no disjunctive cuts and selectively apply `Param Disj, Param Cuts` in such cases.

To explore this, we further analyzed Table 2.9 by breaking down results based on runtime length (Table C.1) and perturbation type (Table C.3) to identify potential patterns. Additionally, we performed a linear regression analysis (Table C.5, Table C.6) using data generated in this work, augmented with tags from the MIPLIB 2017 metadata file, to investigate whether any linear relationships exist regarding the relative advantage of `Param Disj, Param Cuts`. Although these analyses revealed no clear trends, we include them in the appendix for completeness.

2.4 Conclusion

We contributed both theoretical insights and computational results to advance the understanding of warm-starting MILPs with disjunctive cuts. Theoretically, we established key guarantees, including the validity and tightness of PDIs relative to the disjunctive hull, separation conditions for a parametric class of disjunctive cuts and basic solutions, and the NP-hardness of warm-starting MILPs in general. Computationally, we demonstrated that warm-starting with PDIs (`Param Disj, Param Cuts`) consistently outperforms alternative approaches, including the disjunctive cut generation method proposed by Balas and

Kazachkov [12] (`Calc Disj, Calc Cuts`) and its hybrid warm-starting extension (`Param Disj, Calc Cuts`). In many cases, `Param Disj, Param Cuts` also surpassed Gurobi under its default settings (`Default`), highlighting its practical advantages. These results underscore the significant potential of PDIs to enhance MILP solvers, offering a promising direction for future research and solver development.

While PDIs show significant promise, several opportunities exist to further improve their performance. For matrix perturbations, developing a tightening scheme could enhance the effectiveness of generated cuts, as indicated in Table 2.4. Further, decisions such as placing all feasible disjunctive terms in the PRLP, assigning Farkas coefficients of 0 to infeasible terms, and disabling cut generation during disjunction creation merit deeper investigation.

For example, removing pruned yet feasible terms from the PRLP could reduce computation time by shrinking the problem size, but this may weaken the resulting cuts. Similarly, using 0 as a placeholder Farkas coefficient for infeasible terms—while valid—may not be optimal, particularly as perturbations increase the likelihood of those terms becoming feasible. Exploring alternative methods for generating Farkas certificates in such cases is a promising direction for further study.

Additionally, while PDIs recover a significant portion of the dual bound refined by their generating disjunction (Table 2.4), their impact on additional root node gap closure remains muted (Table 2.5). This may occur because Balas and Kazachkov [12] disable cut generation in `Calc Disj`, causing disjunctive cuts and Gurobi's default cuts to refine similar regions of the LP relaxation. Enabling cut generation during disjunction creation might push these cuts to target different parts of the LP relaxation, increasing their combined effectiveness.

Another key question is determining when PDIs should be applied. Although our linear regression analysis revealed no clear relationships between problem features and the relative advantage of PDIs, nonlinear patterns may exist. Training nonlinear models, such as neural networks, could uncover these relationships.

Further computational experiments are also needed to compare PDIs against the warm-starting techniques proposed by Ralphs and Güzelsoy [58]. Finally, given the importance of restarts in satisfiability problems, it would be worth exploring how PDIs could support restart strategies in the context of MILP solvers.

Overall, this work establishes a strong foundation for warm-starting MILPs with PDIs, both theoretically and computationally. By demonstrating their effectiveness across a range of problem settings, we highlight their potential to become a key tool in MILP solvers. However, realizing their full impact requires further refinement, particularly in improving cut generation strategies, identifying when they offer the greatest advantage, and integrating them with existing solver techniques. Future research that addresses these challenges could unlock even greater efficiency gains, ultimately advancing the state of MILP solving and expanding the practical applications of warm-starting methods.

Chapter 3

Strengthening Parametric Disjunctive Inequalities

The PDIs introduced in the previous chapter are constructed to remain valid across a family of instances. However, validity alone does not determine their effectiveness when reused. The strength of a dual bounding function governs the quality of the initial approximation to the value function and therefore influences both the size of the resulting branch-and-bound tree and the stability of reuse across instance modifications.

Weak bounds may transfer formally but provide little practical benefit, as the solver must still reconstruct most of the search tree. Stronger bounds, in contrast, reduce the initial optimality gap and can suppress tree growth more substantially, improving the effectiveness of reuse. The goal of this chapter is therefore not merely to derive stronger inequalities in isolation, but to improve the quality and robustness of reusable dual bounding functions within the framework developed earlier.

3.1 Strengthening Strategies

Strengthening PDIs can be achieved in two ways: by reducing the size of the disjunction or by tightening the cuts to ensure they support it. Throughout this section, let $(A, b, c), (A', b', c') \in \Omega$ be admissible data, where (A, b, c) denotes the source datum that induces any reused certificate and (A', b', c') denotes the current datum at which strength-

ening is performed. Section 3.1.1 and Section 2.2.2 detail each approach, respectively.

3.1.1 Pruning Disjunctive Terms

We can use the primal bound of a known solution to fathom disjunctive terms where better solutions could not possibly exist. Therefore even though the cut generated against this subset of the disjunctive hull is not valid for all integer feasible solutions, it is still valid for the optimal ones, as demonstrated in Figure 3.1.

Theorem 3.1.1. *Let $(A', b', c') \in \Omega$, $\{\mathcal{X}^t\}_{t\in T}$ be a disjunction, and $\{v^t\}_{t\in T}$ be Farkas multipliers. Let $x' \in \mathcal{S}(A', b')$, $z^t(A', b') = \min_{x\in\mathcal{Q}^t(A',b')}\{c'x\}$ for all $t \in T$, and $T' \subseteq T$. If $T' = \{t \in T : z^t(A', b') \leq c'x'\}$, then (α, β) = Theorem 2.2.4$((A', b', c'), \{\mathcal{X}^t\}_{t\in T_{T'}}, \{v^t\}_{t\in T_{T'}})$ is a valid inequality for all $x^* \in \arg\min_{x\in\mathcal{S}(A',b')}\{c'x\}$.*

Proof. Let $x^* \in \arg\min_{x\in\mathcal{S}(A',b')}\{c'x\}$. Since $\mathcal{S}(A', b') \subseteq \{\mathcal{X}^t\}_{t\in T}$, there exists $t \in T$ such that $x^* \in \mathcal{Q}^t(A', b')$. Suppose, for the sake of contradiction, that $x^* \in \mathcal{Q}^t(A', b')$ for some $t \in T \setminus T'$. Because $x^* \in \mathcal{Q}^t(A', b')$, we have $c'x^* \geq z^t(A', b')$. Furthermore, since $t \in T \setminus T'$, it follows that $z^t(A', b') > c'x'$. However, by definition of $x^* \in \arg\min_{x\in\mathcal{S}(A',b')}\{c'x\}$, we must have $c'x' \leq c'x^*$. This results in a contradiction:

$$c'x^* \geq z^t(A', b') > c'x' \leq c'x^*.$$

Therefore, $t \in T'$, and we conclude that $x^* \in \operatorname{cl}\operatorname{conv}\bigcup_{t\in T'}\mathcal{Q}^t(A', b')$. By Theorem 2.2.4, the cut (α, β) is valid for $\operatorname{cl}\operatorname{conv}\bigcup_{t\in T'}\mathcal{Q}^t(A', b')$, and thus valid for x^*. □

Pruning these terms before cut generation can tighten the resulting cut and directs branching toward regions that yield optimal solutions. The idea here is that by preventing branching decisions early in the tree, this could have a dramatic impact on performance given the exponential nature of the search space.

3.1.2 Generate Supporting Inequalities

To strengthen PDIs and ensure they support the disjunctive hull, we must address the three scenarios identified by Kelley et al. [46] in which Theorem 2.2.4 may fail to produce

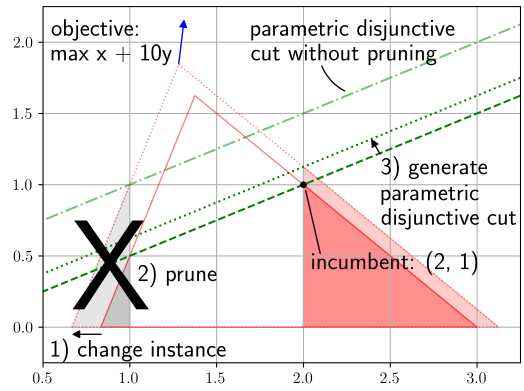


Figure 3.1: Pruning the disjunction with a known solution

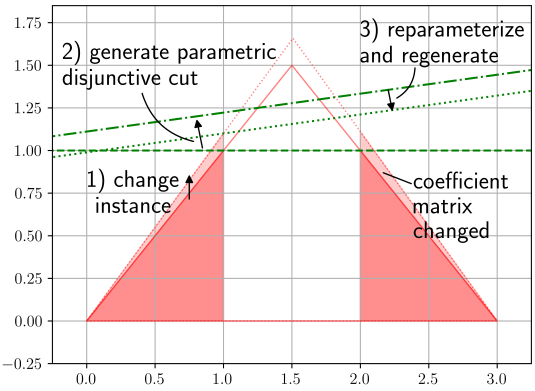


Figure 3.2: Restoring disjunctive hull support under matrix perturbation

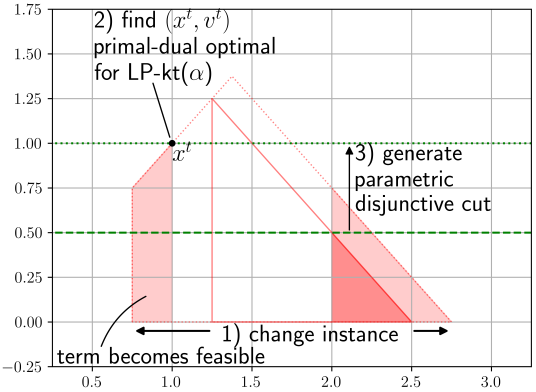


Figure 3.3: Finding a Farkas certificate, v^t, for perturbation-induced feasible terms

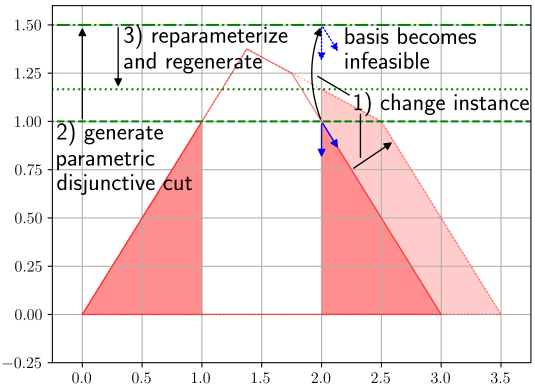


Figure 3.4: Restoring disjunctive hull support for perturbation-induced infeasible bases

a supporting cut: (1) the coefficient matrix changes from the source datum (A, b, c) to the current datum (A', b', c'), (2) a disjunctive term infeasible at (A, b, c) becomes feasible at (A', b', c'), and (3) a basis B^t feasible for $\mathcal{Q}^t(A, b)$ becomes infeasible for $\mathcal{Q}^t(A', b')$. This subsection proceeds in two parts. First, we *reparameterize* by computing a new Farkas certificate for each relevant disjunctive term. Then, we *regenerate* the PDI using the updated certificates to ensure the resulting inequality supports the disjunctive hull. Figures 3.2 to 3.4 illustrate these steps in each of the three failure cases

Reparameterization. We refer to the process of obtaining a supporting cut and its corresponding *supporting certificate*—the Farkas certificate that generates the supporting cut—as *reparameterization*. Fix $t \in T$ and $\alpha \in \mathbb{R}^n$. As shown in Lemma 3.1.2, this involves

solving the primal-dual pair LP-(A', b', α) and Dual-(A', b', α)

$$\min_{x \in \mathcal{Q}^t(A', b')} \alpha^\top x \qquad \text{(LP-}(A', b', \alpha)\text{)} \qquad\qquad \max_{v \in \mathcal{D}^t(A', b', \alpha)} v b'^t \qquad \text{(Dual-}(A', b', \alpha)\text{)}$$

where $\mathcal{D}^t(A', b', \alpha) = \{v \in \mathbb{R}^{1 \times (q+q_t)} : vA'^t = \alpha,\ v \geq 0\}$.

Lemma 3.1.2. *Let $(A', b', c') \in \Omega$, $t \in T$, and $\alpha \in \mathbb{R}^n$. If $\mathcal{Q}^t(A', b')$ is bounded and nonempty, then $(\alpha, \alpha^\top \bar{x})$ such that $\bar{x} \in \arg\min_{x \in \mathcal{Q}^t(A', b')}\{\alpha^\top x\}$ is a supporting inequality for $\mathcal{Q}^t(A', b')$ with supporting certificate $\bar{v}^t \in \arg\max_{v^t \in \mathcal{D}^t(A', b', \alpha)}\{v^t b'^t\}$.*

Proof. Let $\bar{x} \in \arg\min_{x \in \mathcal{Q}^t(A', b')}\{\alpha^\top x\}$ and $\bar{v}^t \in \arg\max_{v^t \in \mathcal{D}^t(A', b', \alpha)}\{v^t b'^t\}$. Since $\bar{x}$ is optimal for LP-(A', b', α), we have that $\alpha^\top x \geq \alpha^\top \bar{x}$ for all $x \in \mathcal{Q}^t(A', b')$, which means that $\alpha, \alpha^\top \bar{x}$ supports $\mathcal{Q}^t(A', b')$. Since $\bar{v}^t$ is feasible for Dual-(A', b', α), we have that $\bar{v}^t A'^t = \alpha$. By strong duality, $\bar{v}^t b'^t = \alpha^\top \bar{x}$, which means that $\bar{v}^t$ is a certificate for the cut $(\alpha, \alpha^\top \bar{x})$ and is thus supporting. □

Further, provided there is no constraint in $\mathcal{Q}^t(A', b')$ parallel to α, the only supporting cut with coefficients α and a non-negative Farkas certificate is the one derived from the optimal solutions to the primal-dual pair LP-(A', b', α) and Dual-(A', b', α). This is essential, since Theorem 2.2.4 guarantees the validity of PDIs only when the Farkas certificate is non-negative.

Corollary 3.1.3. *Let $(A', b', c') \in \Omega$, $t \in T$, and $\alpha \in \mathbb{R}^n$. If $\mathcal{Q}^t(A', b')$ is bounded and nonempty and $\alpha \neq A'^t_{i\cdot}$ for all $i \in [q + q_t]$, then there exist unique $\bar{x} \in \mathcal{Q}^t(A', b')$ and unique $\bar{v}^t \in \mathcal{D}^t(A', b', \alpha)$ such that $\alpha^\top \bar{x} = \bar{v}^t b'^t$.*

Proof. Lemma 3.1.2 provides existence of $\bar{x}$ and $\bar{v}^t$. Since there does not exist $i \in [q + q_t]$ such that $\alpha = A'^t_{i\cdot}$, pivoting from $\bar{x}$ to another $x \in \mathcal{Q}^t(A', b')$ would violate dual feasibility. Therefore, $\bar{x} \in \mathcal{Q}^t(A', b')$ and $\bar{y} \in \mathcal{D}^t(A', b', \alpha)$ such that $\alpha^\top \bar{x} = \bar{y}^\top b'^t$ are unique. □

A couple of observations are worth noting here. First, because the Farkas certificate that generates a supporting cut is almost always unique for given cut coefficients and disjunctive term, solving LP-(A', b', α) is unavoidable. However, in practice, we can often expedite this

step by storing the basis that originally determined the certificate for each term. As a result, re-optimizing after small perturbations is typically fast. Second, this procedure is agnostic to which specific hypothesis of Lemma 2.2.8 is violated. Its sole objective is to recover a certificate for given cut coefficients and disjunctive term such that the resulting inequality supports that term. This is precisely the condition that, as we show next, is required for Theorem 2.2.4 to produce PDIs that support the disjunctive hull.

Regeneration

We leverage Lemma 3.1.2 to construct a Farkas certificate that satisfies the conditions of Lemma 2.2.8, ensuring that Theorem 2.2.4 produces a cut supporting the disjunctive hull. This process is detailed in Algorithm 11.

Algorithm 11 `Disjunctive Hull Supporting Cut Generator`

Require: $(A, b, c), (A', b', c'), \{\mathcal{X}^t\}_{t\in T}, \{v^t\}_{t\in T}, \{B^t\}_{t\in T}$
Ensure: $(\{\mathcal{X}^t\}_{t\in T}, (A, b, c))$ induce $\{v^t\}_{t\in T}$
Ensure: $\{B^t\}_{t\in T}$ determines $\{v^t\}_{t\in T}$.
Ensure: There exists $t \in T$ such that B^t is feasible for $\mathcal{Q}^t(A', b')$ and $v^t \neq 0$.
1: $T_f = \{t \in T : v^t \neq 0, (A'^t_{B^t\cdot})^{-1} b'^t_{B^t} \in \mathcal{Q}^t(A', b')\}$ ▷ Term has certificate and basis still feasible
2: $\{\bar{v}^t\}_{t\in T} \leftarrow \{v^t\}_{t\in T}$
3: $(\alpha, \beta) \leftarrow$ Theorem 2.2.4$((A', b', c'), \{\mathcal{X}^t\}_{t\in T_f}, \{v^t\}_{t\in T_f})$ ▷ Find cut coefficients α
4: **for all** $t \in T$ such that $A^t \neq A'^t$ or $t \notin T_f$ **do** ▷ Terms where support could be lost
5: $\bar{v}^t \leftarrow \underset{y\in\mathcal{D}^t(A',b',\alpha)}{\arg\max} \{y^\top b'^t\}$ ▷ Find supporting certificate for α
6: **end for**
7: $(\alpha, \bar{\beta}) \leftarrow$ Theorem 2.2.4$((A', b', c'), \{\mathcal{X}^t\}_{t\in T_f}, \{v^t\}_{t\in T_f}[\bar{v}^t])$ ▷ Regenerate with supporting certificate
8: **return** $(\alpha, \bar{\beta})$

We can unpack Algorithm 11 as follows. In Step 3, we apply Theorem 2.2.4 at the current datum (A', b', c') to generate an initial inequality valid for the disjunctive hull. Then, in Step 5, we solve the dual of LP-(A', b', α) to obtain a supporting certificate for each term $t \in T$ where disjunctive hull support might be lost, corresponding to the scenarios illustrated in Figures 3.2 to 3.4. By Lemma 3.1.2, each certificate is guaranteed to support its corresponding disjunctive term. Finally, in Step 7, with the conditions of Lemma 2.2.8 satisfied, we reapply Theorem 2.2.4 to regenerate a cut with coefficients α that supports the disjunctive hull, as shown in Theorem 3.1.4.

Theorem 3.1.4. *Let $(A, b, c), (A', b', c') \in \Omega$. Let $\{\mathcal{X}^t\}_{t\in T}$ be a disjunction, $\{v^t\}_{t\in T}$ be Farkas multipliers induced by $(\{\mathcal{X}^t\}_{t\in T}, (A, b, c))$, and $\{B^t\}_{t\in T}$ be a collection of constraint indices determining $\{v^t\}_{t\in T}$. If there exists $t \in T$ such that B^t is feasible for $\mathcal{Q}^t(A', b')$ and $v^t > 0$, then $\alpha, \bar{\beta}$ = Algorithm 11$((A, b, c), (A', b', c'), \{\mathcal{X}^t\}_{t\in T}, \{v^t\}_{t\in T}, \{B^t\}_{t\in T})$ supports* $\operatorname{cl}\operatorname{conv}(\cup_{t\in T}\mathcal{Q}^t(A', b'))$.

Proof. Define T_f as in Step 1, which is nonempty due to the existence of $t \in T$ such that B^t is feasible for $\mathcal{Q}^t(A', b')$ and $v^t > 0$, and let $T_c = \{t \in T_f : A^t = A'^t\}$. As prescribed in Step 2, let $\{\bar{v}^t\}_{t\in T} = \{v^t\}_{t\in T}$, and, additionally, let $\{\bar{B}^t\}_{t\in T} = \{B^t\}_{t\in T}$. By Theorem 2.2.4, Step 3 provides $\alpha = \bar{v}^t A'^t$ for all $t \in T_c$. Consider $t \in T \setminus T_c$ such that $\mathcal{Q}^t(A', b') \neq \emptyset$. Update $\bar{B}^t$ such that $A'^t_{\bar{B}^t\cdot}\bar{x}^t = b'^t_{\bar{B}^t}$ for $\bar{x}^t \in \arg\max_{x\in\mathcal{Q}^t(A',b')}\{\alpha^\top x\}$. By Lemma 3.1.2, Step 5 provides $\bar{v}^t \in \mathbb{R}^{1\times n}_{\geq 0}$ and $\alpha = \bar{v}^t A'^t$. Since $\bar{x}^t$ and $\bar{v}^t$ are dual to each other, we have that $\{i \in [q + q_t] : \bar{v}^t_i > 0\} \subseteq \bar{B}^t$. Therefore, $(\{\mathcal{X}^t\}_{t\in T}, (A', b', c'))$ induce $\{\bar{v}^t\}_{t\in T}$, $\{\bar{B}^t\}_{t\in T}$ determines $\{\bar{v}^t\}_{t\in T}$, and, for all $t \in T_f$, $\bar{B}^t$ is feasible for $\mathcal{Q}^t(A', b')$. Thus, Theorem 2.2.4 provides in Step 7 a cut $(\alpha, \bar{\beta})$ supporting $\operatorname{cl}\operatorname{conv}(\cup_{t\in T_f}\mathcal{Q}^t(A', b')) = \operatorname{cl}\operatorname{conv}(\cup_{t\in T}\mathcal{Q}^t(A', b'))$. □

The key insight of Theorem 3.1.4 is that we can input *any* Farkas certificate from a previously generated valid disjunctive inequality and obtain a cut that is guaranteed to support the disjunctive hull. Because the resulting cuts are guaranteed to be as tight as possible, this framework has the potential to significantly improve the performance of MILP solvers, provided that cut generation remains computationally efficient. Moreover, if these strengthened cuts fail to improve the root relaxation, it indicates a limitation in the choice of parameters rather than in the cut generation procedure itself, offering a clear direction for further research.

We demonstrate the effectiveness of Algorithm 11 in Example 3.1.5, where the algorithm is applied to a case with a perturbed coefficient matrix. The resulting cut continues to support the disjunctive hull, showing that the approach is robust to such changes. Since the same procedure applies to cases involving newly feasible terms (Figure 3.3) and infeasible bases (Figure 3.4), this example highlights how Algorithm 11 can restore disjunctive hull support in general.

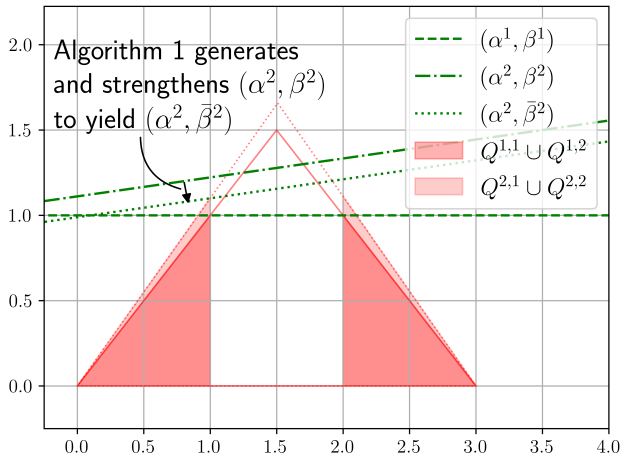


Figure 3.5: Algorithm 11 ensures PDIs support the disjunctive hull when $A^1 \neq A^2$.

$$
\begin{aligned}
\min_{x \in \mathbb{R}^2} \quad & -x_2 \\
x_1 \quad & -x_2 \geq 0 \\
-x_1 \quad & -x_2 \geq -3 \\
x_1 \quad & \geq 0 \quad \text{(MILP-1)} \\
& x_2 \geq 0 \\
x_1, \quad & x_2 \in \mathbb{Z}
\end{aligned}
$$

$$
\begin{aligned}
\min_{x \in \mathbb{R}^2} \quad & -x_2 \\
1.1x_1 \quad & -x_2 \geq 0 \\
-x_1 \quad & -0.9x_2 \geq -3 \\
x_1 \quad & \geq 0 \quad \text{(MILP-2)} \\
& x_2 \geq 0 \\
x_1, \quad & x_2 \in \mathbb{Z}
\end{aligned}
$$

Example 3.1.5. *Let $\{v^1, v^2\} = \{[1, 0, 0, 0, 1], [0, 1, 0, 0, 1]\}$ denote the Farkas certificate induced by MILP-1 and the disjunction $\{\mathcal{X}^1, \mathcal{X}^2\} = \{\{x \in \mathbb{R}^2 : x_1 \leq 1\}, \{x \in \mathbb{R}^2 : x_1 \geq 2\}\}$ from the cut $(\alpha^1, \beta^1) = ([0, -1], -1)$, i.e., $-x_2 \geq -1$. To generate a PDI supporting the disjunctive hull of MILP-2 and $\{\mathcal{X}^1, \mathcal{X}^2\}$, Algorithm 11 proceeds as follows:*

Step 1: Identify disjunctive terms with active constraint sets that are still feasible for MILP-2 and have nonzero Farkas multipliers.

$$T_f = \{1, 2\}$$

Step 2: Default the new Farkas certificate $\{\bar{v}^t\}_{t \in T}$ to the values of the old one $\{v^t\}_{t \in T}$.

$$\{\bar{v}^t\}_{t \in T} = \{[1, 0, 0, 0, 1], [0, 1, 0, 0, 1]\}$$

Step 3: Generate an initial valid PDI for MILP-2 and $\{\mathcal{X}^1, \mathcal{X}^2\}$.

$(\alpha^2, \beta^2) = Theorem\ 2.2.4(2, \{\mathcal{X}^1, \mathcal{X}^2\}, \{v^1, v^2\}) = ([1, -9], -10)$

Step 5: To restore disjunctive hull support, recalculate the Farkas certificates for disjunctive terms where it could be lost. For coefficient matrix perturbations, this can include all terms.

$$\bar{v}^1 = \underset{v \in \mathcal{D}^1(A^2, b^2, \alpha^2)}{\arg\max} \{vb^{2,1}\} = [.9, 0, 0, 0, .89] \quad and \quad \bar{v}^2 = \underset{v \in \mathcal{D}^2(A^2, b^2, \alpha^2)}{\arg\max} \{vb^{2,2}\} = [0, 1, 0, 0, 1.1]$$

Step 7: Regenerate the PDI with the new Farkas certificate.

$(\alpha^2, \bar{\beta}^2) = Theorem\ 2.2.4(2, \{\mathcal{X}^1, \mathcal{X}^2\}, \{\bar{v}^1, \bar{v}^2\}) = ([1, -9], -8.9)$

Since MILP-2 and $\mathcal{X}^1, \mathcal{X}^2$ *induce the Farkas multipliers* $\{\bar{v}^1, \bar{v}^2\}$ *and feasible bases determine them, the resulting cut* $x_1 - 9x_2 \geq -8.9$*, corresponding to* $(\alpha^2, \bar{\beta}^2) = ([1, -9], -8.9)$*, supports the disjunctive hull, as shown in Figure 3.5.*

3.2 Computational Results

The goal of our experiments is to assess how strengthening PDIs affects performance when solving a series of related MILPs. This section is divided into four parts: the experimental setup (Section 3.2.1); two motivating examples (Section 3.2.2); and analyses of the impact on root node processing (Section 3.2.3) and overall solver performance (Section 3.2.4). Together, these results highlight the effectiveness of the proposed strengthening methods and their implications for MILP solving.

3.2.1 Computational Setup

Prior to experimentation, we generated series of randomly perturbed MILP instances based on MIPLIB 2017 and implemented three methods for strengthening PDIs. These methods were evaluated against a baseline using standard MILP solvers on a server cluster, producing

the data analyzed in the sections that follow. This setup provides a controlled and reproducible environment for assessing the impact of disjunctive cuts on solver performance. The **Bold**-font terms introduced below define column headers used throughout Sections 3.2.2 to 3.2.4.

Test Instances. We adopt the test instance generation approach from Kelley et al. [46, Section 4.1.1]. Starting with a set of **Base** instances $\mathcal{B}$, which are presolved MIPLIB 2017 problems taking at least 10 seconds to solve and containing no more than 5,000 variables and 5,000 constraints, we construct a collection of sets of **Test** instances indexed by $\mathcal{T}$. Each test set is identified by a tuple $(k, u, \theta) \in \mathcal{B} \times \{A, b, c\} \times \{.5, 2\}$, where k indexes the base instance, u the **Element** perturbed, and θ the perturbation **Degree**. Each instance in a test set is generated by applying Kelley et al. [46, Algorithm 3] to u^k, replacing the selected component with a new version rotated by an angle θ relative to the original. For each (k, u, θ), we run Kelley et al. [46, Algorithm 3] iteratively, bound by the following criteria: reaching 1,000 attempts, exceeding 4 hours, or successfully producing 5 test instances. As a result, the number of test instances per base instance may vary.

Comparisons Performed. Our experiments evaluate strategies for strengthening PDIs. Let d represent the number of disjunctive **Terms** in the branch and bound queue when a disjunction is constructed from the leaves of the solve's branch and bound tree. For each $d \in \{4, 64\}$ and test set $(k, u, \theta) \in \mathcal{T}$, we assess six disjunctive cut generation approaches unless stated otherwise.

The first three methods serve as baselines. `Default` follows Kelley et al. [46, Algorithm 4], solving the series $\mathcal{T}_{ku\theta}$ using default solver settings without disjunctive cuts. `VPC` corresponds to Kelley et al. [46, Algorithm 5], where a new disjunction is generated for each test instance and used to compute disjunctive cuts from an LP. `PDI` is based on Kelley et al. [46, Algorithm 7], where a previously generated disjunction is reused to produce PDIs as described in Theorem 2.2.4.

The remaining three are our proposed strengthening variants, all built on `PDI`. Each is defined by specific settings of the boolean flags `prune` and `support` in Algorithm 12.

`Prune` sets `prune` to true and `support` to false, pruning the disjunction using a primal bound before generating cuts, as formalized in Theorem 3.1.1. When `prune` is false and `support` is true, `S-PDI` calls Algorithm 11 to generate cuts that support the disjunctive hull. `P&S` enables both flags, combining pruning with the generation of hull-supporting PDIs. `S-PDI` and `P&S` are omitted when the objective is perturbed. This case satisfies Lemma 2.2.8, guaranteeing that PDIs from Theorem 2.2.4 already support the disjunctive hull and rendering Algorithm 11 to have no effect.

Algorithm 12 relies on the subroutines `Branch-and-Cut` and `Branch-and-Bound`. The routine `Branch-and-Cut` solves the instance MILP-k with an initial pool of cutting planes $\Pi \subseteq \mathbb{R}^{n+1}$ and returns a set of feasible solutions $S \subseteq \mathcal{S}^k$. The routine `Branch-and-Bound` performs a similar solve, but disables cut generation and does not accept an initial cutting-plane pool. It returns the terminal disjunction, including all fathomed terms, and is configured according to Balas and Kazachkov [12, Appendix C].

Experimental Setup. Our experiments relied on the following software and hardware. We relied on Gurobi 10 as our `Branch-and-Cut` solver, writing a callback to add our cuts at the root node, and CBC 2.10 for `Branch-and-Bound`. We used the C++ library VPC [3] as the implementation for Balas and Kazachkov [12, Algorithm 1]. The experiments are conducted on a server cluster with 16 AMD Opteron Processor 6128's; 2 CPUs and up to 15GB of RAM were dedicated to each experiment. We allowed 3600 seconds each for generating disjunctive cuts as well as for solving each MILP instance.

3.2.2 High Performing Subset of Experiments

The experiments in Sections 3.2.3 and 3.2.4 are designed to investigate two key patterns observed in selected MILP series. First, strengthening PDI generation can tighten root relaxations with minimal overhead in cut generation time (Tables 3.1 to 3.3). Second, it can lead to substantial reductions in overall solve time (Table 3.4). These findings are examined in detail in Sections 3.2.2 and 3.2.2, respectively.

Table 3.1: Average percent integrality gap closed by disjunctive cuts alone and implied by their generating disjunctions, grouped by degree of perturbation, size of disjunction, and element perturbed for test sets generated from `bm23.mps`.

Degree	Terms	Element	CD	PD	VPC	P&S	Prune	S-PDI	PDI
0.5	4	matrix	14.78	14.78	14.78	14.63	14.62	14.63	14.62
		objective	14.63	14.63	14.63	–	14.63	–	14.63
		rhs	13.85	14.60	13.85	14.22	14.22	14.22	14.22
	64	matrix	71.00	71.35	69.05	67.55	67.05	67.23	66.46
		objective	68.83	71.48	66.74	–	69.37	–	69.37
		rhs	69.03	70.93	66.91	68.17	68.15	68.17	64.77
2	4	matrix	13.38	14.33	13.22	13.00	12.90	13.00	12.90
		objective	14.79	14.79	14.74	–	14.76	–	14.76
		rhs	11.77	13.27	11.61	12.16	12.16	12.16	12.16
	64	matrix	70.44	71.38	67.68	54.79	43.18	49.42	28.17
		objective	66.90	70.41	64.82	–	67.60	–	67.60
		rhs	69.49	63.82	66.88	60.35	52.45	60.35	44.03

Effect of Strengthening on Root Relaxations. Tables 3.1 to 3.3 are designed to illustrate this first pattern. For all three tables, columns 1 through 3 designate their primary keys: degree of perturbation, the number of disjunctive terms, and the element perturbed, respectively.

Table 3.1 reports the percentage of integrality gap closed across 20 random perturbations of `bm23.mps`. `CD` reflects the gap closed by dual bound of the disjunction used by `VPC`, while `PD` reflects the gap closed by the shared disjunction used by `P&S`, `Prune`, `S-PDI`, and `PDI`. The remaining columns show the percentage of gap closed by the LP relaxation after adding disjunctive cuts from each corresponding method.

The patterns observed in Table 3.1 are broadly consistent with expectations. Since `VPC` generates cuts valid for the disjunction used in `CD`, and `P&S`, `Prune`, `S-PDI`, and `PDI` all generate cuts valid for the disjunction used in `PD`, the dual bounds provided by `CD` and `PD` serve as upper bounds on the LP relaxation objective values after cuts are added. As a result, the percentage of integrality gap closed by `CD` upper bounds that of `VPC`, and similarly, the gap closed by `PD` upper bounds those of `P&S`, `Prune`, `S-PDI`, and `PDI`.

Since `VPC` generates Farkas certificates optimized for cut depth, while `P&S`, `Prune`, `S-PDI`, and `PDI` reuse previously computed certificates, `VPC` would generally be expected to close

more of the integrality gap. This trend is reflected in Table 3.1. Exceptions arise when PD closes more gap than CD, which is can happen for smaller disjunctions or low degrees of perturbation. In such cases, P&S, Prune, S-PDI, and PDI benefit from generating cuts against a tighter relaxation without suffering the degradation that can affect them relative to VPC under larger disjunctions or perturbations.

As specified in Algorithm 12, P&S strengthens the cuts generated by both Prune and S-PDI, which in turn strengthen those generated by PDI. Accordingly, the gap closed by P&S is always at least as large as those of Prune and S-PDI, which themselves are no smaller than that of PDI. Consistent with findings in Kelley et al. [46], gap closure tends to increase with disjunction size, since larger disjunctive hulls yield tighter relaxations. In contrast, it tends to decrease with higher perturbation levels, as LP relaxation solutions may shift to positions where the cuts, although still valid, are weakened.

Figures 3.2 to 3.4 show that for perturbations to the constraint matrix or right-hand side, PDI may produce cuts that do not support the disjunctive hull. In such cases, stronger cuts can be obtained by pruning disjunction terms or computing a supporting certificate before applying Theorem 2.2.4. This effect is particularly pronounced at higher perturbation levels or with larger disjunctions, as also reflected in Table 3.1, since both settings introduce greater potential for cut weakening and, correspondingly, greater opportunity for strengthening.

One intuitively appealing pattern that does not appear, however, is that CD might be expected to consistently upper bound PD. Since CD represents disjunctions tailored to each test instance and PD uses a shared disjunction across the test set, it is natural to expect the former to dominate. Yet this is not consistently the case in Table 3.1. This discrepancy is not entirely surprising, as disjunctions are derived from partial branch and bound trees influenced by solver heuristics and settings, and the degree of perturbation may not be large enough to offset this variability for bm23.mps.

Table 3.2 reports the percentage of integrality gap closed by the LP relaxation after root node processing for 20 random perturbations of bm23.mps. These values reflect the effect of all cuts generated at the root node, including the disjunctive cuts.

The general trends in Table 3.2 mirror those in Table 3.1. As before, VPC provides the

Table 3.2: Average percent integrality gap closed after root node processing, grouped by degree of perturbation, size of disjunction, and element perturbed for test sets generated from `bm23.mps`.

Degree	Terms	Element	VPC	P&S	Prune	S-PDI	PDI	Default
0.5	4	matrix	42.63	40.35	40.45	40.62	40.48	42.16
		objective	39.18	–	41.93	–	40.89	39.23
		rhs	43.19	41.87	41.80	41.49	42.00	43.87
	64	matrix	70.30	69.32	69.01	69.06	68.57	42.13
		objective	68.40	–	70.52	–	70.52	39.38
		rhs	68.69	69.41	69.39	69.44	68.08	42.63
2	4	matrix	39.92	40.75	40.91	40.10	41.11	38.71
		objective	40.16	–	39.64	–	39.54	40.02
		rhs	42.92	42.49	41.60	42.49	41.60	41.16
	64	matrix	69.56	59.96	52.73	56.42	48.44	38.82
		objective	67.09	–	69.25	–	69.28	40.01
		rhs	69.34	62.27	58.32	62.25	56.53	41.16

strongest performance, followed by `P&S`, then `Prune` and `S-PDI`, with `PDI` typically trailing. The integrality gap closed increases with disjunction size and decreases with the level of perturbation. Additionally, the benefit of the strengthening variants `P&S`, `Prune`, and `S-PDI` over `PDI` tends to grow as disjunctions get larger and perturbations more severe. That said, because the interaction between cuts is complex and not fully understood, it is possible for a method with stronger individual cuts to result in a smaller overall gap closure after root node processing, especially when those cuts alone recover less of the dual bound.

New patterns also emerge. One might expect `Default` to consistently close less of the integrality gap, as it lacks access to disjunctive cuts. This generally holds true, but exceptions occur when disjunctive cuts contribute little to the dual bound on their own. As shown in Balas and Kazachkov [12], in such cases, default cuts can often tighten the same parts of the feasible region, explaining the comparable performance. The inverse is particularly important. As shown in Table 3.2, when disjunctions are large enough, disjunctive cuts can tighten the root relaxation in ways that Gurobi's default cuts cannot. This same pattern holds when comparing strengthened PDIs to their default counterparts, underscoring the added value of our proposed methods.

Table 3.3 reports root node processing times for 20 random perturbations of `bm23.mps`. The observed trends largely align with expectations. As shown in Kelley et al. [46], `PDI`

Table 3.3: Time to process root node, grouped by degree of perturbation, size of disjunction, and element perturbed for test sets generated from `bm23.mps`.

Degree	Terms	Element	VPC	P&S	Prune	S-PDI	PDI	Default
0.5	4	matrix	0.227	0.122	0.112	0.103	0.113	0.134
		objective	0.200	–	0.112	–	0.122	0.105
		rhs	0.213	0.113	0.120	0.122	0.124	0.162
	64	matrix	1.629	0.253	0.176	0.405	0.176	0.131
		objective	1.661	–	0.174	–	0.173	0.103
		rhs	1.629	0.187	0.172	0.214	0.174	0.127
2	4	matrix	0.219	0.145	0.125	0.143	0.126	0.102
		objective	0.209	–	0.120	–	0.119	0.124
		rhs	0.227	0.134	0.119	0.135	0.118	0.105
	64	matrix	1.702	0.347	0.202	0.548	0.214	0.104
		objective	1.633	–	0.176	–	0.176	0.118
		rhs	1.589	0.251	0.195	0.364	0.198	0.106

significantly reduces root node time compared to `VPC` because it generates disjunctive cuts directly from Farkas certificates and avoids solving an LP. However, this still incurs some cost, so `Default` generally provides a lower bound on `PDI`. Similarly, `P&S`, `Prune`, and `S-PDI` tend to fall between `VPC` and `Default`. Like `VPC`, `P&S` and `S-PDI` also solve LPs, but warm starts and, here, smaller formulations preserve a time advantage.

We also observe that enabling pruning usually reduces processing time. For example, `P&S` is typically faster than `S-PDI`, and `Prune` is often faster than `PDI`. This is expected, as pruning decreases the number of disjunctive terms and therefore the number of calculations during disjunctive cut generation.

Minor deviations from these patterns are not surprising. Because the interactions between disjunctive and default cuts are not fully known, a slightly more expensive disjunctive cut procedure can occasionally reduce overall processing time by decreasing the need for default cuts later at the root.

The main takeaway is that our strengthening methods preserve much of the time advantage of PDIs over `VPC`. Combined with the gap-closing results in Table 3.2, they often deliver tighter relaxations with minimal additional time, confirming the existence of the first pattern we set out to explore.

Solve-Time Effects of Strengthening. Table 3.4 is structured as follows. The first three columns mirror those in Section 3.2.2, indicating the degree of perturbation, number of disjunctive terms, and element perturbed. The fourth column identifies the base instance from which each test series was generated. The next four columns report overall solve times (in seconds) for the corresponding cut generator. The final column gives the sample size, representing the number of instances in each test series.

Table 3.4: Solve time (in seconds) for a subset of series with significant solve time improvement when strengthening PDIs with Algorithm 11.

Degree	Terms	Element	Instance	`Default`	`VPC`	`PDI`	`S-PDI`	Count
0.5	4	matrix	danoint	1481	1597	1349	657.5	3
		rhs	gen-ip002	1380	1183	1319	695.5	2
	64	matrix	k16x240b	314.8	243.4	245.7	173.6	3
		rhs	timtab1	244.0	251.3	219.0	172.2	4
2.0	4	matrix	timtab1	56.18	55.78	44.02	28.52	2
		rhs	gen-ip036	295.7	250.5	265.3	193.2	4
	64	matrix	gen-ip021	384.3	440.0	359.6	204.9	4
		rhs	icir97_tension	1028	1343	1700	953.8	3

Table 3.4 varies the degree of perturbation, number of disjunctive terms, perturbed element, and base instance to demonstrate that strengthening-induced solve time improvements persist across a wide range of settings. As shown in Table 3.1 and later in Table 3.7, the strength of PDIs often increases monotonically with both disjunction size and perturbation degree. However, because the relationship between cut strength and solve time remains only partially understood, we do not expect a direct correlation. In some cases, smaller disjunctions or larger perturbations yield greater reductions in solve time. This does not diminish the main takeaway of the table: strengthening PDIs can substantially reduce solve time relative to `Default`, `VPC`, and `PDI` across a variety of problem characteristics. In some cases, it even determines whether disjunctive cuts help at all.

3.2.3 Root Node Performance

Building on the findings of Section 3.2.2, which showed that strengthened PDIs can tighten the root relaxation without increasing generation time, we now evaluate how broadly this

Table 3.5: The number of base and test instances where VPC generation succeeds for all combinations of possible degree of perturbation and terms.

Element	Base Instances	Test Instances
matrix	45	106
rhs	24	49

Table 3.6: Counts of infeasible nodes that become feasible under perturbation, feasible nodes with supporting bases that become infeasible, and the percentage of cuts with modified coefficients, grouped by perturbed element, degree of perturbation, and number of disjunctive terms, for test instances included in Table 3.5.

Element	Degree	Terms	Term Δ	Basis Δ	Cut Δ (%)
matrix	0.5	4	0.00	2.27	92.7
		64	1.11	20.9	97.9
	2.0	4	0.02	2.93	99.6
		64	7.71	25.0	99.9
rhs	0.5	4	0.00	3.67	0.00
		64	1.12	40.6	0.00
	2.0	4	0.00	3.67	0.00
		64	2.14	37.0	0.00

effect holds across a larger set of instances. We restrict attention to test cases where `Calc Disj, Calc Cuts` completes within the one-hour time limit and where Lemma 2.2.8 does not hold, allowing the possibility for Algorithm 11 to take effect. Due to an insufficient number of cases with small disjunctions or low perturbation degrees in which pruning took effect, we exclude `Prune` and `P&S` results and focus on the `S-PDI` strengthening method. Table 3.5 reports the number of qualifying base and test instances for each perturbed element.

Root Optimality Gap Under Strengthening. We begin our root node analysis of strengthened PDIs by evaluating their potential to close additional root optimality gap relative to `PDI`. In Table 3.6, we quantify this potential by tracking how often, under perturbation, infeasible disjunctive terms become feasible, supporting bases for feasible terms become infeasible, and the coefficients of PDIs change. These conditions correspond precisely to cases where Lemma 2.2.8 does not hold, i.e. situations in which tightening the cuts may be possible.

Table 3.7: Average percent integrality gap closed by disjunctive cuts and implied by disjunctions, grouped by degree of perturbation, size of disjunction, and element perturbed for test sets generated from MIPLIB 2017.

Degree	Terms	Element	CD	PD	VPC	S-PDI	PDI
0.5	4	matrix	5.96	2.67	3.38	1.90	1.90
		rhs	8.58	3.69	3.32	2.23	2.23
	64	matrix	12.61	7.88	5.94	3.51	3.31
		rhs	17.66	9.99	11.16	6.47	5.21
2.0	4	matrix	8.33	2.53	4.30	1.10	1.10
		rhs	18.63	3.00	3.14	0.48	0.48
	64	matrix	16.26	5.75	7.43	1.18	0.44
		rhs	27.38	7.31	9.13	1.16	1.08

Table 3.6 reports the changes in status for disjunctions and PDIs under perturbation. The first three columns separate test instances by the element perturbed, degree of perturbation, and disjunctive terms, respectively. Column 4 gives the number of disjunctive terms on average that go from infeasible to feasible for the given subset of test instances as illustrated by Figure 3.3. Column 5 gives the average number of feasible terms with supporting bases that become infeasible for the given subset of test instances as illustrated by Figure 3.4. Column 6 gives the average percentage of PDIs whose coefficients change for the given subset of test instances as illustrated by Figure 3.2.

The patterns in Table 3.6 align with expectations. The frequency of infeasible disjunctive terms becoming feasible, supporting bases for feasible terms becoming infeasible, and changes in the coefficients of PDIs almost always increase with greater perturbation and larger disjunctions. This is consistent with the intuition that more perturbation and more terms lead to a higher likelihood of status changes in nodes and bases. Similarly, greater perturbation increases the number of entries in the coefficient matrix that change, and larger disjunctions raise the chance that such changes affect the resulting cuts from Theorem 2.2.4. As expected, we observe no changes in cut coefficients for right-hand side perturbations, since only matrix perturbations can alter coefficients under the `S-PDI` method. We use these result to help explain the patterns we see, or lack thereof, in `S-PDI`'s ability to close additional root optimality gap relative to `PDI` as shown in Table 3.7.

Table 3.7 reports the percentage of integrality gap closed across our collection of test

sets. Column definitions match those in Table 3.1, with the final three columns showing the gap closed after adding disjunctive cuts from each corresponding method. As before, `CD` and `PD` represent the gap closed by the dual bounds of their respective disjunctions.

The patterns observed in Table 3.7 are generally consistent with earlier findings. `VPC` continues to outperform `S-PDI`, which in turn outperforms `PDI`. Gap closure increases with disjunction size and tends to decrease with higher degrees of perturbation. Strengthened cuts yield greater benefits as either disjunction size or perturbation level increases, as previously seen. Notably, `CD` closes more gap at higher perturbation levels, helping to explain why `VPC` also improves under these conditions—a departure from the typical pattern but consistent with the underlying data.

One might expect disjunctive methods to close more of the integrality gap overall, but their performance depends heavily on where the cuts exert their strength. As shown in Balas and Kazachkov [12], disjunctive cuts—such as those in `VPC`, `S-PDI`, and `PDI` —often tighten regions of the LP relaxation that lie far from the LP relaxation solution. This limitation is amplified in parametric methods, where cuts may be generated from bases no longer near the LP solution. Matrix perturbations can further shift the direction of the cuts, compounding this effect and diminishing the practical strength of the resulting cuts.

Additional insight helps explain why `S-PDI` often appears to offer limited improvement over `PDI`. If the supporting term for a PDI does not change, then strengthening has no effect, leaving `S-PDI` equivalent to `PDI`. Further, in the case of 4-term disjunctions, `S-PDI` shows no gain over `PDI`, likely due to the small number of infeasible terms that become feasible, as seen in Table 3.6. This matters because `PDI` assigns a zero vector as the default certificate for infeasible terms. When these terms become feasible, the corresponding change in cut direction and bound can be substantial. Additionally,

Still, this does not mean that disjunctive methods, and `S-PDI` in particular, lack value. While they may not refine the LP relaxation near its optimal solution, as we will see next, the gap they do close often arises from regions that Gurobi's default cuts do not address. In Section 3.2.4, it may be this orthogonality that plays a key role in overall solver performance improvements.

Table 3.8 reports the percentage of integrality gap closed in the LP relaxation after root

Table 3.8: Average percent integrality gap closed after root node processing, grouped by degree of perturbation, size of disjunction, and element perturbed for test sets generated from MIPLIB 2017.

Degree	Terms	Element	VPC	S-PDI	PDI	Default
0.5	4	matrix	73.26	73.25	73.62	73.34
		rhs	51.76	48.47	49.65	50.20
	64	matrix	73.58	73.59	73.40	73.42
		rhs	53.47	53.02	52.69	49.72
2	4	matrix	80.24	80.39	80.41	80.40
		rhs	52.80	52.45	52.89	53.77
	64	matrix	80.61	80.24	79.39	80.12
		rhs	56.35	53.47	53.17	53.91

node processing of our collection of test sets, analogous to Table 3.2. Again, these values reflect the effect of all cuts generated at the root node, including the disjunctive cuts.

Table 3.8 shows some trends consistent with those in Table 3.7. On average, `VPC` performs best and `Default` worst, and gap closure increases with disjunction size. The benefit of strengthened variants grows under increased perturbation and terms. Improved effectiveness of disjunctions at higher perturbation again leads to greater gap closure across all disjunctive cut generators.

This time, however, the ordering amongst disjunctive cut generators becomes less clear. Default cut generation variability can obscure the advantage of stronger cuts, particularly when said cuts are not that much stronger.

Notably, for large disjunctions, `S-PDI` tightens the root relaxation in ways that both `Default` and `PDI` cannot, reinforcing the value of the proposed method in general.

PDI Generation Time Under Strengthening. Table 3.9 shows root node processing times. The trends align with earlier findings in Table 3.3: `PDI` is significantly faster than `VPC`, while `Default` typically offers a lower bound. `S-PDI` falls in between `VPC` and `PDI`, leaning more towards the latter due to the availability of warm starts when it solves LPs. As before, the processing times of all disjunctive methods grows with the number of disjunctive terms. These results indicate in general that strengthened PDIs retain most of the time advantage over `VPC` while frequently producing tighter root relaxations than `PDI`, completing the first pattern we aimed to validate for collection of test sets generated from MIPLIB 2017.

Table 3.9: Time to process root node, grouped by degree of perturbation, size of disjunction, and element perturbed for test sets generated from MIPLIB 2017.

Degree	Terms	Element	VPC	S-PDI	PDI	Default
0.5	4	matrix	1.39	0.480	0.435	0.314
		rhs	0.690	0.297	0.300	0.275
	64	matrix	11.9	2.14	1.06	0.336
		rhs	6.43	0.744	0.549	0.313
2	4	matrix	2.45	0.377	0.373	0.296
		rhs	0.794	0.248	0.234	0.239
	64	matrix	9.56	1.70	1.21	0.288
		rhs	4.36	0.546	0.515	0.234

3.2.4 Branch and Cut Performance

This subsection presents the second main finding of our computational study: strengthening PDIs can substantially reduce overall solve effort, even compared to both default disjunctive cut generators. We demonstrate this by evaluating the performance of strengthened versus default PDIs on test instances with solve times between 10 seconds and 1 hour. Results are split by comparisons of average measurements and cumulative distribution (a.k.a. *performance profiles*). Explanations of how this data is structured can be seen immediately below and in Section 3.2.4 for average and cumulative comparisons, respectively.

For average measurements, we compare branch and bound node count and solve time. Tables report the average percentage of wins, where a win is defined as achieving at least a 10% improvement. Column 2 specifices the percent of time `Default` wins against all other methods, and column 3 does analogously for `S-PDI`. The next five columns then show the percentage of wins over `Default` for the corresponding generators. These columns include two aggregate categories: `Any D`, which records wins by any disjunctive cut generator, and `Any P`, which aggregates wins across the parametric generators. We also list the number of base and test instances per group.

Effect og Degree of Perturbation. Tables 3.10 and 3.11 reports the win percentages of our disjunctive cut generators across different degrees of perturbation, with sample sizes noted at the end.

Algorithm 12 `Strengthened Parametric Disjunction, Parametric Cuts`

Require: $d, k, u, \theta, \mathcal{T}$, `prune`, `support`

$\{\mathcal{X}^t\}_{t\in T} \leftarrow$ `Branch-and-Bound` (MILP-k, d) ▷ Generate initial disjunction

$\Pi^k \leftarrow$ Balas and Kazachkov [12, Algorithm 1](MILP-k, $\{\mathcal{X}^t\}_{t\in T^k_{\text{feas}}}$) ▷ Get initial cuts from PRLPs

$\mathcal{V} \leftarrow \emptyset$, $B_{\text{all}} \leftarrow \emptyset$

for all $(\alpha, \beta) \in \Pi^k$ **do**. ▷ For each cut

  $\{v^t\}_{t\in T} \leftarrow \{0_{q+q_t}\}_{t\in T}$, $\{B^t\}_{t\in T} \leftarrow \{\emptyset\}_{t\in T}$

  for all $t \in \{\mathcal{X}^t\}_{t\in T^k_{\text{feas}}}$ **do** ▷ For each feasible term

    $B^t \leftarrow B$ such that $A^{\ell t}_{B\cdot}\bar{x}^t = b^{\ell t}_B$ and $\bar{x}^t \in \arg\max_{x\in\mathcal{Q}^{\ell t}}\{\alpha^\top x\}$ ▷ Get term's optimal basis

    $v^t \leftarrow \arg\max_{v\in\mathcal{D}^t(A^k,b^k,\alpha)}\{vb^{kt}\}$ ▷ Calculate its certificate

  end for

  $\mathcal{V} \leftarrow \mathcal{V} \cup \{\{v^t\}_{t\in T}\}$, $B_{\text{all}} \leftarrow B_{\text{all}} \cup \{\{B^t\}_{t\in T}\}$

end for

$S \leftarrow$ `Branch-and-Cut` (MILP-k, Π^k) ▷ Initialize the solution pool

for all $\ell \in \mathcal{T}_{ku\theta}$ **do** ▷ For each test instance

  $\Pi^\ell \leftarrow \emptyset$, $T_\ell \leftarrow T^\ell_{\text{feas}}$

  if prune **then** ▷ If pruning is enabled

    $z \leftarrow \min_{x\in\mathcal{S}^k\cap S}\{c^\ell x\}$ ▷ Get primal bound of S

    $T_\ell = \{t \in T : \min_{x\in\mathcal{Q}^{\ell t}}\{c^\ell x\}\} \le z\}$ ▷ Prune disjunction with it

  end if

  for $i \leftarrow 1$ **to** $|\mathcal{V}|$ **do** ▷ For each certificate/active set pair

    $\{v^t\}_{t\in T} \leftarrow \mathcal{V}[i]$, $\{B^t\}_{t\in T} \leftarrow B_{\text{all}}[i]$, $(\alpha, \beta) \leftarrow (0_n, 0)$

    if `support` **then** ▷ If disjunctive hull support desired

      $(\alpha, \beta) \leftarrow$ Algorithm 11$(k, \ell, \{\mathcal{X}^t\}_{t\in T_\ell}, \{v^t\}_{t\in T_\ell}, \{B^t\}_{t\in T_\ell})$ ▷ Generate supporting cut

    else

      $(\alpha, \beta) \leftarrow$ Theorem 2.2.4$(\ell, \{\mathcal{X}^t\}_{t\in T_\ell}, \{v^t\}_{t\in T_\ell})$

    end if ▷ Otherwise

    $\Pi^\ell \leftarrow \Pi^\ell \cup \{(\alpha, \beta)\}$ ▷ Generate valid cut

  end for

  $S \leftarrow S \cup$ `Branch-and-Cut` (MILP-ℓ, Π^ℓ) ▷ Solve with strengthened cuts

end for

Table 3.10: Average node wins by disjunctive cut generator and degree of perturbation.

Degree	Wins vs. All		Wins vs. `Default`					Count	
	`Default`	`S-PDI`	`VPC`	`S-PDI`	`PDI`	Any P	Any D	Base	Test
0.5	14.88	13.41	33.90	38.05	38.54	53.17	59.27	77	410
2.0	12.00	17.00	37.00	42.00	34.67	52.33	60.67	64	300

Table 3.11: Average run time wins by disjunctive cut generator and degree of perturbation.

Degree	Wins vs. All		Wins vs. `Default`					Count	
	`Default`	`S-PDI`	`VPC`	`S-PDI`	`PDI`	Any P	Any D	Base	Test
0.5	23.66	20.73	26.34	40.24	37.80	55.37	61.22	77	410
2.0	21.00	21.33	28.00	39.00	40.00	54.67	59.33	64	300

Table 3.12: Average node wins by disjunctive cut generator and disjunctive terms.

Terms	Wins vs. All		Wins vs. `Default`					Count	
	`Default`	`S-PDI`	`VPC`	`S-PDI`	`PDI`	Any P	Any D	Base	Test
4	13.80	15.01	35.59	39.71	37.77	53.75	59.81	96	413
64	13.47	14.81	34.68	39.73	35.69	51.52	59.93	70	297

We observe the following trends for branch and bound nodes across perturbation levels, mostly aligning with expectations. `S-PDI` improves relative to `PDI` as perturbation increases, since greater perturbation creates more opportunity for strengthening. `Any P` wins less often under higher perturbation, likely because even tight cuts may lie farther from the LP relaxation solution. `PDI` performs significantly worse with more perturbation, consistent with prior findings that default PDIs can refine suboptimal bases and often fail to support the disjunctive hull. These effects help justify the higher cost of `VPC`, which becomes more advantageous as perturbation increases.

One result deviates from our expectations: we anticipated that `S-PDI` would degrade with more perturbation—albeit less severely than `PDI` —but it does not. This is plausible, as the 2-degree disjunctions appear stronger than those at 0.5 degrees, as shown in Table 3.7.

We find no consistent trend emerges in run time when breaking down by degree of perturbation, but this is unsurprising given the unpredictable relationship between disjunctive cuts and simultaneous improvements in node count and solve time [12].

Effect of Number of Disjunctive Terms. Tables 3.12 and 3.13 reports the win percentages of our disjunctive cut generators across different sizes of disjunction, with sample sizes noted at the end.

When examining branch and bound nodes, we see that `S-PDI`'s advantage over `PDI` increases with more disjunctive terms—an expected result, as additional terms introduce

Table 3.13: Average run time wins by disjunctive cut generator and disjunctive terms.

Terms	Wins vs. All		Wins vs. `Default`					Count	
	`Default`	`S-PDI`	`VPC`	`S-PDI`	`PDI`	Any P	Any D	Base	Test
4	17.92	22.76	29.78	42.62	39.47	58.35	64.65	96	413
64	28.96	18.52	23.23	35.69	37.71	50.51	54.55	70	297

Table 3.14: Average node wins by disjunctive cut generator and element perturbed.

Element	Wins vs. All		Wins vs. `Default`					Count	
	`Default`	`S-PDI`	`VPC`	`S-PDI`	`PDI`	Any P	Any D	Base	Test
matrix	13.91	15.0	35.87	38.91	36.3	52.39	60.0	89	460
rhs	13.20	14.8	34.00	41.20	38.0	53.60	59.6	59	250

more opportunities for weakening under perturbation (Table 3.6). While one might anticipate that more terms and stronger cuts (`VPC` and `S-PDI`) would reduce node counts, that pattern does not appear here. This is consistent with the behavior of cuts that close only a small portion of the integrality gap, as shown in Table 3.7 for our MIPLIB 2017 instances. In such cases, adding cuts can actually increase the number of nodes processed; see Shah et al. [59] for further discussion.

When examining run time, the expected patterns emerge. Disjunctive methods become less effective as the number of terms increases, due to the growing cost of cut generation. This aligns with the findings of Kelley et al. [46] for PDIs and Balas and Kazachkov [12] for `VPC`. Given these relationships, it is unsurprising that `VPC` shows a clear separation from the parametric methods in run time, as its cuts are significantly more expensive to generate, again consistent with prior work [46]. As a result, `PDI` degrades the least among the disjunctive methods, while `Default` improves in relative performance.

Effect of Perturbed Element. Tables 3.14 and 3.15 reports the win percentages of our disjunctive cut generators across different elements perturbed, with sample sizes noted at the end.

When examining branch and bound nodes, we find that `S-PDI` consistently outperforms `PDI`, as expected given its stronger cuts. For both methods, rhs exhibits greater root node strength than matrix on average (Table 3.8), which helps explain its relative performance.

Table 3.15: Average run time wins by disjunctive cut generator and element perturbed.

Element	Wins vs. All		Wins vs. `Default`					Count	
	`Default`	`S-PDI`	`VPC`	`S-PDI`	`PDI`	Any P	Any D	Base	Test
matrix	24.78	22.39	25.43	40.00	36.52	54.57	59.78	89	460
rhs	18.40	18.40	30.00	39.20	42.80	56.00	61.60	59	250

One might still expect matrix to outperform rhs due to greater potential for strengthening, especially deeper in the tree. However, even if such strengthening occurs, the non-monotonic behavior observed by Shah et al. [59] could still explain why this advantage does not translate into fewer nodes.

When examining run time, our results largely align with prior literature. `PDI` degrades more than `S-PDI`, consistent with Table 3.6, which shows substantial room for strengthening in both matrix and rhs perturbations. Table 3.8 further indicates that disjunctive cuts have a greater average effect on tightening the root relaxation for rhs than for matrix. Given that such cuts are typically dense, they may remain active deeper in the tree and slow down node processing more [12]. This could explain why `S-PDI` improves over `PDI` for rhs perturbations in terms of nodes processed but loses that advantage when considering run time, while this does not happen for matrix perturbations. Similarly, `VPC` wins less often on run time than on nodes possibly due to its high generation cost and strong cuts, whereas `PDI`, with its weaker and faster cuts, does not degrade as much.

We can also use performance profiles to show that PDIs outperform both VPCs and Gurobi with default settings. We analyze Figures 3.6 to 3.8 to compare overall branch and cut solve time of Gurobi configured with `VPC`, `S-PDI`, `PDI`, and `Default`. In each figure, the left panel is a performance profile of `S-PDI` and `PDI` relative to `VPC` over all test instances, where a value of -0.5 denotes a 50% speedup and +0.5 a 50% slowdown on instances solved by all methods. The right panel compares `VPC`, `S-PDI`, and `PDI` against `Default`, restricted to instances where `Default` required at least 20 minutes. "Best" denotes the virtual best over all configurations. The "Overall" and "Overall > 20" columns of Table 3.16 report, respectively, the counts of unique base instances and total experiments that reached optimality, and the subset for which `Default` took > 20 minutes. These need

not match the count of test instances in "Root Node" because (a) each test instance is rerun for every degree, (b) some runs that finish root-node processing still time out before optimality, and (c) some base instances fail to yield nonparametric disjunctive cuts for certain (degree, term) pairs—so the parametric methods cannot start—whereas many runs that do receive certificates subsequently reach optimality.

Table 3.16: Unique test instance and total experiment count by setup

Element	Root Node		Overall		Overall > 20	
	Base	Test	Base	Experiments	Base	Experiments
Matrix	89	256	89	460	24	51
Objective	106	578	106	1357	30	126
RHS	59	133	59	250	8	21

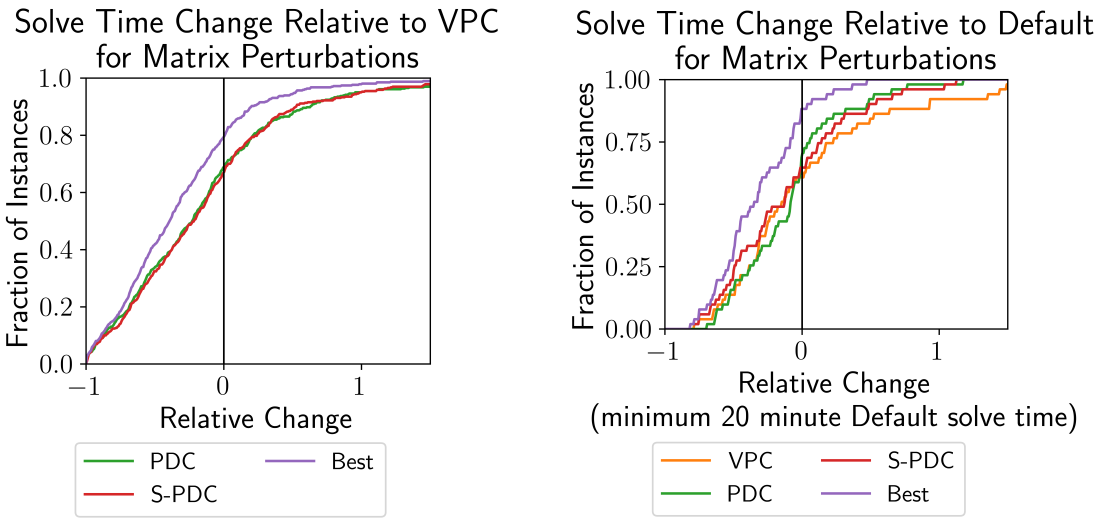


Figure 3.6: Performance profiles for matrix perturbations relative to `VPC` (left) and to default Gurobi on long-running instances (right).

Matrix perturbations: relative to `VPC`. In Figure 3.6 (left), both `S-PDI` and `PDI` beat `VPC` on roughly 65% of instances. The virtual best improves on `VPC` in about 80% of cases, indicating complementary strengths between `S-PDI` and `PDI`. This aligns with Tables 3.7 to 3.9: `S-PDI` and `PDI` trade a small amount of cut strength for markedly lower generation cost, yielding better end-to-end times.

Matrix perturbations: relative to `Default`. In Figure 3.6 (right), `VPC`, `S-PDI`, and `PDI` each outperform `Default` on approximately 60–65% of instances with solve times above

20 minutes. The virtual best wins on nearly 90% of these cases, with about half solving at least twice as fast. This points to a usual branch and bound tradeoff: stronger root cuts increase root processing time, but they tighten the relaxation globally. On harder, long-running instances, implicit pruning from the tighter relaxation can accumulate and the upfront cost is amortized, creating opportunity to lower total solve time.

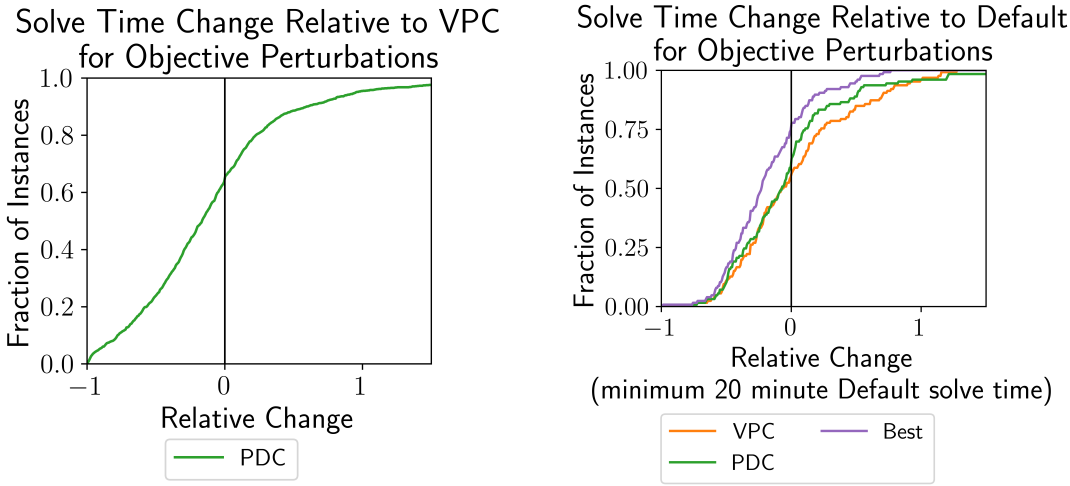


Figure 3.7: Performance profiles for objective perturbations relative to `VPC` (left) and to default Gurobi on long-running instances (right).

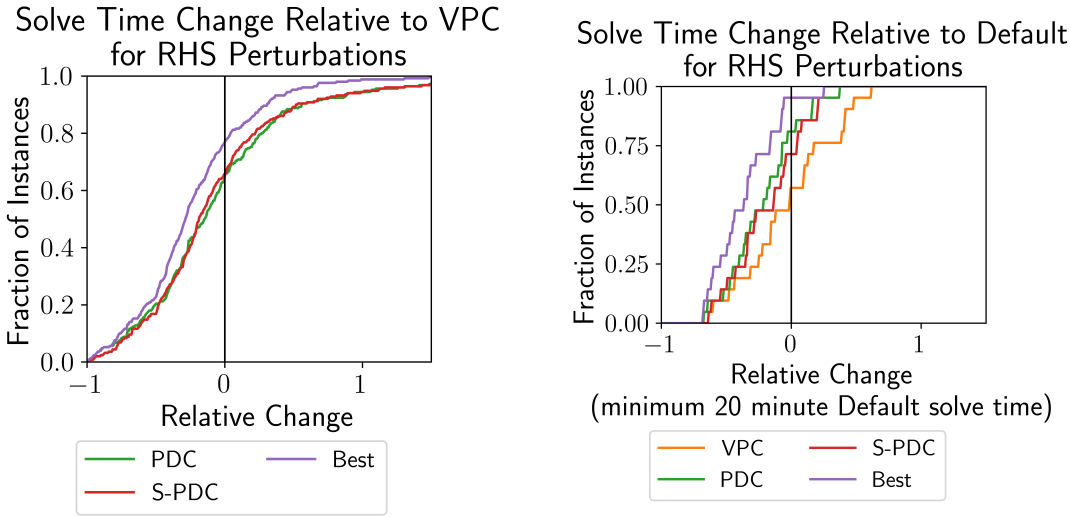


Figure 3.8: Performance profiles for right-hand-side perturbations relative to `VPC` (left) and to default Gurobi on long-running instances (right).

Objective and right-hand side perturbations. The same patterns hold for objective-coefficient and right-hand-side perturbations: `S-PDI` and `PDI` compare favorably to `VPC` on

Table 3.17: Average node wins by disjunctive cut generator and default instance run time bracket.

Bracket	Wins vs. All		Wins vs. `Default`					Count	
	`Default`	`S-PDI`	`VPC`	`S-PDI`	`PDI`	Any P	Any D	Base	Test
short	14.43	15.12	32.99	38.83	39.18	53.95	60.14	74	291
medium	14.71	15.07	35.66	39.34	33.82	51.47	59.56	60	272
long	10.20	14.29	38.78	42.18	38.10	53.06	59.86	36	147

Table 3.18: Average run time wins by disjunctive cut generator and default instance run time bracket.

Bracket	Wins vs. All		Wins vs. `Default`					Count	
	`Default`	`S-PDI`	`VPC`	`S-PDI`	`PDI`	Any P	Any D	Base	Test
short	32.65	20.27	14.09	36.43	35.40	51.20	53.95	74	291
medium	17.65	22.79	31.25	39.34	41.18	56.99	63.60	60	272
long	11.56	19.05	44.90	46.94	40.82	59.18	67.35	36	147

the left panels, and all three disjunctive generators improve on `Default` for long-running instances on the right panels (Figures 3.7 and 3.8). These results reinforce that the observed trends are robust across perturbation types and not specific to the constraint matrix.

Performance by Solve-Time Bracket. Tables 3.17 and 3.18 reports win percentages for each disjunctive cut generator, grouped by *solve time bracket*, with sample sizes shown at the end. **Short** includes test instances that `Default` solves in 10–60 seconds, **medium** covers 60–600 seconds, and **long** includes those requiring more than 600 seconds.

When examining nodes processed by solve time bracket, we observe that longer runs increasingly favor `S-PDI` and `VPC`, in alignment with previously seen trends. This is expected, as both methods produce relatively stronger cuts, and pruning near the root has greater impact in larger branch and bound trees given the size of subtrees avoided. The growing advantage of `S-PDI` with longer solve times aligns with Balas and Kazachkov [12], which observed a similar trend for `Calc Disj, Calc Cuts`. `PDI` fails to exhibit the same monotonic improvement across brackets, and a key distinction may lie in the fact that both `S-PDI` and `VPC` generate cuts that support the disjunctive hull. Without disjunctive hull support, `PDI` may not be able to prune as aggressively near the root where subtrees grow with processing time.

When comparing run time win percentages across brackets, we observe that short runs

follow expected patterns: disjunctive methods win less often on time than on nodes, likely due to cut density slowing LP solves [12]. Surprisingly, the opposite occurs in long runs, where run time wins exceed node wins. One explanation may be that disjunctive hull-supporting cuts enable the solver to skip more cut generation later, offsetting the cost of denser LPs. Alternatively, we could have shallower trees, which lead to more effective parallelization and improvements in overall solve time despite increased node counts.

Relative Performance of `S-PDI`. Some patterns hold consistently across all groupings and metrics. Across every table, disjunctive cuts (`Any D`) achieve substantial win rates in both nodes processed and run time, consistent with their known strength. The methods also exhibit orthogonality: a given disjunctive approach often outperforms `Default` on instances where others do not. Notably, `S-PDI` outperforms all other methods 15–20% of the time. These findings underscore the practical value of using a diverse set of disjunctive cut generators, including `S-PDI`, as they complement one another and collectively enhance performance. They also reinforce this paper's second main claim: that strengthening PDIs can significantly improve overall branch and cut effectiveness.

3.3 Conclusion

Theoretically, our results demonstrate that pruning disjunctions using an incumbent solution yields smaller, more targeted disjunctions without compromising validity, as the resulting cuts remain correct for any optimal solution. We also show that the dual solutions to our cut-tightening LPs uniquely certify support for individual disjunctive terms, and when embedded in the parametric framework of Kelley et al. [46], produce cuts that provably support the full disjunctive hull. Empirically, both strengthening routines, used separately or together, tighten the root relaxation while preserving much of the original framework's speed advantage. At the branch and cut level, our best strengthened variant consistently outperforms the baseline in solve time. More broadly, disjunctive cuts improve solve time in the majority of cases across our test set.

These results come with important caveats. It remains an open question both when

disjunctive cuts improve MILP performance and which generator will be most effective for a given instance. Prior work has shown that disjunctive cuts tend to reduce the number of nodes more often than overall run time [12, 46], likely because their density increases the cost of node processing. Yet, for our longest solves, we observe the opposite: disjunctive cuts improve run time more often than node count, a result that warrants further investigation. In particular, for matrix perturbations, it may be beneficial to explore generating the supporting certificate from the initial (non-parametric) cut rather than the parameterized form. This would ensure the parametric cut remains parallel to the original and could potentially yield further gains in solver performance.

Several natural extensions emerge from this work. First, given the growing use of learning-based methods to improve MILP performance, a promising direction would be to train a neural network to predict which disjunctive cut generators to apply and under what conditions. Second, since our approach implicitly warm-starts a MILP using a disjunction, it would be valuable to compare its performance with that of explicitly warm-starting the MILP using the same disjunction, to better understand the trade-offs between the two strategies. Finally, our strengthening routine is not limited to disjunctive cuts. Because it can accept arbitrary cut coefficients, it may be worth exploring its use in parameterizing other expensive cuts across related MILPs, such as interdiction cuts in bilevel optimization, using Algorithm 11.

Our results carry several key implications. Theoretically, we provide guarantees for maximizing the strength of PDIs. Computationally, our findings offer valuable guidance for warm-starting the dual side of a MILP solve, particularly the pool of cutting planes, which is often both neglected in the literature and a frequent bottleneck in practice. From an applied perspective, this opens the door to significantly improving the solve times of real-time and decomposed MILPs, which are increasingly important as problem size and speed demands grow. Overall, this work adds to the growing body of evidence that solver performance can be improved not only by exploiting single-instance structure, but also by leveraging structural patterns across sequences of related problems

Chapter 4

Comparing Node and Cut Pool Warm Starts

This chapter presents a controlled comparison of two warm-starting strategies for branch and cut that reuse the *same* disjunction extracted from a previously solved (or partially solved) MILP instance. Both strategies aim to accelerate reoptimization on a related instance by transferring a disjunction-induced dual approximation rather than reconstructing it from scratch. The central question is practical: when a disjunction is available, should it be reused by explicitly initializing the node pool, or by implicitly strengthening the root relaxation via a warm-started cut pool?

The first approach, proposed by Güzelsoy [40, Section 4.3], performs a *node warm start* by explicitly reusing the disjunction encoded by the leaves of a partial branch-and-bound tree. Each leaf defines a disjunctive term whose relaxation can be updated to the new instance and reoptimized from stored LP bases, thereby seeding the search with a set of candidate nodes and their associated bounds. The second approach, developed in this dissertation, performs a *cut warm start* by implicitly reusing the same disjunction through a certificate of validity for a parametric family of disjunctive cuts. On a new instance, these certificates are evaluated to instantiate valid cutting planes that tighten the initial relaxation and improve the dual bound without explicitly instantiating multiple term subproblems.

A key challenge in comparing these approaches is that warm starts can differ in strength

at initialization due to their underlying disjunctions. To isolate the algorithmic trade-offs, we construct a single disjunction and reuse it in both methods, ensuring that neither is given an inherent advantage at the root. The comparison therefore reflects differences in how disjunctive information propagates through branch and cut: explicit node reuse emphasizes reusing multiple term relaxations and their bases, while implicit cut reuse emphasizes transferring the same structural information through globally valid inequalities.

The remainder of the chapter is organized as follows. Section 4.1 provides a disjunction-focused guide to warm starting, formalizing how disjunctions induce dual and primal bounding functions and how those bounding objects are re-evaluated on a related instance under explicit and implicit reuse. We then turn to computational evidence: Section 4.2.1 describes the test-set construction and experimental protocol, and Section 4.2 reports solve-time, node, and LP-iteration results that identify regimes in which each approach is preferred. We conclude by summarizing the observed trade-offs and their implications for warm-start design.

4.1 Warm Starting with Disjunctions

This section provides a practical, disjunction-focused guide to warm starting a MILP solve. The goal is to make explicit how disjunctions induce bounding functions and how those bounding functions can be transferred from a solved (or partially solved) instance to a related, unsolved instance. Throughout, we emphasize warm starts that reuse disjunctions either *explicitly* through a partial branch-and-bound tree or *implicitly* through disjunctive inequalities certified by a reused proof of validity. In practice, non-learning based reoptimization can also reuse auxiliary objects such as partial solutions, pseudocosts, and solver parameters; Patel [54] provides a broad review of these additional options. For simplicity, we assume a pool of previously found solutions S is available.

4.1.1 Preliminaries and Setup

Fix a base instance $k \in \mathcal{K}$ that has already been solved (or partially solved) by branch and cut. Consider the partial branch-and-bound tree produced during the solve of k. We

assume it contains enough information to reconstruct, for each leaf, the constraints induced by branching decisions along its root-to-leaf path, and any reduced-cost fixings discovered during strong branching. (Here, *reduced-cost fixing* refers to variable fixings identified while evaluating candidate branching directions that can be pruned either by infeasibility or by bound.)

This partial tree induces a valid disjunction by treating each leaf as a disjunctive term. Intuitively, each term corresponds to an LP relaxation augmented with the branching constraints and fixings accumulated along that leaf path. To use such a disjunction for warm starting, we need to (i) extract the disjunction, (ii) ensure it is valid for the intended MILP family, and (iii) evaluate the induced dual and primal bounding functions on a new instance.

4.1.2 Step 1: Extract a Disjunction from a Partial Tree

Recall $\{\mathcal{Q}^{kt}\}_{t\in T}$ denotes the collection of term relaxations induced by the leaves of tree for base instance k. Each $\mathcal{Q}^{kt}$ is obtained by starting from the LP relaxation $\mathcal{P}^k$ and appending the branching constraints and reduced-cost fixings associated with leaf t, which can be found in disjunctive term $\mathcal{X}^t$.

A subtlety arises because reduced-cost fixings are sometimes recorded only on one side of a branching decision. During strong branching, one direction may be proven prunable and therefore never instantiated as a node in the tree; as a consequence, the surviving branch may accumulate additional fixings beyond those imposed by the parent's explicit branching constraint, without any explicit record of a complementary sibling term. To preserve disjunction validity, each such reduced-cost fixing can be modeled as an implicit branching decision: one child corresponds to the pruned direction, while the other continues with the accumulated fixings. Repeating this construction for each strong-branching fix yields a sequence of binary splits that culminates in a leaf whose path encodes precisely the full set of fixings applied in the original term.

We generally recommend *not* storing cutting planes that were added locally to tighten individual terms during the base solve, since such cuts can usually be regenerated quickly when resolving the term on a new instance. Nevertheless, it is possible to parameterize and reuse such cuts as well; see Güzelsoy [40] for a detailed discussion in the context of

right-hand-side perturbations.

The exception arises in the case of implicit warm starting via PDIs, whose computational cost and strength motivate reuse across instances. We generate cuts against the disjunctive hull $\mathcal{P}_D^k$, and for each generated inequality, we compute, when necessary, and store a corresponding Farkas certificate $\{v^t\}_{t\in T}$ establishing its validity. Let $\mathcal{V}$ denote the resulting collection of stored certificates.

Finally, to support efficient reoptimization, we store an optimal simplex basis for each term LP relaxation. This allows fast reconstruction of primal/dual solutions and efficient re-solving when the term is re-evaluated for a modified instance.

Additional implementation details appear in the pseudocode of appendix B and in the reference implementation of `VPCEventHandler::saveInformationWithPrunes` in vpc [3].

4.1.3 Step 2: Check Disjunction Validity

For warm starting, the extracted disjunction must be valid for the integer feasible set of interest. In the simplest setting, where all branching constraints and reduced-cost fixings correspond to integrality-preserving bound changes, validity reduces to verifying that the induced leaf structure partitions all integer-feasible points. Operationally, this amounts to requiring that the term structure forms a full binary branching tree over the recorded branching decisions. The routine Algorithm 17 (implemented in `partialBBdisjunction.cpp` vpc [3]) enforces this condition by confirming that every branching decision has two complementary child terms that differ only in the final bound, thereby ensuring no integer-feasible region is omitted.

This viewpoint extends naturally to bounded MILPs and to instance families: when a bound change is not matched by a complementary branch, validity can still be maintained provided the "unmatched" bounds are interpreted as bounds that hold over the entire MILP family being warm started. In other words, validity is preserved if every bound tightening that is treated as global is indeed a defining restriction of $\mathcal{K}$ (or of the specific family $\mathcal{P}_D^k$ under consideration).

4.1.4 Step 3: Re-evaluate the Dual Bounding Function

Once a valid disjunction has been extracted, the first task in warm starting a new instance is to reconstruct a valid dual bounding function. In branch and cut, the dual certifies global progress and informs node pruning, so restoring dual feasibility is the essential prerequisite for meaningful reuse. We therefore begin by describing how to evaluate the disjunction-induced dual bound on a new instance, either explicitly through term relaxations or implicitly through cutting planes. The primal update is addressed subsequently.

Explicit Disjunctive Warm Starts

Given a valid disjunction $\{\mathcal{X}^t\}_{t\in T}$ extracted from a base solve, an *explicit* disjunctive warm start initializes the new solve of an instance $\ell \in \mathcal{K}$ by directly evaluating the disjunction-induced dual bound and, optionally, by re-solving term relaxations to restore feasibility certificates.

Applying the disjunction to a new instance. For the warm-started instance ℓ, we prepend each disjunctive term with the LP relaxation $\mathcal{P}^\ell$ to apply the term-specific branching constraints and fixings defining that term. We then recompute the LP tableau using the stored basis B^t as a warm start for the term LP. If local cuts were stored for a term, they may be re-evaluated and re-added, but we generally omit them to keep the procedure lightweight.

Thorough versus minimal evaluation. There are two natural modes for explicitly reusing a disjunction:

- *Thorough:* Resolve *all* term LPs to optimality under instance ℓ.
- *Minimal:* Resolve only those terms that have lost dual feasibility and lack a proof of primal infeasibility, stopping once a dual feasibility is recovered.

Let $\{B^t\}_{t\in T}$ be updated to reflect the active basis after resolving with either approach. Notice that for right-hand-side perturbations, the term bases do not change in the minimal

approach as dual solutions maintain feasibility, requiring no re-solving. For more general perturbations, however, some terms may lose dual feasibility and require reoptimization.

The essential requirement is dual feasibility. As we will see, a dual-feasible solution for each term yields a valid lower bound on that term's relaxation, and the minimum of these term bounds provides a valid lower bound on the MILP itself.

Dual bound evaluation. Let $x^t = (A^{\ell t}_{B^t\cdot})^{-1} b^{\ell t}_{B^t}$ for all $t \in T$. Under either approach, the dual bounding function induced by the disjunction is evaluated as

$$F_{\{\mathcal{X}^t\}_{t\in T}}((A^\ell, b^\ell, c^\ell)) \coloneqq \min_{t\in T} c^\ell x^t.$$

In the thorough case, this is equivalent to

$$F_{\{\mathcal{X}^t\}_{t\in T}}((A^\ell, b^\ell, c^\ell)) \coloneqq \min_{t\in T} \min_{x\in\mathcal{Q}^{\ell t}} c^\ell x.$$

Choosing between the thorough and minimal approaches is a central question in disjunctive warm starting. The two present a clear trade-off. The thorough approach typically produces stronger initial bounds and increases the likelihood of pruning nodes by bound, but incurs greater upfront computational cost. In contrast, the minimal approach computes weaker initial bounds with lower immediate cost, at the expense of potentially reduced pruning and additional work later in the branch-and-cut process.

Implicit Disjunctive Warm Starts

An *implicit* disjunctive warm start transfers the effect of a disjunction through cutting planes rather than through explicit evaluation of its terms. Specifically, we generate inequalities valid for the disjunctive hull $\mathcal{P}^k_D$ together with their corresponding certificates of validity $\mathcal{V}$, thereby inducing a parametric family of valid inequalities (see Definition 2.2.3). These certificates enable efficient instantiation of family members on related instances while preserving validity.

Instantiating cuts on a new instance. For the warm-started instance ℓ, we evaluate the parameterization induced by each stored certificate $\{v^t\}_{t\in T} \in \mathcal{V}$ to obtain a cut set Π^ℓ. In our implementation, this corresponds to applying Theorem 2.2.4 or Algorithm 11 (as desired) to $(\ell, \{\mathcal{X}^t\}_{t\in T}, \{v^t\}_{t\in T})$ for all $\{v^t\}_{t\in T} \in \mathcal{V}$.

Dual bound evaluation. The objective is again to obtain a valid lower bound on the MILP. In the implicit case, this is achieved by strengthening the LP relaxation with inequalities certified to be valid for the MILP feasible region, thereby refining the associated dual bounding function. The resulting dual bound is obtained by evaluating the LP relaxation augmented with the instantiated cut pool Π^ℓ:

$$F_{\Pi^\ell}((A^\ell, b^\ell, c^\ell)) := \min\{c^\ell x : x \in \mathcal{P}^\ell,\ \alpha^\top x \geq \beta\ \ \forall(\alpha, \beta) \in \Pi^\ell.$$

4.1.5 Step 4: Update the Primal Bounding Function

Warm starting, however, requires more than restoring a dual bound. To initialize branch and cut meaningfully, we must also provide a valid primal bounding function. We therefore next describe how previously computed solutions, or solutions recovered from term relaxations, can be transferred to the new instance.

A primal update can be obtained by evaluating the solution pool on the new instance. As in Section 4.1.4, the solution pool S can be constructed in either a thorough or minimal manner:

- *Thorough:* Resolve *all* term LPs to optimality under instance ℓ, and let S be the set of relaxation optimal solutions that satisfy MILP feasibility.
- *Minimal:* Let S be the set of MILP-feasible solutions identified in a previous solve.

By assumption, S initially takes the minimal form, but it may be updated as follows if the thorough approach is employed:

$$S = \mathcal{S}^\ell \cap \{X_t^\ell : t \in T\} \text{ where } X_t^(\ell) := \arg\min_{y\in\mathcal{Q}^{\ell t}} c^\ell y$$

In either case, the primal bound for the warm start is evaluated as

$$G_S((A^\ell, b^\ell, c^\ell)) \coloneqq \min_{x \in S} G_x((A^\ell, b^\ell, c^\ell)).$$

As with the dual update, the two approaches involve a trade-off. The minimal approach is computationally inexpensive but may omit solutions that were infeasible in the base instance and become feasible under the current instance. The thorough approach addresses this possibility but requires resolving all term relaxations, which can be expensive. As before, stronger primal bounds reduce the size of the branch-and-bound tree needed to prove optimality; however, any reduction in search effort must offset the computational cost incurred in generating the bound.

4.1.6 Step 5: Warm-Started Solve and Termination

At this stage, we have constructed initial dual and primal bounding functions for instance ℓ. Warm starting consists precisely of initializing branch and cut with these bounding objects and determining whether they already certify optimality. If they do not, they serve as the starting point for further refinement through branching and cut generation.

Warm starting provides initial bounding functions for instance ℓ. The solve may terminate immediately if the primal and dual bounds are sufficiently tight; otherwise, branch and cut proceeds using whichever subset of warm-start information is available.

Optimality gap and termination test. Let L be the best available dual bound and U the best available primal bound for instance ℓ. A common relative optimality gap is

$$\mathrm{gap}(\ell) \coloneqq \frac{U - L}{\max\{1, |U|\}},$$

and the solve may be terminated whenever $\mathrm{gap}(\ell)$ is below a prescribed tolerance.

Calling branch and cut with warm-start objects. If the gap test fails, we call branch and cut on instance ℓ using any available warm-start data:

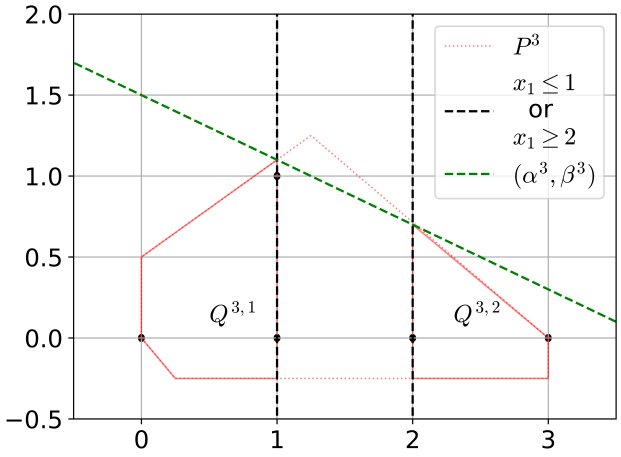
- If S is available, initialize the incumbent with $\arg\min_{x \in S} G_x((A^\ell, b^\ell, c^\ell))$.

- If a disjunction $\{\mathcal{X}^t\}_{t\in T}$ is available, initialize the node pool using those terms that (i) do not have a proof of primal infeasibility and (ii) have a term bound better than the current global primal bound.
- If a cut pool Π^ℓ is available, instantiate the solver's cut pool with it and proceed with cut management as in a default solve.

Note that since Π^ℓ is meant to account for the branching decisions encoded in $\{\mathcal{X}^t\}_{t\in T}$, a user would warm start with one of them, not both.

4.1.7 Tutorial

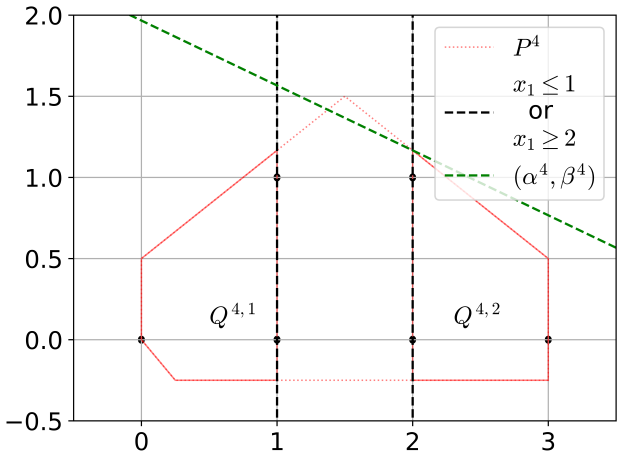
This subsection provides a guided example of disjunctive warm starting.



$$
\begin{aligned}
\min_{x\in\mathbb{R}^2} \quad & -x_2 \\
x_1 \quad & \geq 0 \\
3x_1 \quad & -5x_2 \geq \tfrac{5}{2} \\
-5x_1 \quad & -7x_2 \geq -15 \\
-x_1 \quad & \geq -3 \\
& x_2 \geq -\tfrac{1}{4} \\
x_1 \quad & +x_2 \geq 0 \\
x_1, \quad & x_2 \in \mathbb{Z}
\end{aligned}
\tag{MILP-3}
$$

Figure 4.1: The LP relaxation of MILP instance 3 with the disjunction $\{\mathcal{X}^1, \mathcal{X}^2\}$ and disjunctive inequality $((\alpha^3, \beta^3))$ applied to it.

Example 4.1.1. *Consider a solved base instance MILP-3, artifacts from which will warm start instance MILP-4. Let $\mathcal{X}^1 := \{x \in \mathbb{R}^2 : -x_1 \geq -1\}$ and $\mathcal{X}^2 := \{x \in \mathbb{R}^2 : x_1 \geq 2\}$ define disjunction, $(\alpha^3, \beta^3) = ([-2, -5]^\top, -7.5)$ define a disjunctive cut, and $S = \{[1, 1]^\top\}$, all valid for MILP-3.*



$$
\begin{aligned}
\min_{x \in \mathbb{R}^2} \quad & -x_2 \\
x_1 \quad & \geq 0 \\
3x_1 \quad & -5x_2 \geq \tfrac{5}{2} \\
-5x_1 \quad & -7x_2 \geq -18 \\
-x_1 \quad & \geq -3 \\
& x_2 \geq -\tfrac{1}{4} \\
x_1 \quad & +x_2 \geq 0 \\
x_1, \quad & x_2 \in \mathbb{Z}
\end{aligned}
\tag{MILP-4}
$$

Figure 4.2: The LP relaxation of MILP instance 4 with the disjunction $\{\mathcal{X}^1, \mathcal{X}^2\}$ and PDI $((\alpha^4, \beta^4))$ applied to it.

Step 1: Extract the Disjunction. *For ease of exposition, we work with $\{\mathcal{X}^1, \mathcal{X}^2\}$. MILP-3 has constraints indexed [1, ..., 6] with disjunctive constraints indexed 7 for each term. We record each term's optimal basis below. If an implicit warm start is desired, generate disjunctive cuts $\Pi^3 = \{(\alpha^3, \beta^3)\}$ and their corresponding Farkas certificates of validity.*

$$
\begin{aligned}
B^1 &= \{2, 7\}, \qquad B^2 = \{3, 7\}, \\
\mathcal{V} &= \{v^1, v^2\} = \{Lemma\ 2.1.2(\alpha^3, A^{3,1}_{B^1\cdot}), Lemma\ 2.1.2(\alpha^3, A^{3,2}_{B^2\cdot})\} \\
&= \{[0, 1, 0, 0, 0, 0, 5]^\top, \{[0, 0, \frac{5}{7}, 0, 0, 0, \frac{11}{7}]^\top\}
\end{aligned}
$$

Step 2: Check Disjunction Validity. *$\{\mathcal{X}^1, \mathcal{X}^2\}$ is valid by inspection.*

Step 3 (Explicit, Thorough): Re-evaluate Dual Bounding Function. *If explicitly warm starting, apply the disjunction $\{\mathcal{X}^1, \mathcal{X}^2\}$ to $\mathcal{P}^4$ yielding the term relaxations $\{\mathcal{Q}^{31}, \mathcal{Q}^{32}\}$. Thoroughly evaluate the dual bound*

$$
F_{\{\mathcal{X}^1, \mathcal{X}^2\}}((A^4, b^4, c^4)) = \min_{t \in \{1,2\}} \min_{x \in \mathcal{Q}^{4,t}} -x_2 = min\{-\frac{7}{6}, -\frac{7}{6}\} = -\frac{7}{6}
$$

Step 3 (Implicit): Re-evaluate Dual Bounding Function. *If implicitly warm starting, generate PDIs* Π^4 *from Farkas certificates* $\mathcal{V}$*, disjunction* $\{\mathcal{X}^1, \mathcal{X}^2\}$*, and MILP-4, applying them to* $\mathcal{P}^4$*, from which dual bound is evaluated.*

$$\Pi^4 = \{Theorem\ 2.2.4(\{v^1, v^2\}, \{\mathcal{X}^1, \mathcal{X}^2\}, MILP - 4)\} = ([-2, -5]^\top, \frac{59}{6})$$
$$F_{\Pi^4}((A^4, b^4, c^4)) = \min_{x \in \mathcal{P}^4} \left\{ -x_2 : -2x_1 - 5x_2 \geq \frac{59}{6} \right\} = -\frac{7}{6}$$

Step 4 (Minimal): Update the Primal Bounding Function. *Check the solution pool for the best solution that is feasible for MILP-4 and record its objective value.*

$$G_S((A^4, b^4, c^4)) = [0, -1][1, 1]^\top = -1$$

Step 5: Warm-Started Solve and Termination*. Since gap(4) = 1/6, solve MILP-4 to optimality with* `branch and cut`*, using* $(S, \{\mathcal{X}^1, \mathcal{X}^2\})$ *for explicit warm starting or* $\left(S, \Pi^4\right)$ *for implicit warm starting.*

4.1.8 Discussion and Practical Guidance

The explicit and implicit approaches reuse the same disjunction-induced dual bounding function in different ways. Their effectiveness is regime-dependent.

Explicit warm starting, as studied in Güzelsoy [40], is most effective when the stored partial tree constitutes a meaningful portion of the terminal branch-and-bound tree for the new instance. In such cases, term bounds and stored bases translate directly into reduced search, since the disjunction encodes structural information that remains relevant. However, explicit warm starts are sensitive to instance perturbation. If the modified instance differs sufficiently from the base instance, previously pruned regions may need to be reconsidered, and stored bounds may no longer reflect the structure of the new tree. In the worst case, this can lead to additional node processing relative to a default solve. More commonly, if the transferred tree is too shallow relative to the new search space, the warm start simply

has negligible impact.

Implicit warm starting, as developed in chapters 2 and 3 and analyzed computationally in Balas and Kazachkov [12], transfers the same structural information through disjunctive inequalities rather than explicit term evaluation. This approach offers greater flexibility: cuts can be managed dynamically by the solver, and ineffective inequalities can be aged out or removed as the solve progresses. Moreover, because the strengthened relaxation is global, implicit warm starts avoid instantiating multiple independent subtrees and instead begin from a single, tighter root relaxation. When effective, this can substantially reduce tree growth and overall solve time.

The trade-off arises from LP cost. Disjunctive cuts are typically dense, and their addition increases the per-iteration cost of simplex pivots. As demonstrated in chapters 2 and 3, implicit warm starts are most beneficial when the baseline tree is sufficiently large that reductions in node processing offset the increased LP iteration cost. In smaller or moderately sized solves, this density cost can dominate, leading to limited improvement or even degradation.

Taken together, the literature indicates that explicit warm starts are most reliable when structural similarity between instances is high and tree depth is modest, whereas implicit warm starts become more attractive as tree size grows and cut-based strengthening can amortize its LP overhead. The computational study in this chapter is designed precisely to investigate these regime-dependent effects in a controlled comparison.

4.2 Computational Results

4.2.1 Set-Up

The goal of this chapter is to understand the regimes in which one warm starting approach is preferable to the other. We evaluate implementations of both approaches on MILP instance families from randomly perturbed MIPLIB instances against a baseline in which no warm starting occurs. All runs use standard MILP solvers on a shared server cluster; details below. Bold terms match figure legends and table headers.

Test Set Generation

From **Base** instances $\mathcal{B}$ (presolved MIPLIB 3, 2003, and 2010; solve time ≥ 5 seconds; $\leq$ 5,000 variables and constraints), we build **Test** sets indexed by $(k, u, \theta) \in \mathcal{B} \times \{b, c\} \times \{.5, 2\}$, where **Element** $= u$ and **Degree** $= \theta$. We intentionally avoid perturbing the coefficient matrix since symphony does not currently support warm starting under such circumstances. Each instance ℓ in (k, u, θ) is produced by Algorithm 6, which rotates u^k by angle θ. For each (k, u, θ) we iterate until we obtain 1 instance, hit 1,000 attempts, or reach 4 hours; hence test-set sizes may vary by base instance. We use these test sets to approximate the diversity of possible MILP sequences while avoiding decomposition-specific or real-time–specific structure, enabling a general assessment of warm starting methods.

Methods Compared

Let d denote the number of **Terms** in the disjunction used for warm starting. For each (k, u, θ) and instance ℓ, we warm start with:

- `NWS`: Node warm starts via Güzelsoy [40, Section 4.3];
- `CWS`: Cut warm starts via Algorithm 12 with `prune` false and `support` true;
- `Default`: default solver (no warm start).

The `NWS` and `CWS` approaches share the same disjunction $\{\mathcal{X}^t\}_{t \in T}$ and LP solution bases $\{B^t\}_{t \in T}$, from which $\{v^t\}_{t \in T}$ is computed for reuse by `CWS`. To emphasize that both methods rely on the same underlying disjunction, we denote the copy used by `NWS` as `ND` and the copy used by `CWS` as `CD`.

Software and Hardware Dependencies

All experiments use the COIN-OR MILP solver Symphony [57] for branch and cut with warm starting enabled depending on set up. We leverage the VPC and PDC C++ libraries [3, 2] as the implementations for generating Farkas certificates and PDIs. VPC is configured with CBC 2.10 [1] to extract disjunctions from partial branch and bound trees. Runs execute

Table 4.1: Integrality gap closed at the root node by node and cut warm starts.

Element	Degree	Terms	ND	CD	CWS	Test Instances
objective	0.5	4	0.084	0.084	0.047	87
		64	0.181	0.181	0.106	64
	2.0	4	0.078	0.078	0.041	87
		64	0.156	0.156	0.083	65
rhs	0.5	4	0.063	0.063	0.027	75
		64	0.120	0.120	0.060	62
	2.0	4	0.075	0.075	0.021	58
		64	0.136	0.136	0.047	50

on a shared cluster (16 AMD Opteron 6128); each job uses 1 CPU and 2 GB RAM. Time limits are 1,800 s for each solve.

4.2.2 Strength of Root Relaxations

The overarching goal of our experiments is to identify regimes in which one warm-starting method should be preferred over the other. To enable a fair comparison, neither approach is given an inherent advantage at initialization. In this subsection, we verify that the effectiveness of each warm start exhibits the expected behavior with respect to both the number of disjunctive terms and the degree of perturbation in our experimental setting.

Table 4.1 reports the percentage of integrality gap (as defined in appendix A) closed at the root node by the disjunctions and disjunctive cuts used in our warm-starting methods across the test instances. The column `CD` reports the gap closed by the dual bound of the disjunction used by `CWS`, while `ND` reports the gap closed by the disjunction used by `NWS`. The column `CWS` gives the additional gap closed by the disjunctive cuts generated by `CWS` upon their inclusion in the root LP relaxation. We do not report separate values for `NWS`, as it explicitly reuses the same disjunction to initialize the node pool and therefore achieves the same root-node gap closure as `ND`.

The patterns observed in Table 4.1 confirm that both warm-starting methods begin from the same effective starting point and exhibit the expected trends as the size of the disjunction and the degree of perturbation vary. In particular, `CD` and `ND` take identical values throughout, reflecting the fact that the same disjunction is used to initialize both

Table 4.2: Instances with the largest relative termination-time improvements under node warm starting.

Base Instance	Element	Degree	Terms	`Default`	`CWS`	`NWS`
bell5	objective	0.5	64	38.55	3.302	0.813
bell5	rhs	2.0	64	21.13	35.04	1.439
stein27	objective	0.5	64	60.03	59.07	5.453
bell5	rhs	2.0	4	20.99	27.39	1.907
stein27	objective	2.0	64	45.52	46.18	4.676
bell5	objective	0.5	4	38.41	9.488	6.043
neos-1396125	rhs	2.0	4	1798	1800	291.8
p0282	rhs	0.5	4	1775	1047	376.1
dsbmip	rhs	0.5	4	16.47	7.708	3.707
rentacar	objective	2.0	4	21.74	27.40	5.738

methods. This consistency provides further evidence that the experimental setup does not bias the comparison.

As expected, `CWS` recovers only a portion of the strength reflected in `CD`, with the fraction recovered decreasing as the degree of perturbation increases, mirroring the behavior observed in the previous two chapters. More generally, the strength of both the shared disjunction and the cuts generated from it increases with the number of disjunctive terms and decreases with the degree of perturbation. The sole exception occurs for RHS perturbations, where disjunction strength increases with perturbation. This behavior is not unexpected, given differences in the underlying base instances used to generate the test sets for each combination of perturbation degree and number of terms.

Since the root-node behavior aligns with theoretical expectations, we can rule out any inherent starting advantage or major implementation issues. This provides a clean baseline and supports the validity of the comparative results presented later in this chapter.

4.2.3 High Performing Subset of Experiments

We begin by examining the test instances for which each warm-starting method achieves its largest relative improvements in termination time over the default solve. The goal is to identify preliminary patterns that can inform when one warm-starting method should be preferred over the other and to motivate hypotheses tested more broadly in the remainder of this subsection.

Table 4.3: Instances with the largest relative termination-time improvements under cut warm starting.

Base Instance	Element	Degree	Terms	`Default`	`CWS`	`NWS`
pigeon-19	rhs	0.5	64	545.5	16.38	340.8
qnet1	rhs	0.5	4	1794	89.07	1796
qnet1	objective	2.0	4	1794	101.5	1794
qnet1	rhs	2.0	4	1796	107.9	1795
qnet1	objective	0.5	4	1796	137.2	1795
bell5	objective	0.5	64	38.55	3.302	0.813
mod010	rhs	2.0	64	160.5	14.99	77.73
misc06	rhs	2.0	64	55.35	9.710	54.33
mod010	rhs	2.0	4	160.1	39.26	62.67
bell5	objective	0.5	4	38.41	9.488	6.043

Tables 4.2 and 4.3 report the instances exhibiting the largest relative termination-time improvements for node and cut warm starts, respectively. Both tables share the same structure. The first four columns define the test instance, consisting of the base instance being perturbed, the perturbed element, the degree of perturbation, and the number of terms in the disjunction used for warm starting. The remaining columns report the termination time, in seconds, under the default solve (`Default`), cut warm starting (`CWS`), and node warm starting (`NWS`).

The instances highlighted in Tables 4.2 and 4.3 are consistent with patterns observed in prior work on both warm-starting approaches. Güzelsoy [40] note that node warm starts can become unwieldy when terminal disjunctions change substantially, a situation that frequently arises in instances with longer default run times. Inversely, as shown in chapter 3, cut warm starting often benefits longer-running instances, where reductions in the number of processed nodes can offset the increased cost of LP solves caused by the density of cuts added to relaxations.

These trends are reflected in the tables. The strongest relative improvements for `NWS` tend to occur on instances with comparatively short default solve times, while the strongest improvements for `CWS` are concentrated among instances with longer default run times. In line with earlier results, `NWS` generally underperforms `CWS` on longer runs, while the opposite behavior is more common on shorter runs. Importantly, strong performance for either method is not confined to a specific base instance, perturbed element, degree of

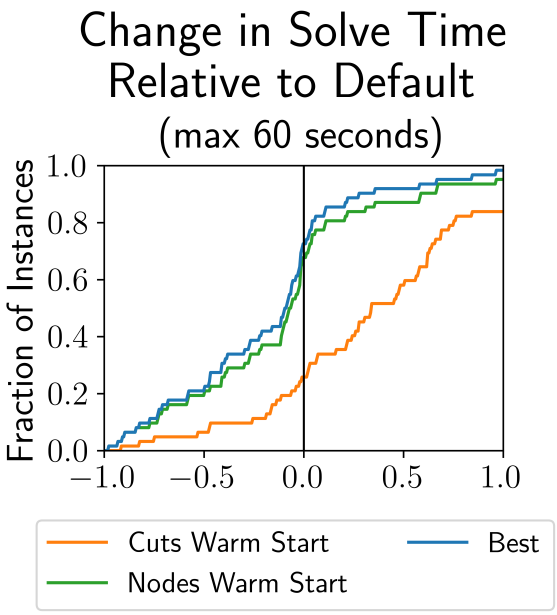


Figure 4.3: CDF of relative improvements in termination time for test instances solved via `Default` *in under 60 seconds.*

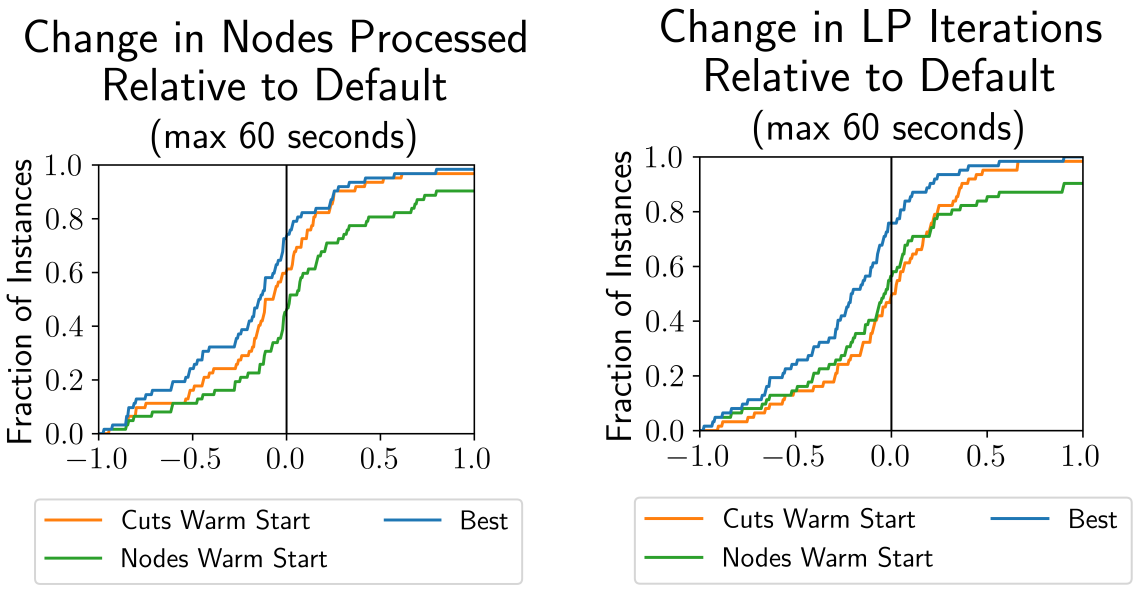
perturbation, or disjunction size, as evidenced by the diversity of configurations represented in both tables.

4.2.4 Branch and Cut Performance

These observations raise the question of whether the patterns suggested by the extreme cases persist across the broader test set. To investigate this, we partition the analysis and plots in this subsection into two groups: the 62 test instances solved by `Default` in at most 60 seconds and the 44 test instances requiring at least 600 seconds under `Default`.

Throughout this subsection, we use performance profiles to show the relative performance of `CWS` and `NWS` compared to `Default`. In each plot, a value of -0.5 denotes a 50% improvement and +0.5 a 50% degradation on instances of the corresponding group. "Best" denotes the virtual best relative improvement between the two warm start configurations.

Max 60 Second `Default` Solve Time Figure 4.3 reports the distribution of relative termination-time improvements for test instances that are solved by `Default` in at most 60 seconds. For this subset, both warm-starting methods behave in line with the patterns suggested by the extreme cases discussed in Section 4.2.3. In particular, `NWS` consistently



(a) Nodes *(b) Iterations*

Figure 4.4: CDF of relative improvements in nodes processed and LP iterations for test instances solved via `Default` *in under 60 seconds.*

dominates `CWS` across the entire distribution. The absence of a gap between `NWS` and the best-performing method indicates that even when `CWS` is beneficial, `NWS` is typically at least as effective and often strictly better.

This behavior is consistent with prior observations. As shown by Güzelsoy [40], node warm starting is most effective when successive branch and bound trees differ only modestly, a condition commonly satisfied by short-running instances with relatively small perturbations. In such cases, branch and cut tends to generate similar disjunctions across solves, allowing `NWS` to reuse high-quality bases and node information with minimal disruption. In contrast, results in chapter 3 and the analysis of Balas and Kazachkov [12] highlight that cut warm starting can slow LP reoptimization due to the increased density of the LP relaxation, which may outweigh any gains from tree reduction in short solves.

This interpretation is reinforced by the node and LP-iteration results shown in Figure 4.4. For `NWS`, the number of nodes processed relative to `Default` is relatively standard normally distributed (Figure 4.4a), indicating that the structure of the branch and bound tree changes little in most cases. At the same time, we observe a modest but consistent reduction in LP iterations (Figure 4.4b), reflecting the benefit of reusing high-quality bases

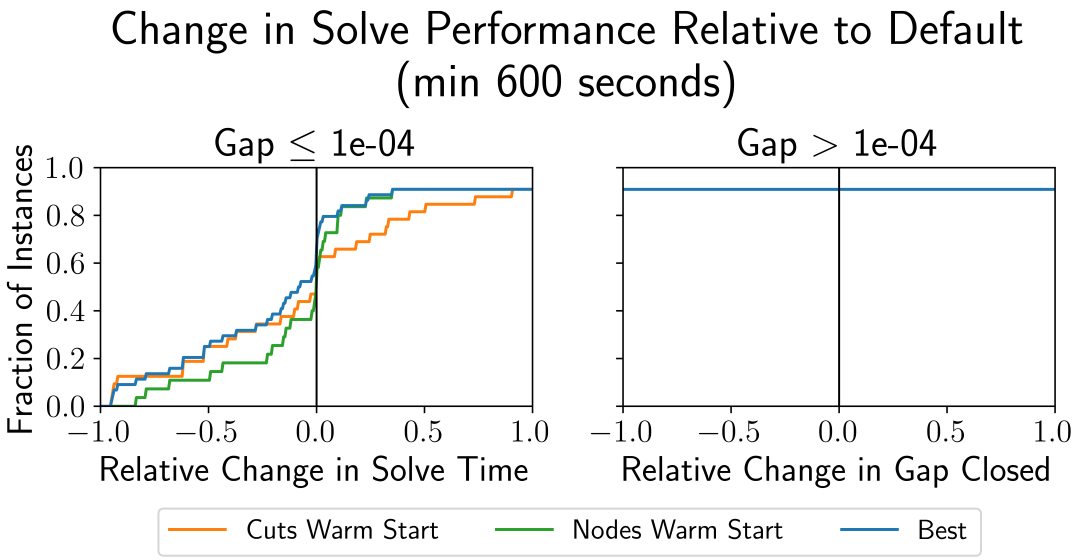
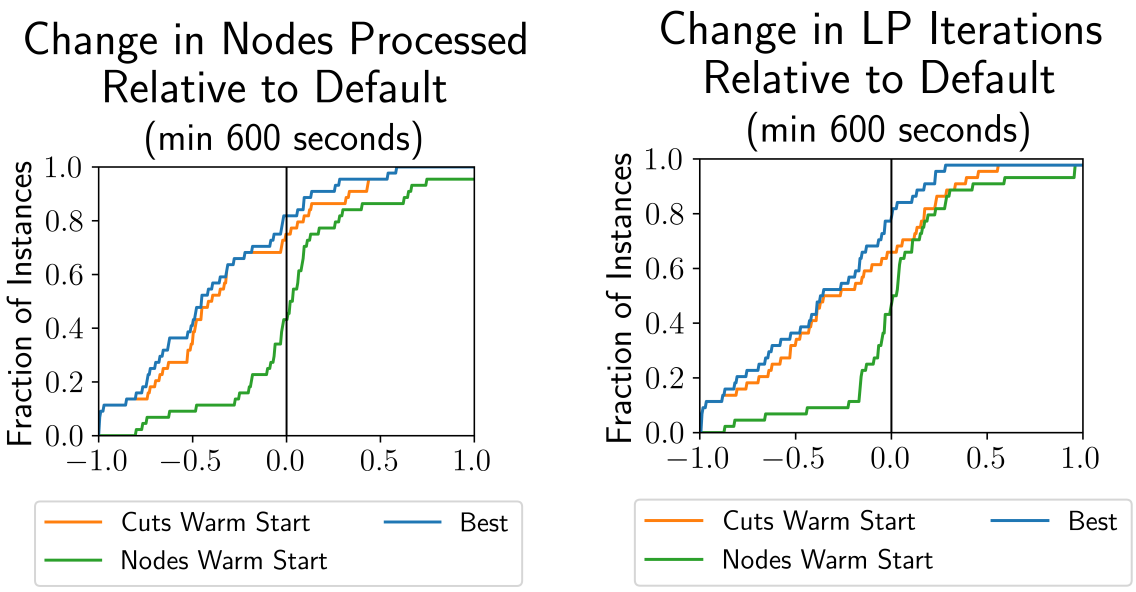


Figure 4.5: CDF of relative improvements in termination time and optimality gap closed for test instances solved via `Default` *in at least 600 seconds.*

associated with the terminal disjunction. Taken together, these effects suggest that `NWS` also benefits from lower time per LP iteration, plausibly due to sparser effective bases and cheaper factorizations deeper in the tree.

For `CWS`, the patterns align with the expected trade-off. While the added disjunctive cuts strengthen the relaxation and lead to some reduction in the number of nodes processed, they also enlarge and densify the LP. As a result, improvements in LP iterations are muted relative to improvements in nodes, and the increased per-iteration cost limits overall time savings. Given the short run times of these instances, the subtrees avoided by disjunctive cuts would not have grown large enough for node reductions to offset the additional LP overhead. This explains the relatively weak improvements—and frequent degradations—observed for `CWS` in Figure 4.3.

Min 600 Second `Default` Solve Time Figure 4.5 reports the distributions of relative improvements in termination time (left) and optimality gap closed (right) for test instances that require at least 600 seconds under `Default`. As in the short-run case, the behavior of both warm-starting methods is broadly consistent with the patterns suggested by the extreme cases discussed in Section 4.2.3, although the effects manifest less directly for this subset.



(a) Nodes *(b) Iterations*

Figure 4.6: CDF of relative improvements in nodes processed and LP iterations for test instances solved via `Default` *in at least 600 seconds.*

From the termination-time distribution, we observe that `NWS` and `CWS` exhibit similar mean improvements, but with substantially greater variance for `CWS`. The optimality-gap distribution shows that approximately 90% of instances reach optimality under both warm-starting methods, indicating that the results are not driven by selectively easier long-running instances that warm starting can solve while `Default` cannot.

These outcomes are consistent with intuition. For `NWS`, the size of the reused disjunction is typically small relative to the size and complexity of the terminal disjunction encountered deep in the tree. As a result, whether helpful or not, the node warm start lacks sufficient structural influence to meaningfully affect solves of this length. In contrast, `CWS` continues to exhibit the trade-offs observed in earlier sections. While the increased LP density raises the per-iteration cost, longer runs provide substantially more opportunity for reductions in the size of the search tree to offset this overhead. The coexistence of these opposing effects naturally leads to both larger improvements and more severe degradations, explaining the higher variance observed for `CWS`.

This interpretation is reinforced by the node and LP-iteration results in Figure 4.6. For `NWS`, the relatively small disjunctions again lead to little change in either the number

of nodes processed (Figure 4.6a) or the number of LP iterations required (Figure 4.6b), consistent with its muted impact on overall solve time in this regime.

For `CWS`, the patterns align with expected behavior in large branch and bound trees. Disjunctive cuts prune subtrees that would otherwise grow substantially, resulting in dramatic reductions in the number of nodes processed. At the same time, the added cuts enlarge the LP relaxation, leading to more modest gains in LP iterations. Together, these effects suggest slower LP pivots offset by significant tree reduction, consistent with the roughly symmetric distribution of termination-time improvements observed for `CWS` in Figure 4.5. This trade-off is unsurprising given the density of the disjunctive cuts and the depth of the trees in this regime.

4.3 Conclusion

The primary objective of this chapter was to identify regimes in which node warm starting and cut warm starting should be preferred over one another. To enable a controlled comparison, we constructed a single disjunction and reused it across both approaches, ensuring that neither method benefited from an inherent initialization advantage. The distinction between the two methods lies in how that disjunction is reused: node warm starting explicitly instantiates the disjunction through the leaves of a partial branch and bound tree, initializing the node pool, whereas cut warm starting implicitly encodes the disjunction via a Farkas certificate that parametrically generates valid disjunctive inequalities for the cut pool.

Under small perturbations and fixed disjunction sizes, clear regime-dependent behavior emerges. For instances solved quickly under `Default` (less than one minute), node warm starting consistently provides stronger improvements. In this setting, the initial partial tree represents a substantial portion of the eventual branch and bound tree, allowing the reused node information and inherited bases to accelerate reoptimization with minimal structural disruption. In contrast, for instances requiring longer solves (greater than ten minutes), cut warm starting exhibits greater potential for improvement. When the terminal tree is large, the subtrees pruned by strong disjunctive cuts can generate savings that outweigh

the additional LP overhead introduced by denser relaxations. These findings align with the structural trade-offs identified in earlier chapters: node warm starts capitalize on structural similarity between solves, while cut warm starts trade per-iteration LP cost for reductions in search-tree size.

Several implementation constraints limit the scope of our experiments. First, enabling node warm starting in SYMPHONY required disabling reduced-cost fixing and cut generation, which restricts the size and difficulty of instances that can be practically solved. Second, SYMPHONY requires a fixed constraint matrix to enable warm starting, limiting the types of perturbations that can be tested. Third, because global cut generation was disabled, cut warm starts remain permanently embedded in the LP relaxation, whereas in practice such cuts may be removed when they become slack to improve LP performance. Finally, while we focused on moderate perturbations to isolate regime effects, future work should examine whether one approach degrades more gracefully under larger structural changes.

Taken together, these results demonstrate that warm-start effectiveness is fundamentally regime-dependent. The choice between node and cut warm starting should be guided not only by the perturbation structure but also by the anticipated size and growth of the branch and bound tree. This motivates the development of adaptive strategies that select or hybridize warm-start mechanisms based on instance characteristics, an avenue we highlight as a promising direction for future research and solver design.

Chapter 5

Conclusion

This dissertation studies how to reuse structural information across related MILPs within modern branch and cut algorithms. In particular, it focuses on the reuse of disjunctions—objects that capture structural separation information at the heart of branch and bound—and investigates how such information can be transferred across solves in both explicit and implicit forms.

Disjunction reuse arises in two primary ways. It can be instantiated explicitly through node pool warm starting, where a previously explored partial branch and bound tree is reused to initialize a subsequent solve, as in Güzelsoy [40]. Alternatively, it can be reused implicitly through the cut pool by encoding a disjunction in a Farkas certificate that parametrically generates valid disjunctive inequalities. This dissertation develops a unified framework for understanding these two mechanisms, filling gaps in the theoretical and computational understanding of each before comparing them directly.

The first paper introduces PDIs, formalizing how families of valid disjunctive inequalities can be generated from a single structural object. It establishes validity and strength guarantees and defines cut warm starting within this framework, including an analysis of its computational complexity. The second paper develops methods for strengthening PDIs, addressing both theoretical and practical aspects of improving their impact within branch and cut. The third paper then compares cut warm starting and node warm starting under a controlled experimental framework, identifying the regimes in which each method is favored.

Across these contributions, a consistent theme emerges: warm-start effectiveness is fundamentally regime-dependent. The relative benefit of node and cut warm starting is governed by the interaction between search-tree growth and LP relaxation density. Node warm starts are most effective when structural similarity between solves allows previously explored tree regions to represent a substantial portion of the terminal tree. Cut warm starts, in contrast, trade increased LP density for reductions in search-tree size, and therefore become more advantageous when terminal trees are large enough that subtree pruning offsets slower LP reoptimization. This perspective reframes warm starting not as a monolithic enhancement, but as a structural design choice conditioned on instance evolution and solver dynamics.

More broadly, this dissertation contributes to the understanding of reoptimization in mixed integer optimization. Many modern applications—including decomposition algorithms, online optimization, and parametric modeling—require solving sequences of closely related MILPs. In such settings, discarding previously discovered structural information is computationally wasteful. The methods and analyses developed here demonstrate that meaningful reuse is possible, but must be guided by an understanding of the structural regime in which the solver operates.

Several avenues for future work follow naturally from these findings. First, the idea of supporting certificate recovery for disjunctions could be extended to other classes of cuts, particularly computationally expensive families such as interdiction-type cuts or other problem-specific inequalities whose generation cost limits their frequent use. Second, the empirical performance of strengthened PDIs suggests that density remains a limiting factor. Developing computationally efficient regularization strategies—particularly for dense families such as VPCs—may enable stronger cuts without incurring prohibitive LP overhead. Finally, the regime-dependent nature of warm-start effectiveness motivates adaptive and learning-based selection mechanisms. Rather than committing statically to node or cut reuse, future solvers could dynamically select or hybridize strategies based on observable instance features, tree growth patterns, or online performance metrics.

In summary, this dissertation establishes both a theoretical and computational foundation for disjunction reuse in mixed integer optimization. By clarifying the mechanisms,

trade-offs, and regimes governing warm-start performance, it provides a structured framework for incorporating reuse into next-generation branch and cut algorithms.

Appendix A

Evaluation

Cuts are evaluated in two different ways: their effect on the root node and their effect on branch and bound. At the root node, we measure the strength of the cuts by the percent integrality (root) gap closed by one round of VPCs or their generating disjunction. Let x_I denote an optimal solution to a MILP, let $\bar{x}$ denote an optimal solution to a MILP's LP relaxation, and let x_0 be an optimal solution to the MILP's LP relaxation after a set of cuts have been added or a disjunction applied. We define the quantity

$$\% \text{ integrality gap closed} := 100 \times \frac{c^T x_0 - c^T \bar{x}}{c^T x_I - c^T \bar{x}}$$

Appendix B

Additional Algorithms

Implementation of Root-Down Disjunction Reconstruction. We now describe how the disjunction induced by a partial branch-and-bound tree is reconstructed in practice. As discussed in Section 4.1.2, the primary challenge is that reduced-cost fixings identified during strong branching may only appear along the surviving branch and not as explicit sibling nodes in the tree. To obtain a valid disjunction, these implicit decisions must be materialized as explicit binary splits.

The reconstruction therefore proceeds in a root-down fashion. For each leaf term, we first ensure that branching decisions are ordered by depth, then recursively traverse toward the root to identify strong-branching fixings that were not explicitly paired. Whenever such a fixing is encountered, we create the missing complementary sibling term and insert it into the disjunction. This guarantees that the resulting collection of terms forms a complete binary structure and therefore defines a valid disjunction.

The algorithms below describe this reconstruction process as implemented in the event handler used during the base solve.

We can create a branch and bound tree in CBC by subclassing CBCEventHandler. We will need to add the following attributes:

- `sorted_nodes_`: a subset of $[|\mathcal{X}|]$ tracking which nodes of the Branch-and-Bound tree have had their disjunctive constraints (D^t, D_0^t) ordered by depth in the tree each was created.

- `checked_nodes_`: a subset of $[|\mathcal{X}|]$ tracking which nodes of the Branch-and-Bound tree have been checked for implicitedly pruned terms between them and their parents.

We define $\Pi : T \to \{0, 1\}$ such that

$$\Pi[t] = \begin{cases} 0 & \text{if } \mathcal{X}^t \text{ pruned in Branch-and-Bound tree for MILP-}k \\ 1 & \text{otherwise} \end{cases}$$

We define $F : T \to \{0, 1\}$ such that

$$F[t] = \begin{cases} 0 & \text{if } \mathcal{Q}^{kt} = \emptyset \\ 1 & \text{otherwise} \end{cases}$$

Algorithm 13 `removeContinuousVariableTightenedBounds`

Require: $\mathcal{X}$

1: **for** $t \leftarrow 1 : |\mathcal{X}|$ **do** ▷ restrict term to branching on integer variables
2: $\quad D^t = \left[[D^t_{j\cdot}] : j \in q_t, D^t_{j\cdot} = \pm e_i \text{ for some } i \in \mathcal{I}\right]$
3: $\quad D^t_0 = \left[[D^t_{0j}] : j \in q_t, D^t_{j\cdot} = \pm e_i \text{ for some } i \in \mathcal{I}\right]$
4: **end for**

Algorithm 13 removes bound tightenings on continuous variables so that the induced disjunction depends only on integer branching decisions. This ensures that each term corresponds to a partition of the integer feasible region rather than an artifact of LP presolve behavior.

Algorithm 14 `sortBranchingDecision`

Require: $\mathcal{X}, P, t,$ `sorted_nodes_`

1: **if** $t \notin$ `sorted_nodes_` **then**
2: $\quad$ **if** $t \in P$ **then**
3: $\quad\quad$ `sortBranchingDecision`$(\mathcal{X}, P, P[t],$ `sorted_nodes_`$)$
4: $\quad$ **end if**
5: $\quad D^t = \begin{bmatrix} [D^t_{j\cdot}] : (D^t_{j\cdot}, D^t_{0j}) \in \mathcal{X}^{P[t]} \\ [D^t_{j\cdot}] : (D^t_{j\cdot}, D^t_{0j}) \notin \mathcal{X}^{P[t]} \end{bmatrix}$ ▷ put parent's branching decisions first
6: $\quad D^t_0 = \begin{bmatrix} [D^t_{0j}] : (D^t_{j\cdot}, D^t_{0j}) \in \mathcal{X}^{P[t]} \\ [D^t_{0j}] : (D^t_{j\cdot}, D^t_{0j}) \notin \mathcal{X}^{P[t]} \end{bmatrix}$
7: $\quad$ `sorted_nodes_` $\leftarrow$ `sorted_nodes_` $\cup \{t\}$
8: **end if**

Algorithm 14 ensures that branching constraints within each term are ordered consistently by tree depth. This ordering is required so that implicit strong-branching fixings can be identified by comparing parent and child term representations.

Algorithm 15 `createStrongBranchingTerms`

Require: $\mathcal{X}, P, T, t$

$u = |\mathcal{X}| + 1$ ▷ next term to create
$p = P[t]$ ▷ next term's parent node
for $m \leftarrow q_{P[t]} + 1 : q_t - 1$ **do**
    $D^u = \begin{bmatrix} [D^t_j.] : j \in [m-1] \\ [-D^t_m.] \end{bmatrix}$ ▷ create the implicitly pruned term
    $D^u_0 = \begin{bmatrix} [D^t_{0j}] : j \in [m-1] \\ [-D^t_{0m} - (1 \text{ if } D^t_{0m} \leq 0 \text{ else } -1)] \end{bmatrix}$
    $T \leftarrow T \cup \{u\}$ ▷ add to active disjunction
    $\mathcal{X} \leftarrow \mathcal{X} \cup \{\{x \in \mathbb{R}^n : D^u x \geq D^u_0\}\}$ ▷ record in lineage
    $P[u] \leftarrow p$
    $u \leftarrow u + 1$
    $D^u = \left[[D^t_j.] : j \in [m]\right]$ ▷ create the ancestor of t
    $D^u_0 = \left[[D^t_{0j}] : j \in [m]\right]$
    $\mathcal{X} \leftarrow \mathcal{X} \cup \{\{x \in \mathbb{R}^n : D^u x \geq D^u_0\}\}$ ▷ record in lineage
    $P[u] \leftarrow p$
    $p \leftarrow u$ ▷ increment indices
    $u \leftarrow u + 1$
end for
$P[t] \leftarrow p$ ▷ update t's parent index

Algorithm 16 `recursivelyCreateStrongBranchingTerms`

Require: $\mathcal{X}, P, T, t,$ `checked_nodes_`

if $t \notin$ `checked_nodes_` **then**
    if $t \in P$ **then** ▷ recurse on parent if it exists
        `recursivelyCreateStrongBranchingTerms`$(\mathcal{X}, P, P[t],$ `checked_nodes_`$)$
    end if
    `createStrongBranchingTerms`$(\mathcal{X}, P, T, t)$ ▷ create implicit terms
    `checked_nodes_` $\leftarrow$ `checked_nodes_` $\cup \{t\}$ ▷ mark this node as checked
end if

Algorithms 15 and 16 implement the root-down materialization of implicit strong-branching decisions. Starting from a leaf, the procedure recursively processes its ancestors. Whenever a bound tightening appears in a child but not in its parent, the missing complementary branch is explicitly created. Repeating this process along the path to the root produces a sequence of binary splits that reconstructs the full implicit branching structure.

Algorithm 17 `isFullBinaryTree`

Require: $\mathcal{X}, P$

$q_{\max} \leftarrow \max_{t \in [|\mathcal{X}|]} \{q_t\}$ ▷ get max depth
for $m \leftarrow q_{\max} : 1$ **do** ▷ iterate from bottom to top of tree
  $T_{\text{depth}} \leftarrow \{t : q_t == m, t \in [|\mathcal{X}|]\}$ ▷ get terms at this depth
  $T_{\text{paired}} \leftarrow \emptyset$ ▷ track paired terms
  for all $t \in T_{\text{depth}}$ **do** ▷ iterate over each term at this depth
    if $t \in T_{\text{paired}}$ **then**
      continue
    end if
    for all $u \in T_{\text{depth}}$ **do** ▷ iterate over all pairs at this depth
      if $t \neq u$ and all$(\{(D^t_{j\cdot}, D^t_{0\,j}) == (D^u_{j\cdot}, D^u_{0\,j}) : j \in [m-1]\})$ **then** ▷ all but last branching decision match if t and u are siblings
        if $D^t_{q\cdot} == -D^u_{j\cdot}$ and $D^t_{0m} - D^u_{0\,m} == (1 \text{ if } D^u_{j\cdot} \leq 0 \text{ else } -1)$ **then** ▷ t and u are siblings
          $T_{\text{paired}} \leftarrow T_{\text{paired}} \cup \{t, u\}$ ▷ record t and u as siblings
          break
        end if
      end if
    end for
    assert $t \in T_{\text{paired}}$ ▷ raise error if no sibling found
  end for
end for

Algorithm 17 verifies that the reconstructed structure is a valid binary disjunction. At each depth level, sibling terms must share identical branching decisions up to the final constraint and differ only in the last bound. Failure to find such pairings indicates that some implicit branch has not been materialized, in which case the disjunction would be invalid.

Algorithm 18 orchestrates the full reconstruction procedure. It removes non-integer tightenings, orders branching decisions, materializes implicit strong-branching splits, and records term feasibility and pruning status. It also stores optimal bases when available and verifies that the resulting structure forms a full binary tree.

The outputs $(\bar{x}, \{B^t\}_{t \in T})$ capture precisely the extra information required for explicit disjunctive warm starting: term optimal values and stored bases. Terms discovered while unfixing strong branching are added to the disjunction in place.

Collectively, these procedures transform the partial branch-and-bound tree into a valid disjunction whose terms partition the integer feasible region explored during the base solve.

Algorithm 18 `saveInformationWithPrunes`

Require: MILP-k, $\mathcal{X}, P, T$

`sorted_nodes_` $\leftarrow \emptyset$ and `checked_nodes_` $\leftarrow \emptyset$ ▷ initialize sets
$\Pi \leftarrow \emptyset$ and $F \leftarrow \emptyset$ ▷ initialize maps
$T_{\text{incomplete}} \leftarrow T$ ▷ create immutable copy of current T
`removeContinuousVariableTightenedBounds`$(\mathcal{X})$
for all $t \in T_{\text{incomplete}}$ **do** ▷ create the missing terms
    `sortBranchingDecision`$(\mathcal{X}, P, t,$ `sorted_nodes_`$)$
    `recursivelyCreateStrongBranchingTerms`$(\mathcal{X}, P, T, t,$ `checked_nodes_`$)$
end for
for all $t \in T$ **do**
    Let $\mathcal{B}^{kt} \in \mathbb{N}^n$ and $\mathcal{N}^{kt} \in \mathbb{N}^n$ ▷ declare solution nonbasis and basis vectors
    $\bar{x}^{kt} \leftarrow$ arg LP-kt ▷ record the term's optimal solution if it exists
    if $\bar{x}^{kt} \neq$ NULL **then** ▷ record optimal solution basis if it exists
        $B^t \leftarrow B : \bar{x}^{kt} = (A^{kt}_{B^t\cdot})^{-1} b^{kt}_{B^t}$
    else
        $B^t \leftarrow NULL$
    end if
end for
`isFullBinaryTree`$(\mathcal{X}, P)$
return $\bar{x}, \{B^t\}_{t \in T}$

By explicitly materializing implicit strong-branching fixings and verifying the full binary structure, the resulting disjunction satisfies the validity requirements described in Section 4.1.2. The stored term bases and feasibility metadata then enable efficient evaluation of the induced dual and primal bounding functions when the disjunction is reused to warm start a related instance.

Appendix C

Additional Tables and Figures

Table C.1: Winning ratios of test instances for `Param Disj, Param Cuts` *(PDI) and* `Default` *by solve time categories. Wins are defined by one disjunctive cut generator solving a test instance, excluding VPC generation, to optimality in 10% less time than the other disjunctive cut generator. Short includes solves less than 60 seconds, Medium from 60 up to 600 seconds, and Long from 600 to 3600 seconds.*

Degree	Terms	Short		Medium		Long	
		PDI	Default	PDI	Default	PDI	Default
0.5	4	38.25	37.54	37.36	39.19	39.57	35.97
	16	36.62	37.09	45.06	38.63	37.17	38.94
	64	38.41	37.09	39.39	35.76	33.71	44.94
2.0	4	36.41	39.32	39.23	32.31	38.46	29.67
	16	34.57	42.59	36.19	31.90	34.52	29.76
	64	36.21	36.21	32.70	37.11	32.20	23.73

Table C.2: Counts of base and test instances by solve time categories. Short includes solves of `Default` *less than 60 seconds, Medium from 60 up to 600 seconds, and Long from 600 to 3600 seconds.*

Degree	Terms	Short		Medium		Long	
		Base	Test	Base	Test	Base	Test
0.5	4	63	285	64	273	35	139
	16	61	213	55	233	29	113
	64	48	151	41	165	25	89
2.0	4	64	206	60	260	28	91
	16	57	162	52	210	24	84
	64	45	116	41	159	19	59

Table C.3: Winning ratios of test instances for `Param Disj, Param Cuts` *(PDI) and* `Default` *by type of perturbation. Wins are defined by one disjunctive cut generator solving a test instance, excluding VPC generation, to optimality in 10% less time than the other disjunctive cut generator.*

Degree	Terms	Matrix		Objective		RHS	
		PDI	Default	PDI	Default	PDI	Default
0.5	4	41.45	35.23	36.76	39.95	37.50	34.38
	16	38.46	36.36	39.76	39.45	44.94	35.96
	64	40.91	38.18	35.48	38.25	39.74	38.46
2.0	4	31.93	34.45	38.74	35.71	44.59	28.38
	16	34.41	32.26	36.51	35.53	30.51	38.98
	64	42.86	25.40	33.48	36.61	23.40	36.17

Table C.4: Counts of base and test instances by perturbation.

Degree	Terms	Matrix		Objective		RHS	
		Base	Test	Base	Test	Base	Test
0.5	4	62	193	86	408	38	96
	16	53	143	78	327	38	89
	64	44	110	64	217	32	78
2.0	4	44	119	88	364	32	74
	16	37	93	79	304	26	59
	64	29	63	63	224	24	47

Table C.5: Linear Regression Model Metadata

Metric	**Value**
Mean Absolute Error (MAE)	0.8929
Mean Squared Error (MSE)	13.1607
R^2 Score	0.0504
Intercept	0.4831

Table C.6: Linear Regression Coefficients (Sorted by Absolute Value)

Feature	**Coefficient**
Root Optimality Gap Improvement	4.2107
indicator	2.1385
numerics	-1.8152
equation_knapsack	1.7023
integer_knapsack	1.4748
bound	-1.4389
mixed_binary	-1.2352
set_packing	-1.1955
cardinality	0.7873
binary	0.6683
objective	0.5729
decomposition	-0.5401
general_linear	-0.5371
matrix	0.5106
binpacking	0.4801
density	-0.4759
aggregations	0.4171
precedence	-0.4094
infeasibleTermsRatio Farkas	0.4019
knapsack	-0.3744
rhs	0.3554
set_covering	-0.3137
variable_bound	0.2485
hard	0.1539
easy	-0.1539
benchmark	-0.0871
benchmark_suitable	-0.0871
degree	-0.0424
set_partitioning	0.0402
invariant_knapsack	-0.0253
actualTerms Farkas	-0.0110
terms	0.0097
numCuts Farkas	0.0028
integers	-0.0001
variables	-0.0001
continuous	0.0000
binaries	-0.0000
constraints	-0.0000
nonz.	0.0000
feasibility	0.0000
no_solution	-0.0000
infeasible	-0.0000
open	-0.0000
infeasibleToFeasibleTermsRatio Farkas	0.0000
zeroInfeasibleToFeasibleTerms Farkas	0.0000
feasibleToInfeasibleTermsRatio Farkas	0.0000

Bibliography

[1] COIN-OR Branch and Cut. https://github.com/coin-or/Cbc, 2024.

[2] Parametric Disjunctive Cuts. https://github.com/spkelle2/pdc, 2026.

[3] V-polyhedral Disjunctive Cuts. https://github.com/spkelle2/vpc, 2026.

[4] Tobias Achterberg. *Constraint Integer Programming.* Phd thesis, Technische Universität Berlin, 2007. URL https://depositonce.tu-berlin.de/items/9f46a10e-2f7b-4dea-8e27-9cae07de5258.

[5] Tobias Achterberg. SCIP: solving constraint integer programs. *Math. Program. Comput.*, 1(1):1–41, 2009. ISSN 1867-2949,1867-2957. URL https://doi.org/10.1007/s12532-008-0001-1.

[6] Tobias Achterberg, Thorsten Koch, and Alexander Martin. Branching rules revisited. *Oper. Res. Lett.*, 33(1):42–54, 2005. ISSN 0167-6377,1872-7468. URL https://doi.org/10.1016/j.orl.2004.04.002.

[7] Alper Atamtürk, George L. Nemhauser, and Martin W. P. Savelsbergh. Conflict graphs in solving integer programming problems. *Eur. J. Oper. Res.*, 121(1):40–55, 2000. ISSN 0377-2217,1872-6860. URL https://doi.org/10.1016/S0377-2217(99)00015-6.

[8] E. Balas, S. Ceria, G. Cornuéjols, and N. Natraj. Gomory cuts revisited. *Oper. Res. Lett.*, 19(1):1–9, 1996. ISSN 0167-6377,1872-7468. URL https://doi.org/10.1016/0167-6377(96)00007-7.

[9] Egon Balas. Intersection cuts—a new type of cutting planes for integer programming.

Oper. Res., 19:19–39, 1971. ISSN 0030-364X,1526-5463. URL https://doi.org/10.1287/opre.19.1.19.

[10] Egon Balas. Facets of the knapsack polytope. *Math. Program.*, 8:146–164, 1975. ISSN 0025-5610,1436-4646. URL https://doi.org/10.1007/BF01580440.

[11] Egon Balas. Disjunctive programming: Properties of the convex hull of feasible points. *Discrete Applied Mathematics*, 89(1):3–44, 1998. ISSN 0166-218X. doi: https://doi.org/10.1016/S0166-218X(98)00136-X. URL https://www.sciencedirect.com/science/article/pii/S0166218X9800136X.

[12] Egon Balas and Aleksandr M. Kazachkov. $\mathcal{V}$-polyhedral disjunctive cuts, 2025. URL https://doi.org/10.1007/s10107-025-02227-y.

[13] Egon Balas and Michael Perregaard. A precise correspondence between lift-and-project cuts, simple disjunctive cuts, and mixed integer Gomory cuts for 0-1 programming. *Math. Program.*, 94(2-3, Ser. B):221–245, 2003. URL http://dx.doi.org/10.1007/s10107-002-0317-y. The Aussois 2000 Workshop in Combinatorial Optimization.

[14] Egon Balas, Sebastián Ceria, and Gérard Cornuéjols. A lift-and-project cutting plane algorithm for mixed 0-1 programs. *Math. Program.*, 58(3):295–324, 1993. ISSN 0025-5610,1436-4646. URL https://doi.org/10.1007/BF01581273.

[15] Egon Balas, Sebastián Ceria, and Gérard Cornuéjols. Mixed 0-1 programming by lift-and-project in a branch-and-cut framework. *Manag. Sci.*, 42(9):1229–1246, 1996. ISSN 00251909, 15265501. URL http://www.jstor.org/stable/2634434.

[16] Cynthia Barnhart, Ellis L. Johnson, George L. Nemhauser, Martin W. P. Savelsbergh, and Pamela H. Vance. Branch-and-price: column generation for solving huge integer programs. *Oper. Res.*, 46(3):316–329, 1998. ISSN 0030-364X,1526-5463. URL https://doi.org/10.1287/opre.46.3.316.

[17] Amitabh Basu, Karen Martin, Christopher T. Ryan, and Guanyi Wang. Mixed-integer linear representability, disjunctions, and variable elimination. *Mathematical Programming*, 158(1-2):343–373, 2016.

[18] John E. Beasley. *Lagrangian relaxation*, page 243–303. John Wiley & Sons, Inc., USA, 1993. ISBN 0470220791.

[19] Berkay Becu, Santanu S. Dey, Feng Qiu, and Alinson S. Xavier. Approximating the Gomory mixed-integer cut closure using historical data, 2024. URL https://arxiv.org/abs/2411.15090.

[20] Robert Bixby and Edward Rothberg. Progress in computational mixed integer programming—a look back from the other side of the tipping point. *Ann. Oper. Res.*, 149:37–41, 2007. URL https://doi.org/10.1007/s10479-006-0091-y. History of integer programming: distinguished personal notes and reminiscences.

[21] Pierre Bonami. On optimizing over lift-and-project closures. *Math. Program. Comput.*, 4(2):151–179, 2012. URL http://dx.doi.org/10.1007/s12532-012-0037-0.

[22] Binyuan Chen, Simge Küçükyavuz, and Suvrajeet Sen. Finite disjunctive programming characterizations for general mixed-integer linear programs. *Oper. Res.*, 59(1):202–210, 2011. URL https://doi.org/10.1287/opre.1100.0882.

[23] Binyuan Chen, Simge Küçükyavuz, and Suvrajeet Sen. A computational study of the cutting plane tree algorithm for general mixed-integer linear programs. *Oper. Res. Lett.*, 40(1):15–19, 2012. URL https://doi.org/10.1016/j.orl.2011.10.009.

[24] V. Chvátal. Edmonds polytopes and a hierarchy of combinatorial problems. *Discrete Math.*, 4:305–337, 1973. ISSN 0012-365X,1872-681X. URL https://doi.org/10.1016/0012-365X(73)90167-2.

[25] Didier Chételat and Andrea Lodi. Continuous cutting plane algorithms in integer programming. *Oper. Res. Lett.*, 51(4):439–445, 2023. ISSN 0167-6377. URL https://www.sciencedirect.com/science/article/pii/S0167637723001074.

[26] Michele Conforti, Gérard Cornuéjols, and Giacomo Zambelli. *Integer programming*, volume 271 of *Graduate Texts in Mathematics*. Springer, Cham, 2014. ISBN 978-3-319-11007-3; 978-3-319-11008-0. URL https://doi.org/10.1007/978-3-319-11008-0.

[27] W. Cook, R. Kannan, and A. Schrijver. Chvátal closures for mixed integer programming problems. *Math. Program.*, 47(2):155–174, 1990. ISSN 0025-5610,1436-4646. URL https://doi.org/10.1007/BF01580858.

[28] George B. Dantzig and Philip Wolfe. Decomposition principle for linear programs. *Oper. Res.*, 8(1):101–111, 1960. ISSN 0030364X, 15265463. URL http://www.jstor.org/stable/167547.

[29] Arnaud Deza and Elias B. Khalil. Machine learning for cutting planes in integer programming: A survey. In *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence*, IJCAI-2023, page 6592–6600. International Joint Conferences on Artificial Intelligence Organization, August 2023. URL http://dx.doi.org/10.24963/IJCAI.2023/739.

[30] Arnaud Deza, Elias B. Khalil, Zhenan Fan, Zirui Zhou, and Yong Zhang. Learn2aggregate: Supervised generation of chvátal-gomory cuts using graph neural networks. 2024. URL https://arxiv.org/abs/2409.06559.

[31] Gabriele Dragotto, Stefan Clarke, Jaime Fernández Fisac, and Bartolomeo Stellato. Differentiable cutting-plane layers for mixed-integer linear optimization, 2023. URL https://arxiv.org/abs/2311.03350.

[32] Marco A. Duran and Ignacio E. Grossmann. An outer-approximation algorithm for a class of mixed-integer nonlinear programs. *Math. Program.*, 36(3):307–339, 1986. ISSN 0025-5610,1436-4646. URL https://doi.org/10.1007/BF02592064.

[33] Samira Fallah, Ted K. Ralphs, and Natashia L. Boland. On the relationship between the value function and the efficient frontier of a mixed integer linear optimization problem, 2024. URL https://arxiv.org/abs/2303.00785.

[34] Julius Farkas. Theorie der einfachen Ungleichungen. *J. Reine Angew. Math.*, 124:1–27, 1902. URL https://doi.org/10.1515/crll.1902.124.1.

[35] Matteo Fischetti, Andrea Lodi, and Andrea Tramontani. On the separation of disjunc-

tive cuts. *Math. Program.*, 128(1-2, Ser. A):205–230, 2011. URL http://dx.doi.org/10.1007/s10107-009-0300-y.

[36] Gerald Gamrath, Benjamin Hiller, and Jakob Witzig. Reoptimization techniques for MIP solvers. In Evripidis Bampis, editor, *Experimental Algorithms - 14th International Symposium, SEA 2015, Paris, France, June 29 - July 1, 2015, Proceedings*, volume 9125 of *Lecture Notes in Computer Science*, pages 181–192. Springer, 2015. doi: 10.1007/978-3-319-20086-6_14. URL https://doi.org/10.1007/978-3-319-20086-6_14.

[37] Maxime Gasse, Didier Chételat, Nicola Ferroni, Laurent Charlin, and Andrea Lodi. Exact combinatorial optimization with graph convolutional neural networks. In *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*, pages 15554–15566, 2019. URL https://proceedings.neurips.cc/paper/2019/hash/d14c2267d848abeb81fd590f371d39bd-Abstract.html.

[38] Ralph E. Gomory. An algorithm for integer solutions to linear programs. In *Recent advances in mathematical programming*, pages 269–302. McGraw-Hill Book Co., Inc., New York-San Francisco-Toronto-London, 1963.

[39] Ralph Edward Gomory. *An algorithm for the mixed integer problem.* RAND Corporation, Santa Monica, CA, 1960.

[40] M. Güzelsoy. *Dual Methods in Mixed Integer Linear Programming.* PhD, Lehigh University, 2009. URL http://coral.ie.lehigh.edu/~ted/files/papers/MenalGuzelsoyDissertation09.pdf.

[41] A. Hassanzadeh and T.K. Ralphs. A Generalization of Benders' Algorithm for Two-Stage Stochastic Optimization Problems with Mixed Integer Recourse. Technical Report 14T-005, COR@L Laboratory, Lehigh University, 2014. URL http://coral.ie.lehigh.edu/~ted/files/papers/SMILPGenBenders14.pdf.

[42] Robert G. Jeroslow. Cutting-plane theory: Algebraic methods. *Discrete Math.*, 23

(2):121–150, 1978. ISSN 0012-365X,1872-681X. URL https://doi.org/10.1016/0012-365X(78)90072-3.

[43] Robert G. Jeroslow. Representability in mixed integer programming, i: Characterization results. *Discrete Applied Mathematics*, 17(3):223–243, 1987.

[44] Miroslav Karamanov and Gérard Cornuéjols. Branching on general disjunctions. *Math. Program.*, 128(1-2):403–436, 2011. ISSN 0025-5610,1436-4646. URL https://doi.org/10.1007/s10107-009-0332-3.

[45] Aleksandr M. Kazachkov and Egon Balas. Monoidal strengthening of simple $\mathcal{V}$-polyhedral disjunctive cuts, 3 2025. URL https://doi.org/10.1007/s10107-024-02185-x.

[46] Shannon Kelley, Aleksandr Kazachkov, and Ted Ralphs. Warm starting of mixed integer linear optimization problems via parametric disjunctive cuts. 2025.

[47] Ailsa H. Land and Alison G. Doig. An automatic method for solving discrete programming problems. volume 28, pages 105–132. Springer, Berlin, Heidelberg, 1960. doi: 10.1007/978-3-540-68279-0_5.

[48] J. T. Linderoth and M. W. P. Savelsbergh. A computational study of search strategies for mixed integer programming. volume 11, pages 173–187. 1999. URL https://doi.org/10.1287/ijoc.11.2.173. Combinatorial optimization and network flows.

[49] Quentin Louveaux, Laurent Poirrier, and Domenico Salvagnin. The strength of multi-row models. *Math. Program. Comput.*, 7(2):113–148, 2015. URL http://dx.doi.org/10.1007/s12532-014-0076-9.

[50] Alexander Martin. General mixed integer programming: computational issues for branch-and-cut algorithms. In *Computational combinatorial optimization (Schloß Dagstuhl, 2000)*, volume 2241 of *Lecture Notes in Comput. Sci.*, pages 1–25. Springer, Berlin, 2001. ISBN 3-540-42877-1. URL https://doi.org/10.1007/3-540-45586-8_1.

[51] Robert R. Meyer. On the existence of optimal solutions to integer and mixed-integer programming problems. *Mathematical Programming*, 7(1):223–235, 1974.

[52] Gonzalo Muñoz, Joseph Paat, and Álinson S. Xavier. Compressing branch-and-bound trees. In *Integer programming and combinatorial optimization*, volume 13904 of *Lecture Notes in Comput. Sci.*, pages 348–362. Springer, Cham, 2023. ISBN 978-3-031-32725-4; 978-3-031-32726-1. URL https://doi.org/10.1007/978-3-031-32726-1_25.

[53] Vinod Nair, Sergey Bartunov, Felix Gimeno, Ingrid von Glehn, Pawel Lichocki, Ivan Lobov, Brendan O'Donoghue, Nicolas Sonnerat, Christian Tjandraatmadja, Pengming Wang, Ravichandra Addanki, Tharindi Hapuarachchi, Thomas Keck, James Keeling, Pushmeet Kohli, Ira Ktena, Yujia Li, Oriol Vinyals, and Yori Zwols. Solving mixed integer programs using neural networks. 2021. URL https://arxiv.org/abs/2012.13349.

[54] Krunal Kishor Patel. Progressively strengthening and tuning mip solvers for reoptimization, 2024. URL https://arxiv.org/abs/2308.08986.

[55] Michael Perregaard and Egon Balas. Generating cuts from multiple-term disjunctions. In *Integer programming and combinatorial optimization (Utrecht, 2001)*, volume 2081 of *Lecture Notes in Comput. Sci.*, pages 348–360. Springer, Berlin, 2001. ISBN 3-540-42225-0. URL https://doi.org/10.1007/3-540-45535-3_27.

[56] Ted K. Ralphs. Duality, Branch and Bound, and Parametric Valid Inequalities. Technical Report COR@L Technical Report 16T-001, Department of Industrial and Systems Engineering, Lehigh University, 2016.

[57] T.K. Ralphs and M. Güzelsoy. The SYMPHONY Callable Library for Mixed Integer Programming. In *Proceedings of the Ninth INFORMS Computing Society Conference*, pages 61–76, 2005. doi: 10.1007/0-387-23529-9_5. URL http://coral.ie.lehigh.edu/~ted/files/papers/SYMPHONY04.pdf.

[58] T.K. Ralphs and M. Güzelsoy. Duality and Warm Starting in Integer Programming. In *The Proceedings of the 2006 NSF Design, Service, and Manufacturing Grantees*

and Research Conference, 2006. URL http://coral.ie.lehigh.edu/~ted/files/papers/DMII06.pdf.

[59] Prachi Shah, Santanu S. Dey, and Marco Molinaro. Non-monotonicity of branching rules with respect to linear relaxations, 2024. URL https://arxiv.org/abs/2402.05213.

[60] Sahar Tahernejad, Ted K. Ralphs, and Scott T. DeNegre. A branch-and-cut algorithm for mixed integer bilevel linear optimization problems and its implementation. *Math. Program. Comput.*, 12(4):529–568, 2020. ISSN 1867-2949,1867-2957. URL https://doi.org/10.1007/s12532-020-00183-6.

[61] F. Wesselmann and U. H. Suhl. Implementing cutting plane management and selection techniques. Technical report, University of Paderborn, December 2012.

[62] Álinson S. Xavier, Feng Qiu, and Shabbir Ahmed. Learning to solve large-scale security-constrained unit commitment problems. *INFORMS J. Comput.*, 33(2):739–756, 2021. ISSN 1091-9856,1526-5528. URL https://doi.org/10.1287/ijoc.2020.0976.

Biography

Shannon Kelley is a doctoral candidate in the Industrial and Systems Engineering Department at Lehigh University under the advisement of Ted Ralphs. Her primary research interests include decomposition methods, warm starting, and the software development of MIP solvers. Prior to entering into candidacy for her doctorate, Shannon earned a B.S. in Industrial and Enterprise Systems Engineering from the University of Illinois at Urbana-Champaign and a M.S. in Pure Mathematics from DePaul University. During that time, she was also a Software Engineer and Operations Research Scientist at Opex Analytics and LLamasoft in Chicago, Illinois. Shannon has funded her tenure at Lehigh by developing software to design ammunition storage facilities for the U.S. Army, to create school district bus routes for Zum Services, and to improve presolve and warm-starting routines for The Optimization Firm. Upon her graduation, Shannon is moving to New York City to build out Veho's middle mile network optimization services.